5주차 · 웨이포인트 유도 — atan2 와 LOS

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

★ 이번 학기 녹화 강의 영상은 여기에 올라감 — 수업에서 놓친 부분은 영상으로 확인할 것

https://youtube.com/playlist?list=PLK_f1-krzJG8

- 매주 수업 뒤 그 주차 영상이 이 재생목록에 추가됨
- 아래 "참조 강의" 는 **지난 학기까지의 선수 과목**이고, 이 재생목록은 **이번 학기 본 수업** 녹화임

★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 **본 과목 담당 교수가 직접 강의한 것**이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

#	과목	수준	언어	영상	자료·코드
1	**제어공학** — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대	학부	한국어	재생목록	드라이브
2	**제어시스템설계** — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현	학부	한국어	재생목록	드라이브
3	**캡스톤디자인** — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지	학부	한국어	재생목록	드라이브
4	**제어공학특론** — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림	대학원	영어	재생목록	드라이브
5	**센서신호처리 및 융합** — 센서 모델, 잡음, 추정, 다중센서 융합	대학원	영어	재생목록	드라이브

MATLAB·Simulink 가 처음이라면 4주차 실습 전에 아래를 끝낼 것. 본 과목의 제어기 실습은 전부 Simulink 로 진행함. Onramp 는 무료이며 각각 몇 시간이면 끝남

도구	시작 지점
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부

★ 강의자료 저장소 — 처음 한 번만 `git clone`, 그 뒤로는 `git pull`

저장소	주소	받는 자리 (WSL)
강의자료 — 주차 문서 · Simulink 모델 · MATLAB 스크립트	**https://github.com/wkyouncnu/Capstone-Design**	`~/Capstone-Design`
ROS 2 예제 패키지 `usv_basics` — 2주차부터 쓰는 노드 코드	**https://github.com/wkyouncnu/usv_basics**	`~/capstone_ws/src/usv_basics`

언제	명령 (VS Code 의 WSL 창 터미널)	하는 일
처음 한 번	`cd ~ && git clone https://github.com/wkyouncnu/Capstone-Design.git`	강의자료 전체를 받음 (약 770 MB · 2026-10-16 실측 1분 40초)
처음 한 번	`mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src && git clone https://github.com/wkyouncnu/usv_basics.git`	예제 패키지를 받음. 이어서 `cd ~/capstone_ws && colcon build --symlink-install`
매주 수업 전	`cd ~/Capstone-Design && git pull` · `cd ~/capstone_ws/src/usv_basics && git pull`	**바뀐 파일만** 받음. 다시 clone 하지 않음
무엇이 바뀌었는지	`git log --oneline -10` · `git show --stat HEAD`	교수가 갱신한 내역을 확인

- 두 저장소 모두 공개 — 로그인 없이 받아짐
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/` 에 있음. MATLAB 은 같은 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 로 엶
- `usv_basics` 를 받은 뒤 새 노드가 생겼으면 `colcon build --symlink-install` 을 한 번 더 돌림
- 본인이 고친 파일 때문에 `git pull` 이 멈추면 `git stash` 로 치워 두고 다시 받음 → 강의자료는-한-번-받고-git-pull-로-갱신한다

- **과목**: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 주차 학습 내용**: 웨이포인트를 주고 배가 **경로를 따라가게** 만듦. 유도법칙 두 가지를 비교함

★ 시작 전 확인

- **강의자료부터 갱신**: VS Code WSL 창 터미널에서 `cd ~/Capstone-Design && git pull` — 문서와 코드·모델이 같은 판이 됨 (강의자료는-한-번-받고-git-pull-로-갱신한다, 처음 받는 법은 2주차 2-6)
- 4주차 모델(`W04_4_inner_loop.slx`)이 동작했어야 함
- MATLAB **R2024b + Simulink + ROS Toolbox**
- 배포 폴더 `W05_simulink/` 와 `W05_matlab/`

★ 이번 주차 실행 요약 — Gazebo(VRX) ↔ Simulink 는 이 세 덩어리만 치면 됨

자세한 설명은 2부 E 절. 아래는 2026-10-23 기준 노트북에서 그대로 쳐서 확인한 순서임 (완주 232.2 s · 동기 점검 k = 1.005)

① 터미널 1 — Gazebo 띄우기 (창이 뜨면 이 터미널은 닫지 않고 둠)

```
cd ~/Capstone-Design && git pull
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W03_vrx_lite
bash stop_vrx.sh
bash run_vrx.sh
```

② 터미널 2 — 토픽 확인 (`ground_truth_odometry` 가 보이면 됨)

```
ros2 topic list | grep -E "thrusters|ground_truth"
```

③ MATLAB — Simulink 를 Gazebo 에 붙여 돌리기

```
cd('\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\1_2026-2학기_강의자료\10-주차별-강의자료\W05_simulink')
setappdata(0, 'W05_skip_run', true);  W05_setup;  W05_vrx_run
```

이럴 때	할 것
Gazebo 없이 오프라인만	MATLAB 에서 `W05_setup` 한 줄
`run_vrx.sh` 가 `화면 이상` 을 찍고 멈춤 · 로그만 나오고 **Gazebo 창이 안 뜸**	Windows PowerShell 에서 `wsl --shutdown` → 터미널을 새로 열고 ① 부터 (E-1 의 경고 상자)
다음 실험	① 의 `bash stop_vrx.sh` 부터 다시 — 실험마다 VRX 를 새로 띄움

ℹ 4주차는 제어만 했음

- 4주차: "선수각을 45도로 맞춰라", "속도를 1.5 m/s 로 맞춰라"
- 그런데 **누가 45도를 정해 주는가?**
- 그것을 정하는 것이 **유도(Guidance)** 이며, 이번 주차의 학습 대상임

이 주차를 마치면 할 수 있어야 하는 것

1. 유도 · 항법 · 제어를 구분해서 설명하기
2. **선수각 ψ · 침로각 χ · 크랩각 β · 횡류각 β_r** 의 차이를 해류 삼각형으로 설명하기
3. **cross-track error** 를 계산하고, 그 식이 왜 회전행렬 하나인지 설명하기
4. **atan2 유도**와 **LOS 유도**를 각각 구현하고 차이를 수치로 제시하기
5. lookahead distance Δ 가 무엇을 바꾸는지 실험으로 보이기
6. **조류가 있을 때** 두 유도법칙이 어떻게 달라지는지 설명하기
7. 같은 제어기를 **오프라인 모델**과 **Gazebo** 양쪽에서 돌리기

준비물

항목	내용
환경	1~4주차 WSL2 + ROS 2 + VRX
MATLAB	R2024b + Simulink + ROS Toolbox
배포 파일	`W05_simulink/` (모델 2개 + 스크립트 11개: setup · plot · animate · anim_capture · compare · sweep · lookahead_demo · crab_demo · vrx_run · build · tidy_layout), `W05_matlab/` (연구실 참고 코드 — 목록은 문서 끝 "배포 파일")

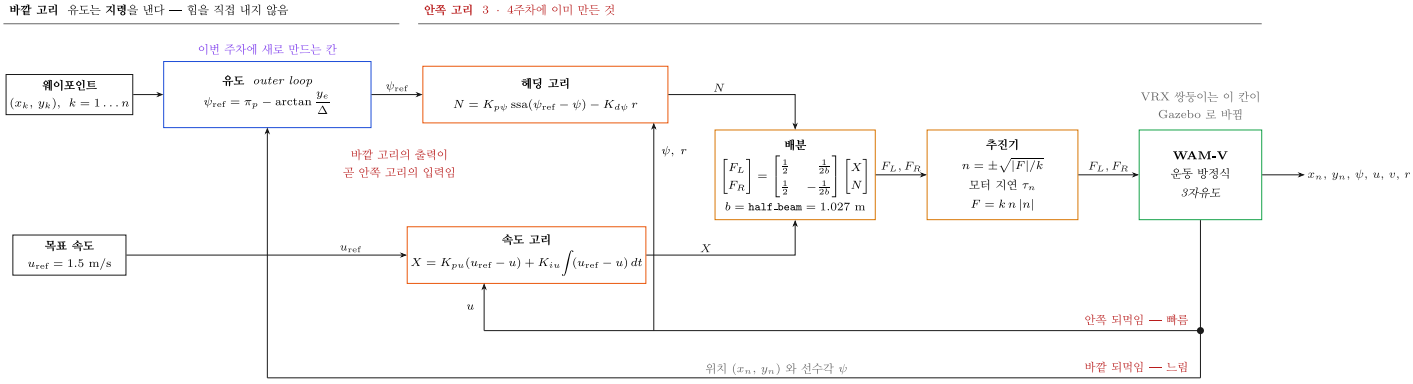
1부 · 이론

1-1. 유도 · 항법 · 제어

> ★ 자율운항은 세 덩어리로 나뉨

이름	답하는 질문	본 과목에서
항법 (Navigation)	나는 지금 어디에 있는가	4주차 ground truth odometry
유도 (Guidance)	어디로 향해야 하는가	**이번 주차**
제어 (Control)	그러려면 무엇을 얼마나 밀어야 하는가	4주차 헤딩·속도 제어

- 영문 머리글자를 따서 **GNC** 라고 부름
- 이번 주차에 작성할 모델도 이 순서 그대로 왼쪽에서 오른쪽으로 배치됨

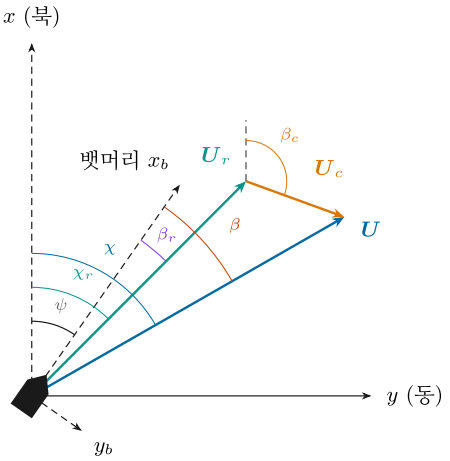


그림이 말하는 것

- **고리가 둘**임. 바깥 고리가 유도, 안쪽 고리가 제어임
- 바깥 고리(유도)가 내는 것은 **힘이 아니라 지령**임 — ψ_{ref} 와 u_{ref}
- 바깥 고리의 **출력이 그대로 안쪽 고리의 입력**임. 두 고리가 맞물리는 자리임
- 안쪽 고리(헤딩 고리 · 속도 고리 · 배분 · 추진기)는 3~4주차에 이미 만든 것임.
  이번 주차에 새로 만드는 칸은 **왼쪽 유도 하나**임
- 배분이 내는 것은 좌우 추력 $F_L,\ F_R$ 이고, **그것이 곧 추진기에 보내는 값**임 (뉴턴).
  프로펠러 법칙 $F = kn|n|$ 은 제어기가 아니라 Gazebo 플러그인 **안**에 있음.
  추진기 칸은 그 플러그인이 하는 일을 **소스와 같은 순서로 옮겨 적은 것**임 (§2-5)
- 되먹임도 둘임 — 안쪽은 제어기가 매 스텝 보는 빠른 되먹임($u,\ \psi,\ r$),
  바깥은 위치가 웨이포인트에 닿았는지만 보는 느린 되먹임($x_n,\ y_n,\ \psi$)
- VRX 쌍둥이 모델은 맨 오른쪽 **WAM-V 운동 방정식 칸이 Gazebo 로 바뀌는 것**이 전부임.
  나머지 칸은 오프라인 모델과 글자 하나까지 같음

1-2. 뱃머리가 향한 곳과 배가 가는 곳은 다르다

- 배가 실제로 움직이는 속도는 **두 개를 더한 것**임
  - 물에 대해 나아가는 속도 — 추진기가 만듦
  - 물 자체가 흐르는 속도 — 조류
- 그 합이 뱃머리에서 틀어져 있으면, 뱃머리를 아무리 잘 맞춰도 배는 **옆으로 밀린 채** 나아감



해류 삼각형

ψ 선수각 — 뱃머리가 향한 방향

χ 침로각 — 배가 **땅에 대해** 가는 방향

χ_r 대수 침로각 — 배가 **물에 대해** 가는 방향

β 크랩각 — 뱃머리와 진행 방향의 차이

β_r 횡류각 — 뱃머리와 물살 방향의 차이

β_c 조류가 흘러가는 방위 — 북에서 잰다

$\boldsymbol{U} = \boldsymbol{U}_r + \boldsymbol{U}_c$ 대지속도 = 대수속도 + 조류

$U = \sqrt{u^2 + v^2}, \quad U_r = \sqrt{u_r^2 + v_r^2}, \quad U_c = \sqrt{u_c^2 + v_c^2}$

$\chi = \psi + \beta, \quad \beta = \text{atan2}(v,\ u)$

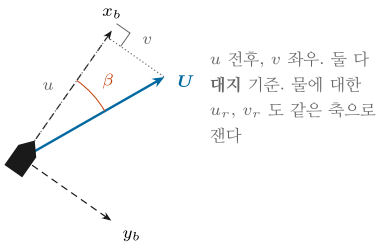
$\chi_r = \psi + \beta_r, \quad \beta_r = \arcsin(v_r / U_r)$

무엇이 무엇을 재는가 — GNSS 는 χ 와 U, 컴퍼스는 ψ, ADCP 는 조류를 잰다. β 는 재는 것이 아니라 $\chi - \psi$ 로 얻는다

조류가 없으면 $\boldsymbol{U} = \boldsymbol{U}_r$ 이고 $\beta = \beta_r$ 이 된다. 이 그림의 두 각이 갈라지는 원인은 조류 하나다

선수각 ψ 만 제어하면 배는 경로에서 옆으로 밀린 채 나아간다. 조류에서 경로 이탈이 남는 이유다

선체 축으로 본 대지속도 — u 와 v



u 전후, v 좌우. 둘 다 **대지** 기준. 물에 대한 u_r, v_r 도 같은 축으로 잰다

세 속도 — 그림의 삼각형

$$\boldsymbol{U} = \boldsymbol{U}_r + \boldsymbol{U}_c$$

벡터	이름	뜻	무엇이 재는가
$\boldsymbol{U_r}$	대수속도 (relative)	**물에 대한** 속도. 추진기가 만드는 것	대수속도계
$\boldsymbol{U_c}$	조류 (current)	**물 자체**가 흐르는 속도	ADCP, 또는 조류표
$\boldsymbol{U}$	대지속도 (over ground)	**땅에 대한** 속도. 실제로 움직이는 것	GNSS

- 삼각형은 **닫힘**. 셋 중 둘을 알면 나머지 하나가 정해짐
- 속력은 성분의 크기임. 성분은 전부 **선체 축**으로 잰 값임

$$U = \sqrt{u^2 + v^2}, \qquad U_r = \sqrt{u_r^2 + v_r^2}, \qquad U_c = \sqrt{u_c^2 + v_c^2}$$

성분	뜻
$\boldsymbol{u}$, $\boldsymbol{v}$	**대지속도**의 전후 · 좌우 성분
$\boldsymbol{u_r}$, $\boldsymbol{v_r}$	**대수속도**의 전후 · 좌우 성분
$\boldsymbol{u_c}$, $\boldsymbol{v_c}$	**조류**를 선체 축으로 옮긴 성분

- 셋의 관계도 성분별로 그대로 성립함 — $\boldsymbol{u = u_r + u_c}$, $\boldsymbol{v = v_r + v_c}$
  - 모델이 쓰는 식은 이것을 뒤집은 꼴임 (2부 B절) — $\boldsymbol{u_r = u - u_c}$, $\boldsymbol{v_r = v - v_c}$

북에서 재는 네 방위

기호	이름	북에서 어디까지 재는가	어떻게 얻는가
$\boldsymbol{\psi}$ (psi)	선수각 (heading)	**뱃머리**	컴퍼스 · 자이로가 잼
$\boldsymbol{\chi}$ (chi)	침로각 (course)	$\boldsymbol{U}$ — 배가 **땅에 대해** 가는 방향	GNSS 가 잼
$\boldsymbol{\chi_r}$	대수 침로각	$\boldsymbol{U_r}$ — 배가 **물에 대해** 가는 방향	$\boldsymbol{\psi + \beta_r}$ 로 계산
$\boldsymbol{\beta_c}$	조류 방위	조류가 **흘러가는** 쪽	설정값 (`current_direction`)

뱃머리에서 재는 두 각

기호	이름	뱃머리와 무엇 사이인가	식
β (beta)	**크랩각** (crab angle)	실제 진행 방향 $\boldsymbol{U}$	$\beta = \chi - \psi = \operatorname{atan2}(v,\ u)$
β_r	**횡류각** (sideslip angle)	물살 방향 $\boldsymbol{U_r}$	$\beta_r = \chi_r - \psi = \arcsin(v_r / U_r)$

$$\chi \ = \ \psi + \beta, \qquad \chi_r \ = \ \psi + \beta_r$$

- 두 각이 갈라지는 원인은 **조류 하나**임
  - 조류가 없으면 $\boldsymbol{U} = \boldsymbol{U_r}$ 이므로 $\beta = \beta_r$
  - 조류도 없고 옆으로 미끄러지지도 않으면 $v = 0 \Rightarrow \beta = 0 \Rightarrow \chi = \psi$
- 조류가 옆에서 밀면 $v \neq 0$ → **뱃머리는 앞을 보는데 배는 옆으로 흘러감**
- 본 과목이 제어하는 것은 ψ, 경로를 정하는 것은 χ 임. 둘의 차이가 β 임

> ⚠ **β 가 세 번 나옴 — 첨자로만 구분됨**
>
> | 기호 | 뜻 | 기준 |
> |---|---|---|
> | β | 크랩각 | 뱃머리 ψ 에서 진행 방향 χ 까지 |
> | β_r | 횡류각 | 뱃머리 ψ 에서 물살 방향 χ_r 까지 |
> | β_c | 조류 방위 | **북**에서 조류가 흘러가는 방향까지 |
>
> 앞의 둘은 **뱃머리**에서, 마지막 하나는 **북**에서 잼. 기준이 다름
> Fossen 교재는 크랩각을 β_c, 횡류각을 β, 조류 방위를 γ_c 로 적음
> 본 과목은 모델 변수 이름(`beta_c` = 조류 방위)을 따르므로 교재와 다름. 함께 볼 때 바꿔 읽을 것

> ℹ **게가 옆으로 걷는 모습에서 온 이름임**
>
> 비행기에서도 같은 일이 일어남. 옆바람 착륙에서 기수를 바람 쪽으로 틀고
> 활주로를 따라 내려오는 것을 crab 이라고 부름

왜 지금 이 얘기를 하는가

- 이번 주차에 작성할 제어기는 **선수각 ψ 를 제어**함
- 그런데 경로를 따라가려면 **침로각 χ 가 맞아야** 함
- 조류가 있으면 ψ 를 아무리 잘 맞춰도 **경로에서 옆으로 밀린 채** 나아감
- 이 사실이 이번 주차 실험의 핵심임

1-3. 가장 단순한 유도 — atan2

- 다음 웨이포인트를 **곧장 겨냥**함

$$\psi_{\text{ref}} \ = \ \operatorname{atan2}(y_{wp} - y,\ x_{wp} - x)$$

- 식 한 줄로 구현됨. 연구실 파이썬 노드(`wamv_pid_control_v2.py`)도 이 방식임
- 웨이포인트에 충분히 다가가면 다음 점으로 넘어감. **그 판정을 어떻게 하는지는 §1-8 에서 다룸**

> ✎ `atan2` **를 쓰는 이유**
>
> `atan(Δy/Δx)` 은 −90°~+90° 만 나옴. 뒤쪽으로 가야 할 때를 표현하지 못함
> `atan2` 는 두 인자의 부호를 함께 보고 −180°~+180° 전체를 줌

1-4. 그런데 그것으로 부족하다

> ⚠ **atan2 유도는 경로 정보를 사용하지 않음**
> 아는 것은 "다음 점이 저기 있다" 뿐임. 지금 경로에서 얼마나 벗어났는지 모름

- 배가 경로 옆으로 벗어나 있어도, 목표점 방향만 맞으면 그대로 감
- 결과: **비스듬히 가로질러** 목표점에 도착함
- 목표점에는 닿지만 **지나온 자리가 계획한 선이 아님**

왜 문제인가

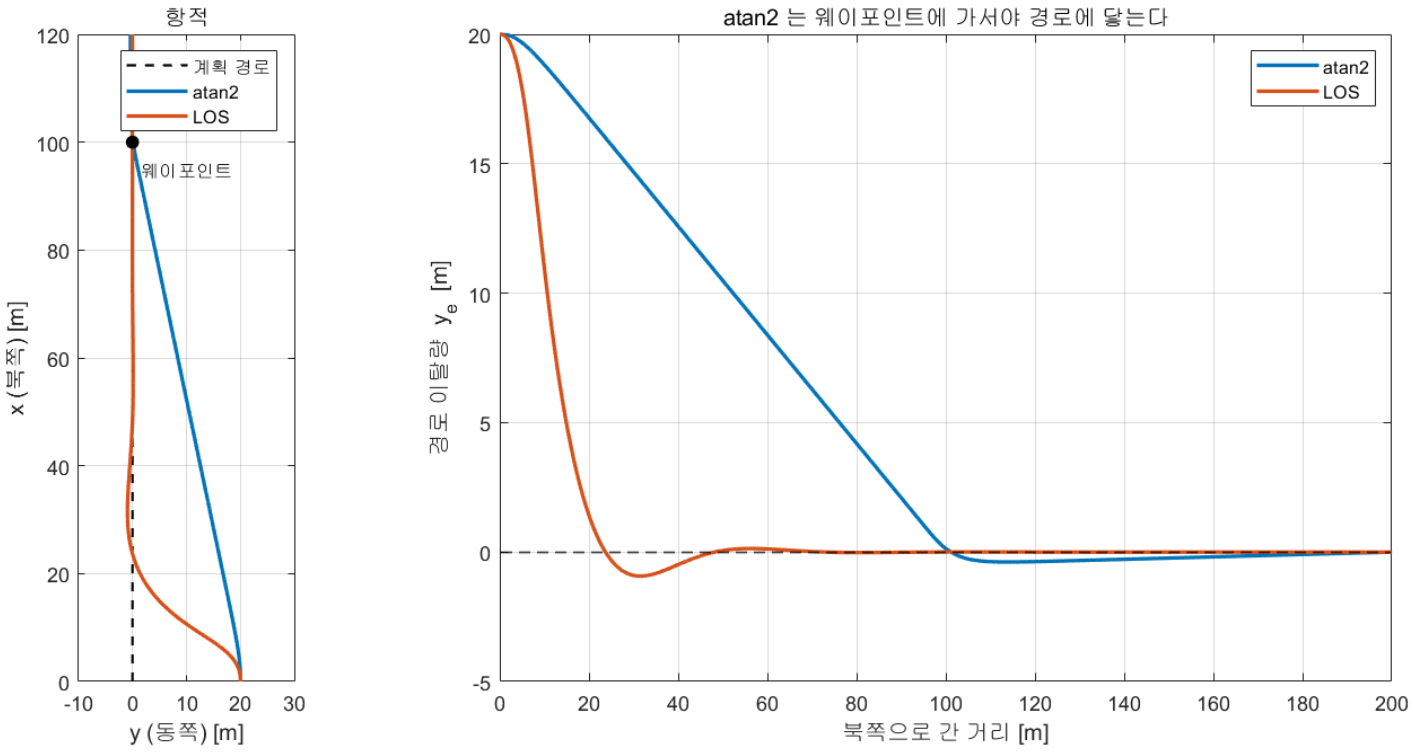
상황	경로를 벗어나면
장애물 필드 통과 (12주차 충돌회피)	계획한 안전 통로를 벗어남
좁은 수로	좌초
도킹 접근 (14주차)	접근각이 틀어져 접안 실패

실제로 재 보면 — 점을 겨냥할 때와 선을 겨냥할 때

- 말로만 두면 "조금 비스듬히 갈 뿐 아닌가" 로 읽힘. 같은 조건에서 **유도법칙만** 바꿔 재 봄

```
>> W05_setup
>> W05_aim_demo
```

- 조건: 북쪽으로 곧게 편 직선 경로 (웨이포인트 $N = 0, 100, 200, 300$ m, $E = 0$), 배는 경로에서 **동쪽 20 m** 벗어난 자리에서 출발, 조류 없음, $\Delta = 10$ m
- 사각형 경로 대신 직선 하나만 쓰는 까닭: 선회와 웨이포인트 전환이 섞이면 "점이냐 선이냐" 가 묻힘



유도	y_e @ 북쪽 50 m	@ 100 m	@ 150 m
atan2	10.49 m	0.13 m	−0.23 m
LOS	0.06 m	0.00 m	0.00 m

- 읽는 법
  - atan2 는 구간의 **절반을 왔는데도 10.49 m** 벗어나 있음. 경로에 닿는 자리가 웨이포인트(북쪽 100 m)임
  - 점까지의 거리는 내내 줄었음. 줄지 않은 것은 **선까지의 거리**임

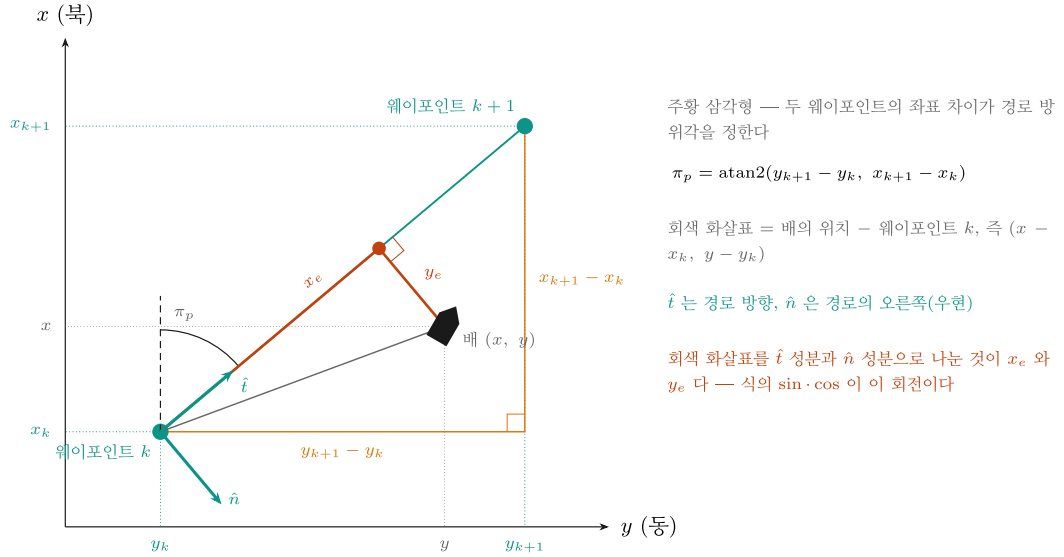
    - LOS 는 같은 자리에서 0.06 m 임. 겨냥하는 대상이 점이 아니라 선이기 때문임
- 웨이포인트를 지난 뒤에는 둘 다 경로 위를 감. 차이는 **구간 중간에서만** 드러남 — 도착점만 보면 보이지 않음

> **ℹ 이 스크립트를 돌린 뒤에는 설정이 직선 경로로 바뀌어 있음**
>
> `W05_aim_demo` 는 안에서 `W05_setup` 을 다시 부른 뒤 웨이포인트와 출발 위치를 직선 시나리오로 덮어씀
> 사각형 경로로 돌아가려면 `W05_setup` 을 다시 실행할 것

- 같은 실험이 대학원 과목「센서신호처리 및 융합」5주차 실험 5-2 에 있음. 모델이 다르므로 수치는 같지 않고, **성질이 같은지**를 봄

1-5. 두 가지 오차 — 경로 방향과 경로 이탈

- 경로를 따라간다는 말을 숫자로 바꾸려면 **오차가 두 개** 필요함. 하나로는 부족함
- 경로는 자기 좌표계를 만듦. 그 좌표계로 배의 위치를 옮겨 적으면 두 오차가 **한꺼번에** 나옴



먼저 경로 방위각

- 웨이포인트 k 에서 $k+1$ 로 가는 직선이 향한 방향. **북에서 잼**

$$\pi_p \;=\; \operatorname{atan2}(y_{k+1}-y_k,\; x_{k+1}-x_k)$$

- 한 구간 안에서는 **상수**임. 웨이포인트가 바뀔 때만 다시 계산함
- `atan2` 를 쓰는 이유는 §1-3 과 같음. 구간이 네 방향 중 어디로 가든 구분해야 함

경로가 만드는 두 방향 — $\hat{t}$ 와 $\hat{n}$ 이 왜 그 꼴인가

- $\hat{t}$ 는 경로를 따라가는 방향, $\hat{n}$ 은 경로의 **오른쪽(우현)** 임
- 둘 다 **길이가 1 인 방향벡터**임. 길이는 정보가 없고 방향만 나름

$$\hat{t} = \begin{bmatrix} \cos\pi_p \\ \sin\pi_p \end{bmatrix}, \qquad \hat{n} = \begin{bmatrix} -\sin\pi_p \\ \cos\pi_p \end{bmatrix} \qquad (\text{성분 순서는 } [\text{북}, \text{동}])$$

- 외울 식이 아님. **방위각의 정의에서 바로 나옴**

단계	내용
① 방위각의 정의	방위각은 **북에서 시계방향**으로 잰 각임 (3주차 NED)
② 그 방향의 성분	방위각이 α 인 단위벡터는 **북 성분 $\cos\alpha$, 동 성분 $\sin\alpha$** 임
③ $\hat{t}$	경로의 방위각이 π_p 이므로 $\hat{t} = (\cos\pi_p,\ \sin\pi_p)$
④ $\hat{n}$	우현은 경로에서 시계방향 $90°$ — 방위각 $\pi_p + 90°$
⑤ 삼각함수로 정리	$\cos(\pi_p{+}90°) = -\sin\pi_p$, $\sin(\pi_p{+}90°) = \cos\pi_p$

- 맞는지는 **네 방향에 넣어 보면** 바로 앎

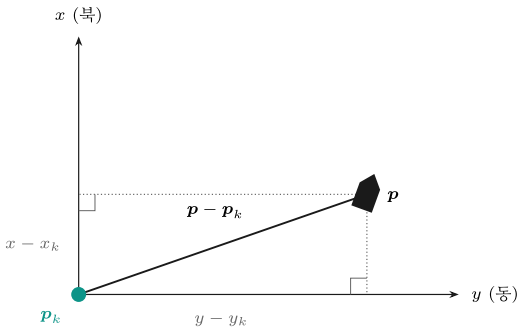
π_p	구간이 가는 쪽	$\hat{t}$ (북, 동)	$\hat{n}$ (북, 동)	$\hat{n}$ 이 가리키는 쪽
$0°$	북	$(1,\ 0)$	$(0,\ 1)$	동 — 북으로 갈 때의 우현 ○
$90°$	동	$(0,\ 1)$	$(-1,\ 0)$	남 — 동으로 갈 때의 우현 ○
$180°$	남	$(-1,\ 0)$	$(0,\ -1)$	서 — 남으로 갈 때의 우현 ○

- 두 성질이 뒤에서 전부 쓰임

$$\hat{t}^{\top}\hat{t} = \hat{n}^{\top}\hat{n} = 1 \quad (\text{길이 } 1), \qquad \hat{t}^{\top}\hat{n} = -\cos\pi_p \sin\pi_p + \sin\pi_p \cos\pi_p = 0 \quad (\text{서로 수직})$$

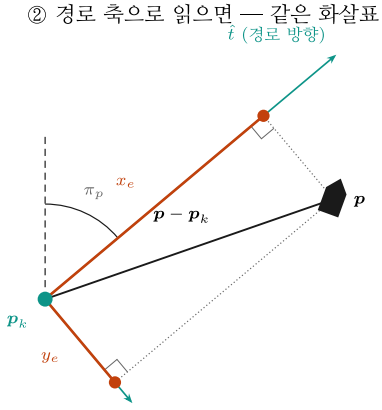
- 길이 1 이고 서로 수직 → 어떤 벡터든 이 둘로 **한 가지 방법으로만** 쪼개짐

① 북 · 동 축으로 읽으면



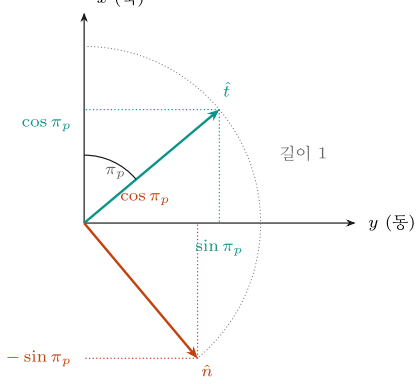
축이 북과 동이므로 성분은 좌표를 빼면 바로 나온다.
경로가 어디로 가는지는 들어 있지 않다

② 경로 축으로 읽으면 — 같은 화살표



축을 π_p 만큼 돌려 놓았을 뿐 화살표는 그대로다 — 길이도 $\sqrt{x_e^2 + y_e^2}$ 로 변하지 않는다

③ $\hat{t}$ 와 $\hat{n}$ 이 왜 그 꼴인가



성분을 읽는 일 = 회전행렬 곱하기

$\hat{t}$ 는 방위각이 π_p 인 단위벡터 — 북 성분 $\cos\pi_p$, 동 성분 $\sin\pi_p$
$\hat{n}$ 은 $\hat{t}$ 를 시계방향으로 $90°$ 돌린 것, 곧 방위각 $\pi_p + 90°$ 인 단위벡터 — $\cos(\pi_p{+}90°) = -\sin\pi_p$, $\sin(\pi_p{+}90°) = \cos\pi_p$

둘은 길이 1 이고 서로 수직이므로, 성분은 내적 하나로 나온다

$$x_e = \hat{t}^{\top}(\boldsymbol{p} - \boldsymbol{p}_k), \qquad y_e = \hat{n}^{\top}(\boldsymbol{p} - \boldsymbol{p}_k)$$

$$\begin{bmatrix} x_e \\ y_e \end{bmatrix} = \begin{bmatrix} \cos\pi_p & \sin\pi_p \\ -\sin\pi_p & \cos\pi_p \end{bmatrix} \begin{bmatrix} x - x_k \\ y - y_k \end{bmatrix} = \boldsymbol{R}(\pi_p)^{\top}(\boldsymbol{p} - \boldsymbol{p}_k)$$

$\boldsymbol{R}(\pi_p)$ 는 경로 축의 값을 북 · 동으로 되돌리는 행렬이다.
반대 방향이므로 전치가 붙는다 — 회전행렬은 $\boldsymbol{R}^{-1} = \boldsymbol{R}^{\top}$ 이라 전치만으로 역이 된다

그림 읽는 법

그림의 칸	말하는 것
① 북 · 동 축	성분이 $(\boldsymbol{x - x_k},\ \boldsymbol{y - y_k})$ — 좌표를 빼면 바로 나오지만 **경로 정보가 없음**
② 경로 축	같은 화살표의 성분이 $(\boldsymbol{x_e},\ \boldsymbol{y_e})$ — 축만 $\boldsymbol{\pi_p}$ 만큼 돌아갔음
③ 단위벡터	$\hat{\boldsymbol{t}} \cdot \hat{\boldsymbol{n}}$ 의 네 성분이 전부 $\boldsymbol{\sin\pi_p \cdot \cos\pi_p}$ 로 나오는 자리
화살표	**①과 ②에서 같은 화살표**임. 길이도 같음. 바뀐 것은 읽는 축뿐임

두 오차 — 성분을 읽는다는 것

- 단위벡터 방향의 성분은 **내적 한 번**임. 수선을 내려 그 길이를 재는 것과 같음

$$x_e = \hat{t}^{\top}(\boldsymbol{p} - \boldsymbol{p}_k), \qquad y_e = \hat{n}^{\top}(\boldsymbol{p} - \boldsymbol{p}_k) \qquad \left(\boldsymbol{p} = \begin{bmatrix} x \\ y \end{bmatrix},\ \boldsymbol{p}_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}\right)$$

- 두 내적을 풀어 쓰면 본문에서 쓰는 두 줄이 됨

$$
\begin{aligned}
x_e &= (x - x_k)\cos\pi_p \;+\; (y - y_k)\sin\pi_p \\
y_e &= -(x - x_k)\sin\pi_p \;+\; (y - y_k)\cos\pi_p
\end{aligned}
$$

- 두 줄을 한 줄로 묶으면 **행렬 하나**임

$$
\begin{bmatrix} x_e \\ y_e \end{bmatrix} = \begin{bmatrix} \hat{t}^\top \\ \hat{n}^\top \end{bmatrix} (\boldsymbol{p} - \boldsymbol{p}_k) = \underbrace{\begin{bmatrix} \cos\pi_p & \sin\pi_p \\ -\sin\pi_p & \cos\pi_p \end{bmatrix}}_{\boldsymbol{R}(\pi_p)^\top} \begin{bmatrix} x - x_k \\ y - y_k \end{bmatrix}
$$

- 식의 $\sin \cdot \cos$ 은 **회전 하나**임. 외울 것이 아니라 그림에서 읽는 것임

왜 전치가 붙는가

- 회전행렬 $\boldsymbol{R}(\pi_p)$ 의 뜻은 **경로 축의 값을 북 · 동으로 되돌리는 것**임

$$
\boldsymbol{R}(\pi_p) = \begin{bmatrix} \cos\pi_p & -\sin\pi_p \\ \sin\pi_p & \cos\pi_p \end{bmatrix} = \begin{bmatrix} \hat{t} & \hat{n} \end{bmatrix} \qquad \Rightarrow \qquad \boldsymbol{p} - \boldsymbol{p}_k = \boldsymbol{R}(\pi_p) \begin{bmatrix} x_e \\ y_e \end{bmatrix}
$$

- 열이 $\hat{t}$ 와 $\hat{n}$ 임. 경로 축의 성분에 두 방향벡터를 곱해 더하는 것이 곧 북 · 동 성분임
- 지금 필요한 것은 그 **반대 방향**(북 · 동 → 경로 축)이므로 역행렬을 곱해야 함
- 회전행렬은 열이 **서로 수직이고 길이 1** 이라 역행렬이 전치와 같음

$$
\boldsymbol{R}^{-1} = \boldsymbol{R}^\top \qquad \Rightarrow \qquad \begin{bmatrix} x_e \\ y_e \end{bmatrix} = \boldsymbol{R}(\pi_p)^\top (\boldsymbol{p} - \boldsymbol{p}_k)
$$

> ℹ **회전은 길이를 바꾸지 않음**
> $x_e^2 + y_e^2 = (x - x_k)^2 + (y - y_k)^2$ 가 언제나 성립함
> 축을 돌려 놓았을 뿐 화살표는 그대로이기 때문임. 코드를 고치고 나서 이 값이 달라졌으면
> 회전식에 부호나 인자 순서가 틀어진 것임

기호	뜻	쓰는 곳
x_e	**경로 방향 오차** — 구간을 얼마나 왔는가	웨이포인트 전환 판정 (§1-8)
y_e	**경로 이탈량** — 경로에서 얼마나 옆으로 벗어났는가	유도법칙 (§1-6)
x, y	배의 **북쪽 · 동쪽 위치** (NED). x_k, y_k 는 웨이포인트 k 의 위치. 첨자 e 가 붙은 x_e, y_e 와 구분함. N 은 요 모멘트	둘 다
π_p	경로 방위각	둘 다
d	**구간 길이** — 웨이포인트 k 와 $k+1$ 사이의 거리	웨이포인트 전환 판정 (§1-8)

- d 는 두 웨이포인트의 좌표만으로 정해짐. 배의 위치와 무관하고 구간 안에서 **상수**임

$$
d \;=\; \|\boldsymbol{p}_{k+1} - \boldsymbol{p}_k\| \;=\; \sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2}
$$

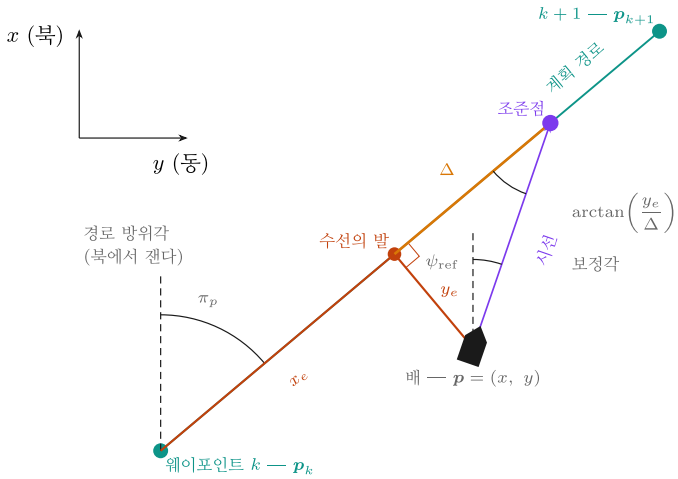
- 따라서 $d - x_e$ 가 **구간에 남은 경로 방향 거리**임. 이 값이 §1-8 의 전환 판정에 그대로 들어감
  - 구간 안에서는 $0 \le x_e \le d$ 임. 웨이포인트 k 위에서 $x_e = 0$, $k+1$ 위에서 $x_e = d$
  - $x_e > d$ 는 **$k+1$ 을 지나쳤다**는 뜻임. 이때 $d - x_e$ 는 음수가 되고, 전환 판정은 그대로 참임

> ℹ **부호 규약**
> $y_e > 0$ 이면 경로의 **오른쪽(우현)**, $y_e < 0$ 이면 왼쪽임
> NED 에서 진행 방향을 바라볼 때 오른쪽이 양수임. 4주차의 요모멘트 부호와 같은 규약임
> 그래서 우현으로 벗어나면 **좌현으로** 꺾어야 하고, §1-6 의 식이 빼기인 이유가 이것임
> y_e 는 현재 − 경로 쪽 값이라 4주차 $e = \psi_{\text{ref}} - \psi$ (목표−현재) 와 방향이 반대 — 그래서 식에 − 가 붙음

> ⚠ **확인용 값 세 개**
>
> 식이 맞는지는 특수한 위치 세 곳에 넣어 보면 바로 앎
>
> | 배의 위치 | x_e | y_e |
> |---|---|---|
> | 웨이포인트 k 위 | 0 | 0 |
> | 웨이포인트 $k+1$ 위 | d (구간 길이) | 0 |
> | 경로에서 우현으로 ε | 그대로 | $+\varepsilon$ |
>
> 셋 다 맞지 않으면 π_p 의 인자 순서나 부호가 틀린 것임

1-6. LOS 유도법칙

- 요지: **웨이포인트가 아니라 경로 위의 한 점을 겨냥함**
  - 그 점은 경로에서 배와 가장 가까운 점보다 Δ 만큼 앞선 점
- 이름의 뜻: LOS = Line Of Sight, **시선(視線)**. 눈으로 목표를 좇는 것과 같음



주어진 것

$p_k = [x_k,\ y_k]^\top$, $p_{k+1} = [x_{k+1},\ y_{k+1}]^\top$ — 미션 파일에 적힌 웨이포인트

$p = [x,\ y]^\top$ — 배의 현재 위치. 항법이 매 걸음 준다

Δ — lookahead distance. 설계자가 고르는 값 (1-7절)

매 걸음 이 순서로 계산한다

① $\pi_p = \operatorname{atan2}(y_{k+1} - y_k,\ x_{k+1} - x_k)$
구간이 바뀔 때만 다시 계산

② $y_e = -(x - x_k)\sin\pi_p + (y - y_k)\cos\pi_p$
1-5절의 회전 한 번

③ $\psi_{\text{ref}} = \pi_p - \arctan(y_e/\Delta)$
구하려는 값 — 헤딩 제어기로 보낸다

이 그림의 값 — $\pi_p = 50°$, $y_e = 6$ m, $\Delta = 10$ m

$\arctan(6/10) = 30.96° \ \Rightarrow\ \psi_{\text{ref}} = 50° - 30.96° = 19.04°$

배가 경로의 오른쪽에 있으므로 지령이 π_p 보다 왼쪽으로 나온다.
$y_e \to 0$ 이면 $\psi_{\text{ref}} \to \pi_p$ — 보정이 저절로 사라진다

그림 읽는 법

- 그림의 오른쪽이 **무엇이 주어지고 무엇을 구하는가**임

자리	내용
주어진 것	$p_k \cdot p_{k+1}$ (미션 파일), p (항법), Δ (설계자가 고름)
매 걸음 계산	① π_p → ② y_e → ③ ψ_{ref} 순서. ①은 구간이 바뀔 때만
구하려는 값	ψ_{ref} 하나. 이것만 헤딩 제어기로 나감

기호	뜻
π_p	경로 방위각. **북에서 잼**
x_e	경로 방향 오차 — 구간을 얼마나 왔는가
수선의 발	경로 위에서 배와 가장 가까운 점. 배에서 경로에 **수직**으로 내린 발
y_e	경로 이탈량. 수선의 발에서 배까지의 **부호 있는** 거리 (우현 +)
Δ	**lookahead distance** — 수선의 발에서 **경로를 따라** 얼마나 앞을 보는가
조준점	수선의 발보다 Δ 앞선 점. **언제나 경로 위에 있음**
시선	배에서 조준점으로 향하는 직선. 이 방향이 곧 지령임
ψ_{ref}	그 시선의 방위각. 헤딩 제어기로 보내는 값

식이 나오는 자리

- 경로 좌표계에서 보면 세 점의 자리가 아주 단순함

점	경로 좌표계에서의 위치
배	$(x_e,\ y_e)$
수선의 발	$(x_e,\ 0)$
조준점	$(x_e + \Delta,\ 0)$

- 배에서 조준점으로 가는 벡터는 빼기 한 번임

$$\text{조준점} - \text{배} = \begin{bmatrix} (x_e + \Delta) - x_e \\ 0 - y_e \end{bmatrix} = \begin{bmatrix} \Delta \\ -y_e \end{bmatrix}$$

- 이 벡터가 **경로에서 얼마나 틀어져 있는가**가 보정각임

$$\operatorname{atan2}(-y_e,\ \Delta) = -\arctan\left(\frac{y_e}{\Delta}\right) \qquad (\Delta > 0 \text{ 이므로})$$

- 경로 자체가 북에서 π_p 만큼 돌아가 있으므로, 둘을 더하면 북 기준 지령이 됨

$$\boxed{\psi_{\text{ref}} = \pi_p - \arctan\left(\frac{y_e}{\Delta}\right)}$$

- 두 항의 역할
  - π_p: **경로와 나란히 서라**
  - $\arctan$ 항: **벗어난 만큼 경로 쪽으로 기울여라**

그림의 숫자로 확인

- 그림은 $\pi_p = 50°$, $y_e = 6$ m, $\Delta = 10$ m 로 그렸음

$$\arctan\left(\frac{6}{10}\right) = 30.96° \qquad \Rightarrow \qquad \psi_{\text{ref}} = 50° - 30.96° = 19.04°$$

- 배가 경로의 **오른쪽**에 있으므로 지령이 π_p 보다 **왼쪽**으로 나옴. 부호가 맞음

극한 네 가지 — 이 법칙이 안전한 이유

조건	결과	뜻
$y_e \to 0$	$\psi_{\text{ref}} \to \pi_p$	경로 위에서는 경로와 나란히. 보정이 **부드럽게** 사라짐
$y_e \to +\infty$	$\psi_{\text{ref}} \to \pi_p - 90°$	우현으로 아무리 멀어도 **경로를 정면으로** 향함
$\Delta \to 0$	$\psi_{\text{ref}} \to \pi_p \mp 90°$ ($y_e > 0$ 이면 −, $y_e < 0$ 이면 +)	아무리 작은 이탈에도 최대로 꺾음 — 켜짐/꺼짐 제어가 됨
$\Delta \to \infty$	$\psi_{\text{ref}} \to \pi_p$	보정을 아예 하지 않음 — 헤딩 유지가 됨

★ arctan 이라서 안전함

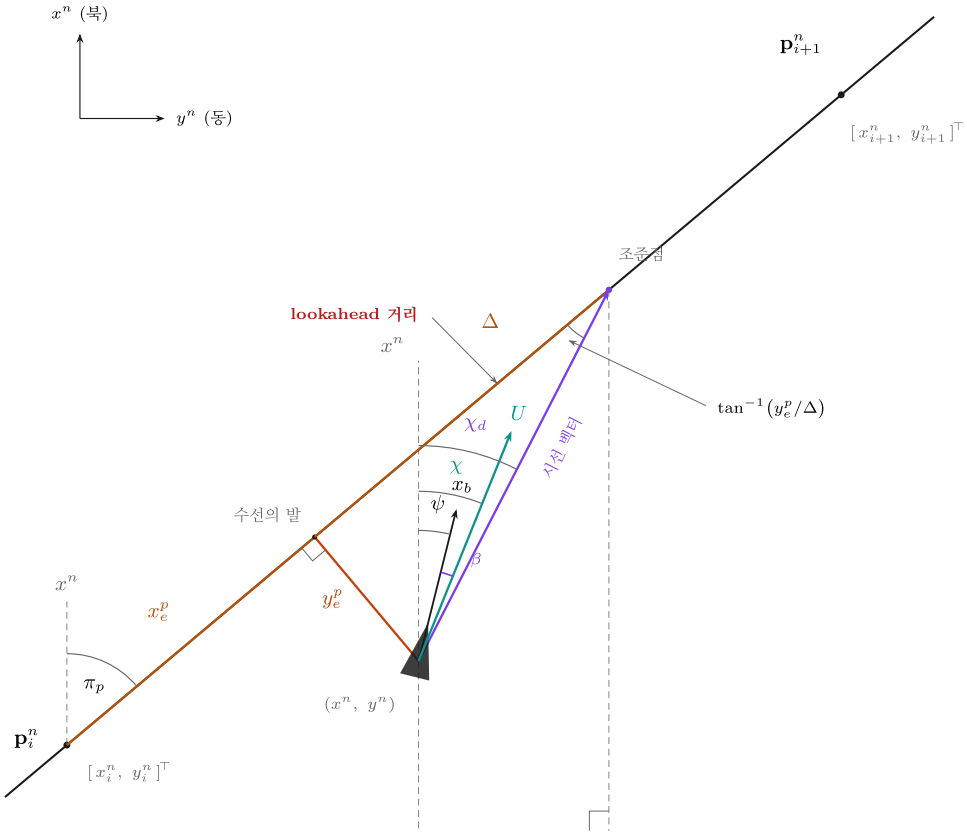
$|\arctan(\cdot)| < 90°$ 이므로 지령은 **경로 방향에서 $90°$ 를 절대 넘지 않음**
아무리 멀리 벗어난 곳에서 출발해도 경로에서 **멀어지라는 지령은 나오지 않음**
$\psi_{\text{ref}} = \pi_p - K_{p,y}\, y_e$ 처럼 비례로만 만들면 이 보장이 없음 ($K_{p,y}$ 는 횡오차 비례 이득 — Nomoto 이득 K 와 다름). y_e 가 크면
한 바퀴를 넘어가는 지령이 나옴

ı Δ 는 배가 아니라 수선의 발에서 잼

조준점은 배에서 Δ 떨어진 점이 아니라, 수선의 발에서 경로를 따라 Δ 앞선 점임
배에서 조준점까지의 실제 거리는 $\sqrt{\Delta^2 + y_e^2}$ 이고 Δ 보다 큼
배에서 Δ 를 재면 보정각이 $\arcsin(y_e/\Delta)$ 가 됨. $y_e \to 0$, $\Delta \to \infty$ 극한은 그대로지만,
$|y_e| > \Delta$ 이면 조준점이 존재하지 않아 **법칙 자체가 정의되지 않음**

교과서 표기로 다시 보기

- 위 그림은 법칙만 보이려고 기호를 줄인 것임. 교과서와 논문은 같은 그림을 **위첨자를 붙여** 그림
- 아래 그림이 그 표준 배치임 (Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2021, §12.3)
- 두 번 보는 까닭: 법칙은 위에서 읽고, **표기**는 여기서 읽음. 참고문헌을 펼쳤을 때 같은 그림으로 보이게 하려는 것임



$$\pi_p = \operatorname{atan2}\big(y_{i+1}^n - y_i^n,\ x_{i+1}^n - x_i^n\big)$$

아는 값

$$y_e^p = -\big(x^n - x_i^n\big)\sin\pi_p + \big(y^n - y_i^n\big)\cos\pi_p$$

아는 값

$$\mathbf{p}_i^n = [\,x_i^n,\ y_i^n\,]^\top, \quad \mathbf{p}_{i+1}^n = [\,x_{i+1}^n,\ y_{i+1}^n\,]^\top$$

아는 값

$$\chi_d = \pi_p - \tan^{-1}\big(K_p\, y_e^p\big), \qquad K_p = \frac{1}{\Delta}$$

구하려는 값

$$y_e^p \to 0, \qquad \chi_d \to \pi_p$$

경로 위에서는 보정이 사라지고 선박이 구간을 따라 곧게 나아감. 이 법칙이 노리는 바를 극한으로 적은 것임.

항은 둘뿐임. π_p 는 선박을 경로와 *나란히* 세우고, $-\tan^{-1}(y_e^p/\Delta)$ 는 선박을 경로 쪽으로 기울임. 그 밖에 들어가는 항이 없으므로 이 법칙이 못 하는 일도 분명함 (1-7절).

보정각은 90° 를 넘지 못함. $y_e^p \to 0$ 이면 선박이 경로 위에 있고 $\chi_d \to \pi_p$ 임. $y_e^p \to \infty$ 이면 $\chi_d \to \pi_p - 90^\circ$ 가 되어 경로를 정면으로 향함 — 경로에서 멀어지는 지령은 나오지 않음.

Δ 는 경로를 따라 잼. 선박이 아니라 수선의 발에서 재는 거리임. 선박에서 조준점까지의 실제 거리는 $\sqrt{\Delta^2 + (y_e^p)^2}$ 이고 Δ 보다 큼.

표기와 이 그림의 수치

웨이포인트는 $\mathbf{p}_i^n = [\,x_i^n,\ y_i^n\,]^\top$ 이고 $\{n\}$ 에서 읽은 값임. 오차 둘에 위첨자 p 가 붙는 까닭은 그 둘이 **경로** 좌표계 $\{p\}$ 에서 읽은 값이기 때문임 — $\{n\}$ 도 $\{b\}$ 도 아님. 위첨자가 세 좌표계를 갈라 놓음.

그림은 $\pi_p = 50^\circ$, $x_e^p = 32\,\mathrm{m}$, $y_e^p = 16\,\mathrm{m}$, $\Delta = 38\,\mathrm{m}$ 로 그린 것임. 보정각은 $\tan^{-1}(16/38) = 22.83^\circ$ 이고 $\chi_d = 27.17^\circ$ 임. 시선 거리는 $41.23\,\mathrm{m}$ 임. 선체는 $\psi = 14^\circ$ 에 크랩각 $\beta = 8^\circ$ 이므로 $\chi = 22^\circ$ 임 — 아직 지령 침로에 올라서지 못한 상태이고, 그래서 선체에 걸린 호 셋이 서로 다름.

침로각인가 선수각인가. 적힌 그대로의 법칙은 *침로각* χ_d 를 지령함. 이 침로각을 받는 것은 4주차의 선수각 제어기이므로, 실습에서는 χ_d 자리에 ψ_d 를 넣은 같은 식을 씀. 둘은 $\beta = 0$ 일 때만 정확히 같음. 그 차이는 1-2절이 정의하고, 1-7절이 경로에서 치르는 값을 잼.

달라진 것은 표기뿐임

위 그림	교과서 그림	무엇이 붙었는가
p_k, p_{k+1}	$\mathbf{p}_i^n$, $\mathbf{p}_{i+1}^n$	위첨자 n — NED 좌표계에서 읽은 값임
x_e, y_e	x_e^p, y_e^p	위첨자 p — **경로 좌표계**에서 읽은 값임
ψ_{ref}	χ_d	선수각이 아니라 **침로각** 지령임
$\arctan(y_e/\Delta)$	$\tan^{-1}(K_p\, y_e^p)$, $K_p = 1/\Delta$	같은 식을 **이득** 꼴로 적은 것임
(없음)	β	크랩각. 선수각과 침로각의 차

- 위첨자가 하는 일: 같은 글자라도 **어느 좌표계에서 읽은 값인지**를 가름

좌표계	기호	이 그림에서 읽는 값
NED	$\{n\}$	웨이포인트, 배의 위치 (x^n, y^n), 북에서 재는 모든 각
선체고정	$\{b\}$	선수 방향 x_b
경로	$\{p\}$	두 오차 x_e^p, y_e^p

선체에 걸린 호 셋

- 세 각 모두 **북에서** 재되, 재는 대상이 다름

호	기호	무엇의 각인가
안쪽	ψ	**뱃머리**가 향한 방향 (x_b 축)
가운데	χ	**배가 실제로 가는** 방향 (속도 U)
바깥쪽	χ_d	유도가 **내보낸 지령** (시선 벡터)

- $\beta = \chi - \psi$ 가 크랩각임 (§1-2). 그림은 $\psi = 14°$, $\beta = 8°$ 이므로 $\chi = 22°$ 임
- 호 셋이 서로 다른 뜻: 아직 **지령 침로에 올라서지 못한** 순간을 그린 것임. 제어기가 일하는 중임
- 세 각이 모두 같아지는 때는 조류가 없고 ($\beta = 0$) 제어가 끝난 ($\chi = \chi_d$) 뒤임

그림의 수치로 확인

- 그림은 $\pi_p = 50°$, $x_e^p = 32$ m, $y_e^p = 16$ m, $\Delta = 38$ m 로 그렸음

$$\tan^{-1}\left(\frac{16}{38}\right) = 22.83° \qquad \Rightarrow \qquad \chi_d = 50° - 22.83° = 27.17°$$

- 배에서 조준점까지의 실제 거리는 Δ 가 아님

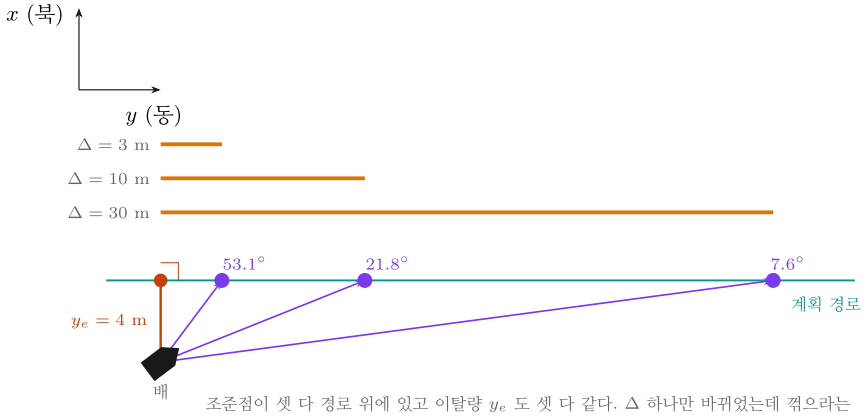
$$\sqrt{\Delta^2 + (y_e^p)^2} = \sqrt{38^2 + 16^2} = 41.23\ \text{m}$$

- $x_e^p = 32$ m 는 보정각에 들어가지 않음. 오직 **구간을 얼마나 왔는지**를 재는 값이고, 웨이포인트 전환 판정에 쓰임 (§1-8)

> ★ **침로각 χ_d 인가 선수각 ψ_{ref} 인가**
> 교과서가 적은 그대로의 법칙은 **침로각** χ_d 를 지령함. 배가 실제로 가는 방향을 직접 명령하는 것임
> 그런데 그 지령을 받는 제어기는 4주차의 **선수각** 제어기임. 그래서 실습 모델은 같은 식에 χ_d 대신 ψ_{ref} 를 넣어 씀
> 두 값은 $\beta = 0$ 일 때만 정확히 같음. 조류가 있으면 $\beta \neq 0$ 이므로 **경로 옆에 붙어서** 가게 됨 — 그 간격이 $y_e = \Delta \tan\beta$ 이고 §1-7 에서 잼

1-7. lookahead distance Δ

- LOS 법칙에서 **자유롭게 고를 수 있는 값은 Δ 하나뿐**임. 나머지는 경로와 위치가 정함
- Δ 는 분모임. 같은 이탈량이어도 Δ 가 작으면 크게 꺾음



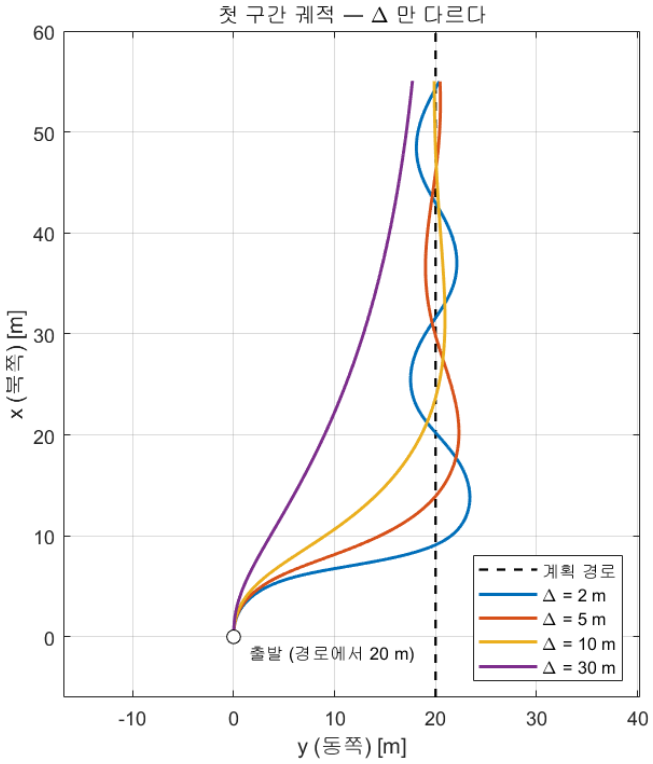
조준점이 셋 다 경로 위에 있고 이탈량 y_e 도 셋 다 같다. Δ 하나만 바뀌었는데 꺾으라는 각이 7 배 차이가 난다

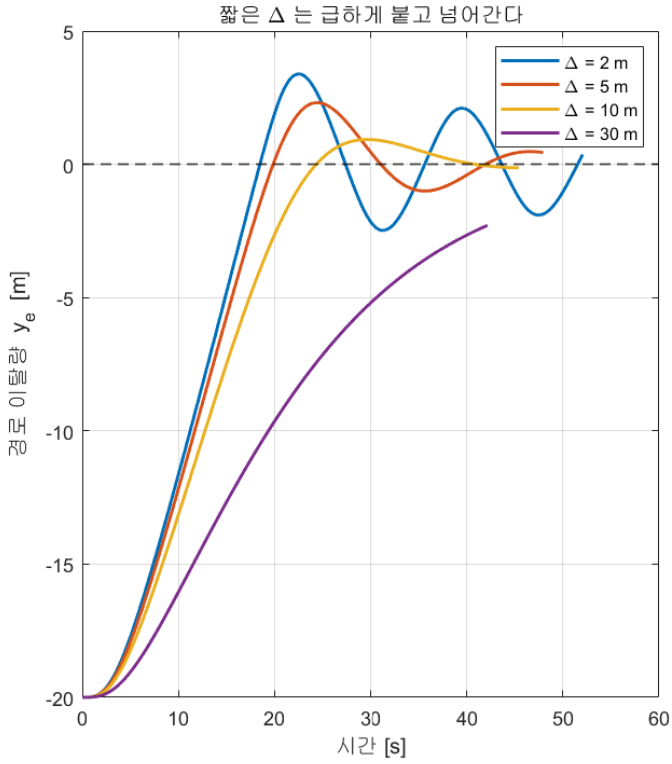
- 그림의 세 경우는 **이탈량이 똑같음.** **Δ** 만 바뀌었는데 꺾으라는 각이 **53.1°** 에서 **7.6°** 로 일곱 배 차이가 남

눈으로 보기 — 같은 자리에서 출발해 Δ 만 바꾼다

```
>> W05_setup
>> W05_lookahead_demo
```

- 네 번 모두 **같은 자리에서 같은 만큼(20 m) 벗어난 채** 출발함. **Δ** 말고는 아무것도 바꾸지 않음
- 첫 구간(북쪽 80 m)만 잘라서 그림. 구간이 바뀌면 전환과 선회가 섞여 **Δ** 의 성격이 묻힘





측정값 — 첫 구간에서의 경로 복귀 (조류 없음)

Δ [m]	$\lvert y_e \rvert < 1$ m 진입 [s]	오버슈트 [m]	경로 가로지름 [회, 첫 구간]
2	17.8	3.390	5
5	18.9	2.312	3
10	22.4	0.930	2
30	진입 못 함	0.000	0

- **Δ 가 짧을수록 급하게 꺾어 들어옴.** $\Delta = 2$ m 는 가장 빨리(**17.8** s) 경로에 닿음
- 그 대가가 오른쪽 그림에 그대로 보임. $\Delta = 2$ m 는 경로를 넘어 **반대쪽으로 3.39 m 까지** 튀었다가 돌아오고, 첫 구간 안에서만 경로를 **다섯 번** 가로지름
- $\Delta = 30$ m 는 정반대임. 한 번도 넘지 않지만(오버슈트 **0**) **첫 구간이 끝날 때까지 경로에 닿지도 못함**

- $\Delta = 10$ m 는 한 번만 살짝 넘고(0.93 m) 곧 붙음

> ★ **이 그림이 Δ 의 전부임**
>
> 짧은 Δ = **빨리 붙는 대신 넘어감.** 긴 Δ = **넘어가지 않는 대신 못 붙음**
> 둘 다 좋게 만드는 값은 없음. 그래서 Δ 는 정답을 찾는 게 아니라 **고르는** 값임
>
> 조류를 켜면 이 균형점이 **짧은 쪽으로 옮겨 감.** 아래 두 번째 표(미션 전체 기준)가 그것을 숫자로 보여 줌
>
> - `W05_lookahead_demo` 는 안에서 `W05_setup` 을 다시 부르며 워크스페이스를 지움
> - 명령 창에서 `current_speed = 0.4` 만 치면 무시됨. **`W05_setup.m` 파일에서** `current_speed = 0.4`, `current_direction = 90` 으로 고친 뒤 다시 돌릴 것

실제로 재 보면

- 본 과목의 오프라인 모델(`W05_0_offline`)로 Δ 만 바꿔 가며 미션 전체를 돌린 값임
- 웨이포인트 4점 사각형(한 변 80 m), $u_{\mathrm{ref}} = 1.5$ m/s, $R = 5$ m, 출발점은 경로에서 20 m 벗어난 곳
- 평균 이탈량은 `W05_plot` 이 찍는 **초기 60초 제외** 값 (`S.mean_ye_ss`). 전 구간 평균이 아님
- 아래 두 표는 `W05_sweep('A')` 로 그대로 다시 나옴 (2026-10-15 재실행 — 추진기 단이 추력을 그대로 통과시키게 바뀌어 모든 칸을 다시 쟀음. §2-5 를 함께 읽을 것)

조류 없음

Δ [m]	수렴 [s]	정착 후 평균 $\lvert y_e \rvert$ [m] (초기 60 s 제외)	평균 $\lvert r \rvert$ [°/s]	경로 가로지름 [회]	완주 [s]
2	17.8	1.424	9.200	22	220.9
3	18.1	1.152	5.958	17	216.7
5	18.9	0.821	3.127	16	210.4
10	22.4	**0.724**	1.917	11	206.1
20	35.6	1.015	1.570	4	203.1
30	미수렴	1.427	1.453	4	203.2

- **수렴 시각**은 Δ 와 함께 계속 늘어남. $\Delta = 30$ m 는 첫 구간이 끝날 때까지 1 m 안에 들어오지 못함
- **평균 이탈량**만 가운데가 가장 작음. $\Delta = 10$ m 가 최저점이고, 양쪽으로 가면 나빠짐
- **평균 요각속도**와 **경로 가로지름 횟수**는 Δ 와 함께 계속 줄어듦. $\Delta = 2$ m 는 $\Delta = 30$ m 보다 평균 요각속도가 **6.3 배** 크고 (그만큼 요 모멘트를 더 씀) 경로를 **5.5 배** 더 많이 가로지름 — 이것이 "지그재그" 의 정체임
- **완주 시각**도 계속 줄어듦. 흔들리며 가면 그만큼 늦게 도착함

조류 0.4 m/s (동쪽으로 흐름, 90°)

Δ [m]	수렴 [s]	정착 후 평균 $\lvert y_e \rvert$ [m] (초기 60 s 제외)	평균 $\lvert r \rvert$ [°/s]	경로 가로지름 [회]	완주 [s]
2	16.2	1.586	9.382	21	223.2
3	16.3	1.270	5.813	18	216.6
5	16.8	**1.151**	3.102	11	210.8
10	18.2	1.531	1.855	9	206.7
20	21.8	2.541	1.471	5	203.9
30	25.1	3.319	1.351	4	204.3

> ★ **조류가 있으면 최적 Δ 가 작아짐**
>
> 조류가 없을 때는 $\Delta = 10$ m 가 최저점이었는데, 조류가 생기자 $\Delta = 5$ m 로 옮겨 갔음
> 그리고 큰 Δ 일수록 오차가 훨씬 크게 늘어남 — $\Delta = 30$ m 는 3.265 m 로 최저점의 **약 2.8 배**임
> 이유는 바로 아래에서 식 하나로 나옴

조류 속에서는 경로 옆에 붙어 간다 — $y_e = \Delta \tan\beta$

- 조류가 경로를 가로질러 밀면, 배는 뱃머리를 조류 쪽으로 틀어야 경로와 나란히 갈 수 있음. 그 각이 **크랩각** β 임 (1-2절)

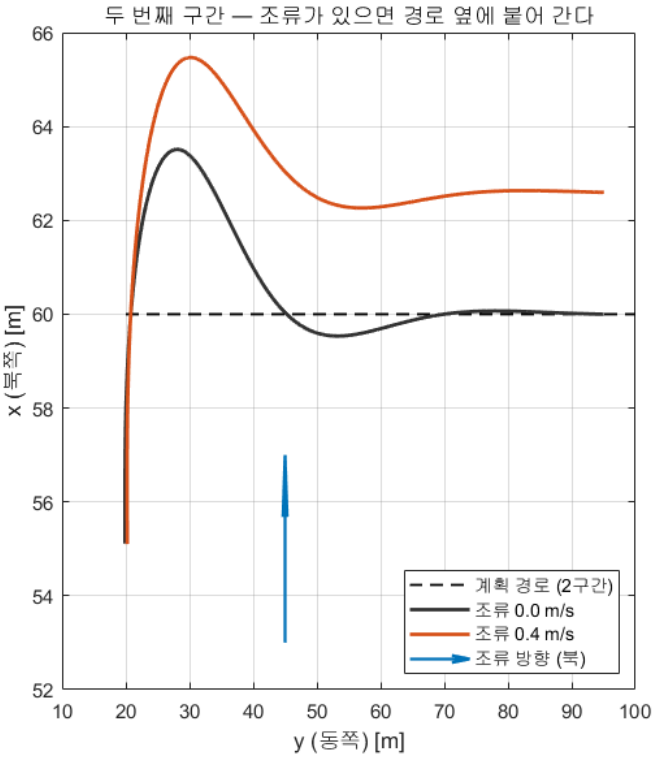
- 그런데 LOS 는 **선수각** 지령을 냄. 정상상태에서 진행 방향이 경로 방향이 되려면 $\psi = \pi_p - \beta$ 여야 하고, LOS 가 그 값을 내려면

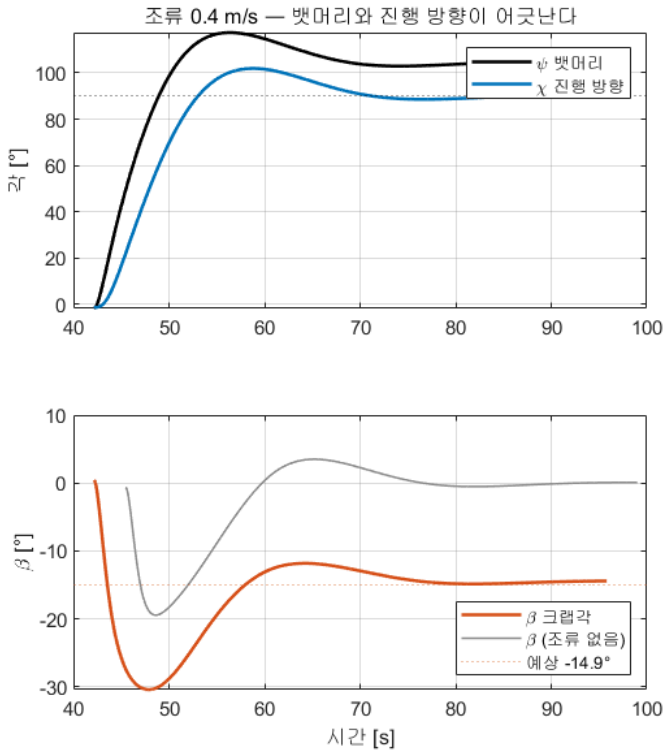
$$\pi_p - \arctan\left(\frac{y_e}{\Delta}\right) = \pi_p - \beta \quad \Longrightarrow \quad \boxed{y_e = \Delta \tan\beta}$$

- 결과: 경로에서 $\Delta \tan\beta$ 만큼 **떨어진 채 나란히** 감
  - β 는 조류가 정함
  - **Δ 가 클수록 그만큼 멀리** 떨어짐 — 위 표에서 큰 Δ 의 오차가 크게 늘어나는 이유
- 대학원 강의 W04 §4-7 과 같은 식임

```
>> W05_crab_demo
```

- 두 번째 구간(정동)에 **북쪽으로 흐르는** 조류 0.4 m/s 를 줌. 구간을 정면으로 가로지름





- 정상 출력 (구간 뒤쪽 절반의 평균)

```
조류[m/s]    psi[°]    chi[°]   beta[°]  beta 예상[°]     y_e[m] Delta*tan(b)[m]
      0.0     89.43     89.36     -0.08       -0.00      0.060          -0.013
      0.4    104.00     89.61    -14.39      -14.93     -2.505          -2.565
```

읽는 법	뜻
$\psi = 104^\circ$, $\chi = 89.6^\circ$	뱃머리는 남쪽으로 14° 틀었는데, 배는 거의 정동(90°)으로 감
$\beta = -14.39^\circ$	$\text{atan}(0.4/1.5) = 14.93^\circ$ 에 가까움. 옆으로 밀리는 속도가 전부 조류에서 옴
$y_e = -2.505$ m	경로에서 북쪽(좌현)으로 2.5 m 떨어져 나란히 감. 궤적 그림의 주황 선
$\Delta\tan\beta = -2.565$ m	식이 예측한 값. 측정과 **약 2 %** 차이로 맞음

> ✎ **이 오차를 없애려면**
>
> 원인이 "선수각을 지령했다" 는 데 있으므로, 크랩각만큼 미리 틀어 주면 됨 — $\psi_{\text{ref}} = \chi_d - \beta$
> 7주차 1-8절이 로이터링에서 이것을 해서 반경 오차를 42 % 줄임. β 를 모를 때 적분으로 지우는 방법(ILOS)은 대학원 강의 W04 §4-8 에 있음

정리

Δ	빨리 붙는가	요 모멘트를 적게 쓰는가	조류에 강한가
작음 (2~3 m)	O	X — 지그재그	△ — 정상 오프셋 $\Delta\tan\beta$ 는 작지만 지그재그로 평균 이탈은 큼 (조류 0.4 m/s, $\Delta=2$ m 에서 1.586 m)
적당 (5~10 m)	보통	보통	보통
큼 (20~30 m)	X	O	X — 옆 오차가 크게 남음

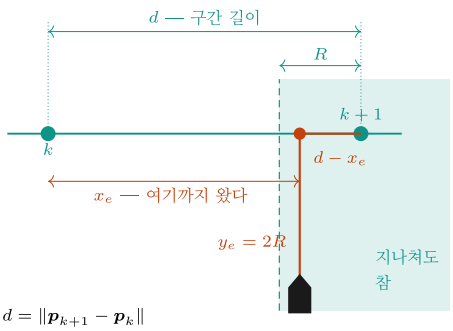
- 어느 값도 세 가지를 다 잘하지 못함. **그래서 이것은 정답이 있는 튜닝이 아니라 맞바꿈(trade-off)임**
- 흔한 기준은 **배 길이의 2~5 배**임. WAM-V 는 약 4.9 m 이므로 10~25 m
- 본 과목의 기본값은 $\Delta = 10$ m 임. 위 표의 첫 번째 조건에서 최저점이고, 기준 범위의 아래쪽 끝임

> ✎ **더 나아가기 — Δ 를 고정하지 않는 방법**
>
> 멀리 있을 때는 작게(공격적으로), 가까이 붙으면 크게(부드럽게) 만드는 방법이 있음
>
> $$\Delta(y_e) = (\Delta_{\max} - \Delta_{\min})e^{-\gamma y_e^2} + \Delta_{\min}$$
>
> 참고 자료(`2. USV 제어기 설계` , 2025)의 측정값으로는 고정 Δ 가 1.6763 / 1.9824 / 2.3202 m ($\Delta = 1.5$ / 6 / 8 m) 인 조건에서 가변 Δ 가 1.5875 m 로 **셋 모두보다 좋았음**
> 본 과목 모델에는 넣지 않았음. 과제 ②의 확장 주제로 쓸 수 있음

1-8. 웨이포인트 전환 — 언제 다음 구간으로 넘기는가

- 구간은 끝나야 함. 무엇을 보고 끝났다고 판단할지가 남았고, **널리 쓰이는 방법이 두 가지인데 둘은 같지 않음**

경로 방향 판정
남은 경로 방향 거리가 R 보다 작은가

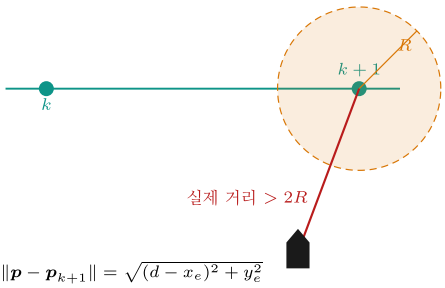


$d = \|\boldsymbol{p}_{k+1} - \boldsymbol{p}_k\|$

$d - x_e < R$ 이므로

전환한다

수락 원 판정
실제 직선거리가 R 보다 작은가



$\|\boldsymbol{p} - \boldsymbol{p}_{k+1}\| = \sqrt{(d-x_e)^2 + y_e^2}$

원 밖이므로

전환하지 않는다 — 계속 기다린다

경로 방향 판정: $d - x_e < R$ 수락 원 판정: $\|\mathbf{p} - \mathbf{p}_{k+1}\| \le R$

기호	뜻	값
d	**지금 구간의 길이** — 웨이포인트 k 와 $k+1$ 사이의 거리	m — 본 과목은 네 구간 모두 80 m
x_e	경로 방향 오차 (§1-5) — 구간을 얼마나 왔는가	m
$d - x_e$	구간에 **남은** 경로 방향 거리	m
R	수락반경 (MSS 원문 R_{switch})	m — `R_LOS` , 기본값 5

d 는 어디서 오는가

- 배의 위치와 **무관**함. 지금 구간의 두 웨이포인트 좌표만으로 정해지는 **상수**임

$$d = \|\boldsymbol{p}_{k+1} - \boldsymbol{p}_k\| = \sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2}$$

- 웨이포인트가 바뀔 때만 다시 계산함. π_p 와 같은 걸음에서 갱신됨
- 본 과목의 미션은 `W05_setup.m` 의 두 줄이 전부임

```
wp_north = [ -20   60   60  -20  -20 ];   % 웨이포인트 x (북쪽) [m]
wp_east  = [  20   20  100  100   20 ];   % 웨이포인트 y (동쪽) [m]
```

- 네 구간의 d 를 직접 계산하면 모두 80 m 임

구간	$p_k \to p_{k+1}$	$x_{k+1}-x_k$	$y_{k+1}-y_k$	d	π_p
1	$(-20,\ 20) \to (60,\ 20)$	$+80$	0	$\sqrt{80^2+0^2}=80$ m	$0°$ (북)
2	$(60,\ 20) \to (60,\ 100)$	0	$+80$	$\sqrt{0^2+80^2}=80$ m	$90°$ (동)
3	$(60,\ 100) \to (-20,\ 100)$	-80	0	80 m	$180°$ (남)
4	$(-20,\ 100) \to (-20,\ 20)$	0	-80	80 m	$270°$ (서)

- 한 변 80 m 짜리 **정사각형**임. 네 구간의 길이가 같은 것은 이 미션이 그렇게 생겼기 때문이지 d 가 언제나 같다는 뜻이 아님. 웨이포인트를 바꾸면 구간마다 달라짐
- d 가 바뀌면 **전환 시점이 바뀜**. 짧은 구간에서 R 이 d 에 비해 크면 구간이 시작하자마자 전환 조건이 만족될 수 있음 — R 을 고르는 기준은 이 절 뒤의 "R 은 얼마로 두는가" 에 있음

둘은 언제 갈라지는가

- **경로 위에 있으면 둘은 같음.** $y_e = 0$ 이면 남은 거리가 곧 직선거리이기 때문임
- 벗어나 있으면 다름. 피타고라스로 바로 나옴

$$\|\mathbf{p}-\mathbf{p}_{k+1}\| \;=\; \sqrt{(d-x_e)^2+y_e^2} \;\geq\; d-x_e$$

- 실제 거리는 **남은 거리보다 작아질 수 없음**
  - 따라서 **수락 원 판정이 언제나 더 엄격함**. y_e 가 클수록 차이가 커짐
- 그림의 배는 남은 거리가 R 보다 작은데 이탈량이 $2R$ 임. 경로 방향 판정은 넘어가고, 수락 원 판정은 넘어가지 않음

실제로 재 보면 — 수락 원은 걸릴 수 있다

- 같은 모델, 같은 미션에서 두 판정을 바꿔 가며 돌린 값임. 조류는 **동쪽으로 흐름** (90°)
- 재현: `W05_sweep('B')` (이 표), `W05_sweep('C')` (아래 여덟 방향)

R [m]	판정	조류 [m/s]	정착 후 평균 $\lvert y_e \rvert$ [m] (초기 60 s 제외)	완주 [s]	전환 횟수
5	경로 방향	0.0	0.724	206.1	3
5	수락 원	0.0	0.724	206.1	3
5	경로 방향	0.4	1.531	206.7	3
5	수락 원	0.4	1.555	207.4	3
2	경로 방향	0.4	1.725	213.7	3
2	**수락 원**	**0.4**	2.582	**미완주**	**0**
2	경로 방향	0.8	2.701	211.2	3
2	**수락 원**	**0.8**	4.807	**미완주**	**0**

- 조류가 없으면 두 판정이 **같은 값**을 냄 (둘 다 0.724 m, 완주 시각도 같음). 조류가 없으면 배가 웨이포인트를 거의 정확히 지나가므로 두 판정이 같은 걸음에서 걸림
- R 이 넉넉하면(5 m) 조류가 있어도 둘 다 통과함
- R 을 2 m 로 줄이고 조류를 주면 **수락 원 판정은 첫 구간 끝의 WP2 (60, 20) 에서 전환하지 못하고 미션이 끝나지 않음**
- 조류 방향을 45° 씩 여덟 방향으로 돌려 봐도($R=2$ m, 0.8 m/s) **여덟 번 모두** 수락 원은 미완주, 경로 방향은 모두 완주했음 (200.8 ~ 211.2 s — 가장 빠른 쪽이 225°)

> ⚠ **수락 원이 왜 걸리는가**
>
> 조류에 밀린 배는 웨이포인트 **옆을 스쳐 지나감.** 옆으로 y_e 만큼 벗어난 채
> 지나가면 그 점까지의 최소 거리가 y_e 보다 작아질 수 없음. $y_e > R$ 이면
> **원 안에 들어가는 순간이 영영 오지 않음**
>
> 그 뒤 배가 어떻게 되는지는 유도법칙이 정함
>
> - LOS 지령 $\pi_p - \arctan(y_e/\Delta)$ 에는 x_e 가 없음 → 웨이포인트를 지나쳐도 되돌아가지 않음
> - 배는 **구간의 연장선을 따라 계속 북쪽으로** 감 (VRX 라면 $x = 60$ m 너머는 육지)
> - 표의 미완주 행 평균 이탈량 2.582 m 가 정상 오프셋 $\Delta \tan\beta$ 수준인 것도 선을 계속 따라가고 있다는 증거
> - 지나친 웨이포인트로 되돌아가려는 것은 웨이포인트를 겨냥하는 **atan2 유도**일 때뿐
>
> 경로 방향 판정에는 이 문제가 없음. 옆으로 얼마나 벗어났든 **구간을 다 달렸으면**
> 넘어가기 때문임. MSS 툴박스의 원본 `GNC/LOSchi.m` · `ILOSpsi.m` 도 이 판정($d - x_e < R$)을 씀
> 배포 폴더 `W05_matlab/LOSchi.m` 은 **연구실 수정본**이라 이 부분이 수락 원 판정으로 바뀌어 있음 (원본 판정은 주석으로 남아 있음)
>
> 본 과목의 기본값도 `sw_mode = 1` **(경로 방향)** 임. 실습 D-3 에서 직접 확인함

R 은 얼마로 두는가

- R 은 튜닝 값이지만 **넘으면 안 되는 한계**가 하나 있음

$$R < \min_k d_k$$

- R 이 가장 짧은 구간보다 클 때
  - 그 구간이 시작되는 **순간에 이미** 전환 조건이 만족됨
  - 결과: 한 걸음도 달리지 않고 웨이포인트를 건너뜀
- 본 과목은 가장 짧은 구간이 80 m 이고 $R = 5$ m 이므로 여유가 큼

R 을 바꿔 가며 잰 값 ($\Delta = 10$ m, 조류 없음)

R [m]	모서리 컷 [m]	완주 [s]
2	0.08	213.5
5	0.82	206.1
10	3.19	195.8
20	7.71	179.6

- "모서리 컷" 은 배가 모서리 웨이포인트(WP2 · WP3 · WP4)에 가장 가까이 간 거리이고, 표는 세 모서리 중 **가장 크게 자른 값**임. 재현: `W05_sweep('D')`
  - 세 모서리를 따로 보면 0.69 · 0.82 · 0.80 m 로, 세 번 모두 비슷하게 자름
- R 이 크면 **미리 돌아** 모서리를 잘라먹고, 그만큼 일찍 도착함
- R 이 작으면 구간 끝까지 달렸다가 급하게 돎. 모서리는 정확하지만 회전이 급해짐
- **어느 쪽이 맞는지는 미션이 정함.** 모서리 정확도가 중요한가, 급회전을 피하는 것이 중요한가

미션 종료

- 마지막 구간에는 넘어갈 다음 웨이포인트가 없음
  - 아무 처리도 하지 않으면 배는 **마지막 구간의 연장선을 따라 계속 감**
- 본 과목의 처리: `gate` 출력
  - 마지막 웨이포인트에서 전환 조건이 만족되면 `wp_idx` 를 n ("완주")으로 올리고 `gate = 0` → 속도 지령 0
  - `wp_idx` 는 한 스텝 늦게 되돌아와 다음 계산의 `idx_prev` 가 되므로 **한 번 n 이 되면 계속 n** — 조류에 밀려 결승선 뒤로 돌아가도 다시 출발하지 않음 (래치)
- **유도법칙이 아니라 미션의 몫임**
  - 제자리 유지 · 주위 선회 · 추진기 정지 중 무엇을 할지는 임무가 정함
  - 8주차에서 Stateflow 가 이 판단을 맡음

1-9. 수식과 코드 — 한 줄씩 맞춰 보기

- 지금까지의 식이 모델 안에서 어떤 코드가 되는지 **한 줄씩** 대응시킨 표임
- 코드는 `Guidance` 서브시스템 안의 MATLAB Function 블록 `GuidanceLaw` 에 그대로 들어 있음
- 실습 B 에서 블록을 열어 이 표와 나란히 놓고 볼 것

하는 일	수식	코드
지금 구간의 양 끝	$\mathbf{p}_k,\ \mathbf{p}_{k+1}$	`xk = wp_x(k); yk = wp_y(k);` `xk1 = wp_x(k+1); yk1 = wp_y(k+1);`
경로 방위각	$\pi_p = \operatorname{atan2}(y_{k+1}-y_k,\ x_{k+1}-x_k)$	`pi_p = atan2(yk1 - yk, xk1 - xk);`
경로 방향 오차	$x_e = (x-x_k)\cos\pi_p + (y-y_k)\sin\pi_p$	`x_e = (x_n - xk)*cos(pi_p) + (y_n - yk)*sin(pi_p);`
경로 이탈량	$y_e = -(x-x_k)\sin\pi_p + (y-y_k)\cos\pi_p$	`y_e = -(x_n - xk)*sin(pi_p) + (y_n - yk)*cos(pi_p);`
atan2 유도 (§1-3)	$\psi_{\text{ref}} = \operatorname{atan2}(y_{k+1}-y,\ x_{k+1}-x)$	`psi_ref = atan2(yk1 - y_n, xk1 - x_n);`
LOS 유도 (§1-6)	$\psi_{\text{ref}} = \pi_p - \arctan\left(\frac{y_e}{\Delta}\right)$	`psi_ref = pi_p - atan(y_e / Delta);`
구간 길이	$d = \lVert\mathbf{p}_{k+1}-\mathbf{p}_k\rVert$	`d = sqrt((xk1 - xk)^2 + (yk1 - yk)^2);`
전환 — 경로 방향	$d - x_e < R$	`reached = (d - x_e) < R;`
전환 — 수락 원	$\lVert\mathbf{p}-\mathbf{p}_{k+1}\rVert \le R$	`reached = sqrt((xk1 - x_n)^2 + (yk1 - y_n)^2) <= R;`
인덱스 올리기	$k \leftarrow k+1$	`elseif reached && (k < n-1), wp_idx = k + 1;`
미션 종료	마지막 구간에서 도달 ⇒ 정지, **한 번 끝나면 계속 정지**	`elseif reached && (k == n-1), wp_idx = n;` · `if idx_prev >= n, wp_idx = n;` · `gate = double(wp_idx < n);`

- 기호와 코드 이름의 대응: x ↔ `x_n`, y ↔ `y_n` (배의 북쪽 · 동쪽 위치), x_k ↔ `xk`, y_k ↔ `yk`, x_{k+1} ↔ `xk1`, y_{k+1} ↔ `yk1`
- 코드의 `n` = 웨이포인트 개수. 구간은 `n-1` 개이므로 마지막 구간의 번호가 `k = n-1` 임

> **★ 코드에는 위첨자를 쓸 수 없음**
> 교재는 오차가 **경로 좌표계**의 값임을 x_e^p, y_e^p 처럼 위첨자로 밝힘
> MATLAB 식별자에는 위첨자를 붙일 자리가 없어서 코드는 `x_e`, `y_e` 로만 씀
> **모델 안의 `y_e` 는 언제나 경로 좌표계의 값이지 동쪽 방향 오차가 아님**
> 이 둘을 섞는 것이 이 주차에서 가장 흔한 착각이고, 두 값은 π_p 만큼 다름

MSS 툴박스와 변수 이름이 조금 다름

MSS 원본 `GNC/LOSchi.m` · `ILOSpsi.m` · `crosstrackWpt.m` 과 맞춰 읽을 때 아래 표로 바꿔 읽음

뜻	MSS 원본	본 과목 (`W05_setup` · `GuidanceLaw`)
경로 방위각	`pi_h`	`pi_p`
경로 방향 오차 · 경로 이탈량	`x_e` · `y_e`	`x_e` · `y_e` (같음)
lookahead distance	`Delta_h`	`Delta`
전환 거리 (수락반경)	`R_switch`	`R_LOS` (설정 파일), `R` (블록 안)

- 식의 형태(회전 두 줄, $\pi - \arctan(y_e/\Delta)$)는 MSS 와 같음
- 배포 폴더 `W05_matlab/LOSchi.m` 은 연구실 수정본임 — `pi_p` · `Delta` · `R_switch` 를 쓰고, 전환 판정이 수락 원임 (§1-8)

2부 · 실습

★ 두 모델의 관계를 먼저 이해할 것

모델	운동모델	조류	Gazebo
`W05_0_offline.slx`	Simulink 안의 3자유도 방정식	**가능**	불필요
`W05_1_vrx.slx`	**Gazebo 의 WAM-V**	불가	필요

유도부·내부루프·추진기·게인은 두 모델이 완전히 같음. 운동모델만 바뀜

실행하면 이런 결과가 나온다

★ 아래는 기준 환경에서 실제로 받은 출력임

값이 크게 다르면 설정 파일을 먼저 확인함. 소수점 마지막 자리는 달라질 수 있음

```
cd('<배포 폴더>/W05_simulink')
W05_setup
```

- `W05_setup` 이 설정 후 오프라인 모델을 **바로 실행하고 그림까지 그림** (`auto_run = 1`). `sim` 을 따로 부를 필요 없음
- 정상 출력

```
W05 설정 완료
  웨이포인트 5개, 유도 = LOS
  Delta = 10 m, R = 5 m, u_ref = 1.5 m/s
  전환 판정 = 경로 방향 (d - x_e < R)
  조류 = 0 m/s @ 0 deg  (오프라인 모델에서만 적용)

오프라인 모델 실행 중  (auto_run = 0 으로 두면 실행하지 않는다)
  실시간 그림 켜짐 — 새 창이 열린다. 끄려면 animate = 0

===== LOS 유도 / 조류 0 m/s =====
  평균 |y_e|              1.996 m   (전 구간)
  평균 |y_e|              0.724 m   (초기 60초 제외)  <- 과제에 쓸 값
  최대 |y_e|             20.000 m
  완주 시각               206.1 s
  평균 |크랩각|            2.95 deg
  정상상태 속도           1.500 m/s  (목표 1.50)
  왕복 검산 max|F_err|   8.53e-14 N    (F -> n -> F, 0 이어야 한다)
```

읽는 법	뜻
전 구간 1.996 m vs 초기 제외 0.724 m	출발 직후 경로에 붙는 동안의 큰 오차가 평균을 끌어올림. **과제에는 초기 제외 값**을 씀
최대 20.000 m	첫 구간 시작점이 경로에서 20 m 떨어져 있음. 오차가 아니라 초기 조건임
평균 크랩각 2.95°	조류가 0인데도 0이 아님. 선회 중 생기는 sway 때문임
정상상태 속도 1.500	목표와 일치. 속도 루프가 정상이라는 뜻
왕복 검산 8.53e-14 N	추진기 칸에 옮겨 적은 프로펠러 법칙 두 식이 서로의 역함수라는 뜻임 (§2-5)

⚠ 창이 하나 더 열리면서 배가 움직이는 그림이 나옴

`animate = 1` 이라서 그러함. 느리면 `W05_setup` 안에서 `animate = 0` 으로 끔

A. 설정 파일 열어 보기 (10분)

```
cd('<배포 폴더>/W05_simulink')
edit W05_setup.m
```

- 이 파일 하나만 고치면 됨. Simulink 의 Constant 블록들이 여기서 만든 **변수 이름을 그대로 참조**함
- 연구실의 기존 모델(`VRX_tilt4_controller_full.slx`)도 똑같은 방식임

학생이 바꾸는 값

```
wp_north = [ -20   60   60  -20  -20 ];   % 웨이포인트 x (북쪽) [m]
wp_east  = [  20   20  100  100   20 ];   % 웨이포인트 y (동쪽) [m]

guidance_mode = 2;      % 1 = atan2,  2 = LOS
Delta         = 10;     % lookahead distance [m]
R_LOS         = 5;      % 수락반경 [m]
sw_mode       = 1;      % 전환 판정. 1 = 경로 방향,  2 = 수락 원
u_ref         = 1.5;    % 목표 속도 [m/s]

current_speed     = 0.0;   % 조류 [m/s]   (오프라인 모델에서만)
current_direction = 0.0;   % 조류 방향 [deg]

Kp_psi = 400;  Kd_psi = 200;   % 헤딩 P-D  (wn 0.78, zeta 0.98 — 4주차 2-9절)
Kp_u   = 300;  Ki_u   = 100;   % 속도 PI
F_max  = 500;                  % 추력 포화 [N]
```

> ℹ **왜 배가 경로 위에서 출발하지 않는가**
>
> 배는 `(0, 0)` 에서 출발하는데 첫 구간은 $y = 20$ m (동쪽 20 m) 선임. **20 m 벗어난 채로 시작**함
> 일부러 그렇게 두었음. 경로 위에서 출발하면 두 유도법칙의 차이가 거의 보이지 않음

> ✕ **좌표 범위를 크게 벗어나면 배가 육지에 얹힘**
>
> `sydney_regatta` 에서 스폰 기준으로 확인된 수역은 대략
> $x \in [-28, 60]$, $y \in [-18, 100]$ 임
> (VRX `wayfinding_task` 의 공식 웨이포인트 3점을 환산해 얻은 값)

실행 — 이것만 하면 된다

```
W05_setup
```

> ★ **설정 파일을 실행하면 곧바로 시뮬레이션이 돎**
>
> 모델을 따로 열고 Run 을 누를 필요가 없음
> 설정만 바꾸고 직접 Run 하고 싶으면 `auto_run = 0` 으로 둠

- 정상 출력

```
W05 설정 완료
  웨이포인트 5개, 유도 = LOS
  Delta = 10 m, R = 5 m, u_ref = 1.5 m/s
  전환 판정 = 경로 방향 (d - x_e < R)
  조류 = 0 m/s @ 0 deg  (오프라인 모델에서만 적용)

오프라인 모델 실행 중  (auto_run = 0 으로 두면 실행하지 않는다)
  실시간 그림 켜짐 — 새 창이 열린다. 끄려면 animate = 0

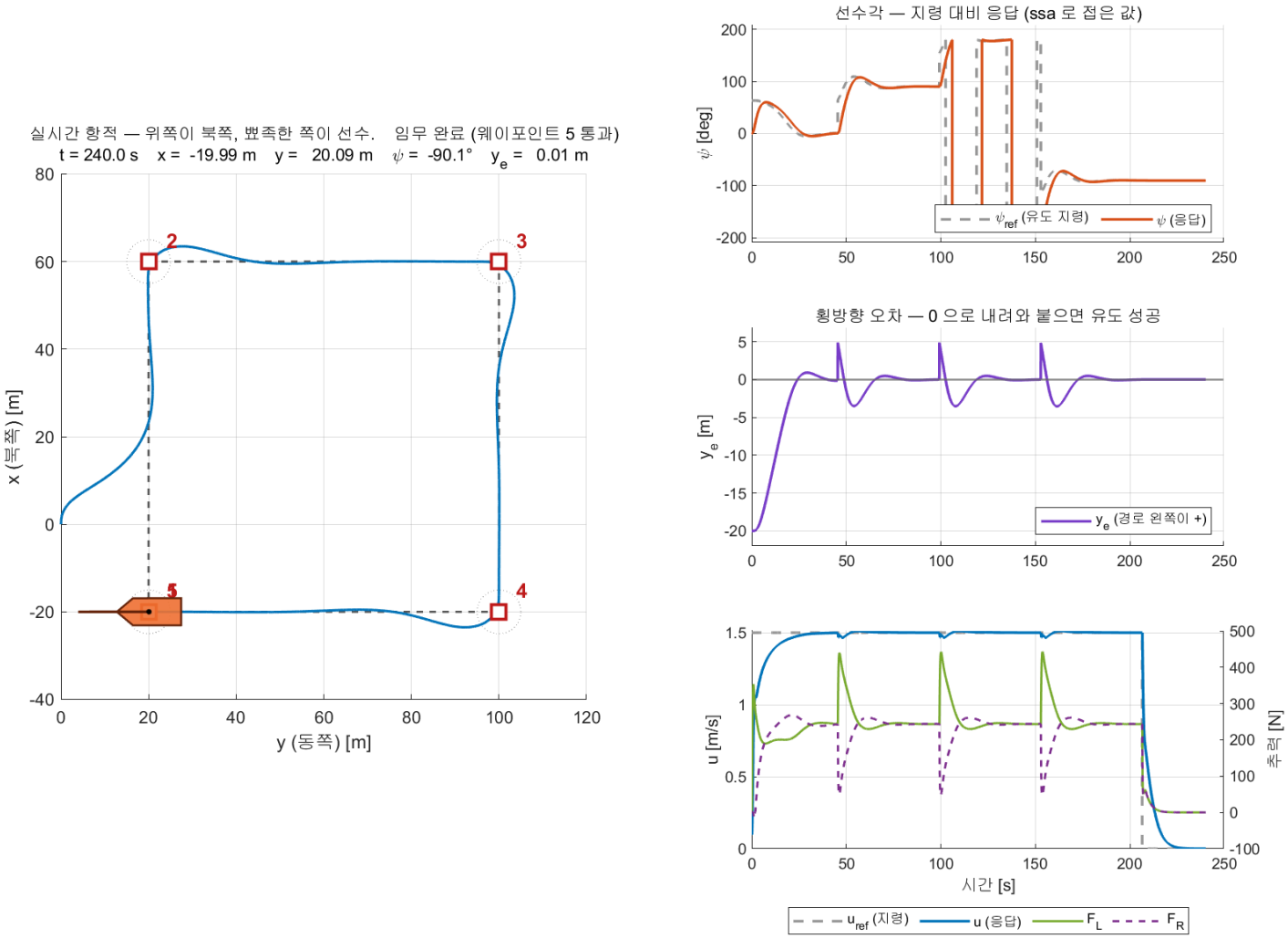
===== LOS 유도 / 조류 0 m/s =====
  평균 |y_e|              1.996 m   (전 구간)
  평균 |y_e|              0.724 m   (초기 60초 제외)  <- 과제에 쓸 값
  최대 |y_e|             20.000 m
  완주 시각               206.1 s
  평균 |크랩각|            2.95 deg
  정상상태 속도           1.500 m/s  (목표 1.50)
  왕복 검산 max|F_err|   8.53e-14 N    (F -> n -> F, 0 이어야 한다)
```

- 전체 3초 정도 걸리고 그림 창 3개가 뜸

창	내용
W05 실시간 유도 화면	도는 동안 항적과 유도 지표를 네 칸으로 보여 줌
W05 (성능)	궤적 · 이탈량 · 헤딩 · 크랩각 · 속도 5장
W05 추진기	추력 · 프로펠러 회전수 · 왕복 검산 오차

실시간 화면 — 네 칸의 뜻

- 4주차 화면이 답한 질문은 "시킨 대로 따라갔는가" 였음 (4주차 2-0-1)
- 5주차가 답해야 하는 질문은 하나 더 있음 — **경로를 따라가고 있는가**
- 그래서 4주차와 같은 네 칸 배치를 쓰되 오른쪽 칸의 내용이 5주차 것으로 바뀜



- 조건 — `W05_0_offline`, LOS 유도, $\Delta = 10$ m, $R = 5$ m, $u_{ref} = 1.5$ m/s, $K_{p,\psi} = 400 \cdot K_{d,\psi} = 200$, $K_{p,u} = 300 \cdot K_{i,u} = 100$, 조류 0, 240 s 주행 (2026-09-24 실측)

자리	그리는 것	왜 그 자리에 있는가
왼쪽 큰 칸	항적 + **웨이포인트 · 수락반경** + 선체 모양과 선수 방향	궤적선만으로는 뱃머리가 어디를 보는지 안 보임. 초록 원이 **지금 겨누는 웨이포인트**임
오른쪽 ①	ψ_{ref} (회색 점선) 와 ψ (주황) 를 **겹쳐서** (deg, `ssa`)	유도가 낸 지령을 헤딩 루프가 따라가는지 한 칸에서 읽힘
오른쪽 ②	y_e **와 0 선**	5주차의 핵심 지표. 항적만 보면 "대충 맞는 것 같다" 에서 멈추고 몇 미터인지 읽히지 않음
오른쪽 ③	왼쪽 축에 $u_{ref} \cdot u$, 오른쪽 축에 $F_L \cdot F_R$	그 응답을 무엇이 만들었는지

- 가로축은 `y (동쪽)`, 세로축은 `x (북쪽)` — **위쪽이 북쪽**이며 해도와 같은 방향임 (3주차 1-5)
- ③ 이 `yyaxis` 로 축을 둘로 나눈 이유는 자릿수임. u 는 1.x m/s, F 는 수백 N 이라 한 축에 겹치면 u 가 직선으로 보임
- ③ 의 지령선은 설정값 u_{ref} 가 아니라 $u_{ref} \times \mathbf{gate}$ 임. 완주하면 `gate` 가 0 이 되어 지령이 0 으로 떨어짐 (그림에서 207 s 부근)

어디를 보면 유도가 되고 있는가

볼 곳	정상이면	아니면
② 의 y_e	출발 직후 −20 m 에서 **0 으로 올라와 붙음.** 웨이포인트를 돌 때만 ± 5 m 안쪽으로 튀었다 다시 0	0 으로 안 붙으면 유도가 아니라 헤딩 루프를 의심 (① 을 볼 것)
① 의 두 선	응답선이 지령선에 **붙어서 같이 감**	붙지 않으면 $K_{p,\psi}$ 가 작거나 추력이 포화임
① 의 세로선	$\pm 180^\circ$ 이음매를 지날 때만 생김 — `ssa` 로 접은 값이라 그럼. 고장이 아님	—
왼쪽 칸의 초록 원	수락반경에 닿을 때마다 **다음 웨이포인트로 옮겨 감**	한 점에 머물러 있으면 전환 판정이 걸린 것 (C 절 `sw_mode` 참고)
③ 의 F_L · F_R	직선 구간에서 **같은 값으로 모임** — 요 모멘트가 0	계속 벌어져 있으면 아직 도는 중

설정 (`W05_setup.m`)	뜻
`animate = 1`	0 으로 두면 그림을 그리지 않아 훨씬 빨리 돎
`animate_every = 0.25`	몇 초마다 다시 그릴지. 키우면 빨라짐
`boat_scale = 3`	선체를 몇 배로 크게 그릴지. 1 이면 실제 크기(4.9 m)

- 선체 모양은 `W05_matlab/shipModel.m` 과 같음 — **선수는 세모, 선미는 네모**
- 빨간 사각형이 웨이포인트, 점선 원이 수락반경 R

> ★ **오프라인 모델과 VRX 쌍둥이가 화면 함수 하나(`W05_animate.m`)를 같이 씀**
>
> 갈아 끼우는 것은 신호를 어디서 뽑았는지뿐임 — 운동방정식이냐, 참값 오도메트리냐
> 따라서 위 그림과 E-4 의 VRX 화면은 **같은 함수가 그린 같은 배치**이며 나란히 놓고 읽을 수 있음

> ℹ **모델 안에서는 포트 없는 회색 상자 `Animate` 하나임**
>
> 신호는 `From` 태그로 받으므로 **제어 신호 사슬에 선이 한 줄도 늘지 않음**
> 받는 태그는 `x_n` · `y_n` · `psi` · `psi_ref` · `y_e` · `u` · `gate` · `FL` · `FR` · `wp_idx` 열 개이며, 전부 **이미 있던 태그**임
> MATLAB Function 블록은 코드 생성 대상이라 기본적으로 그래픽 출력을 지원하지 않음.
> `coder.extrinsic` 으로 선언하면 **컴파일하지 않고 MATLAB 함수를 그대로 부름**

```
coder.extrinsic('W05_animate');
```

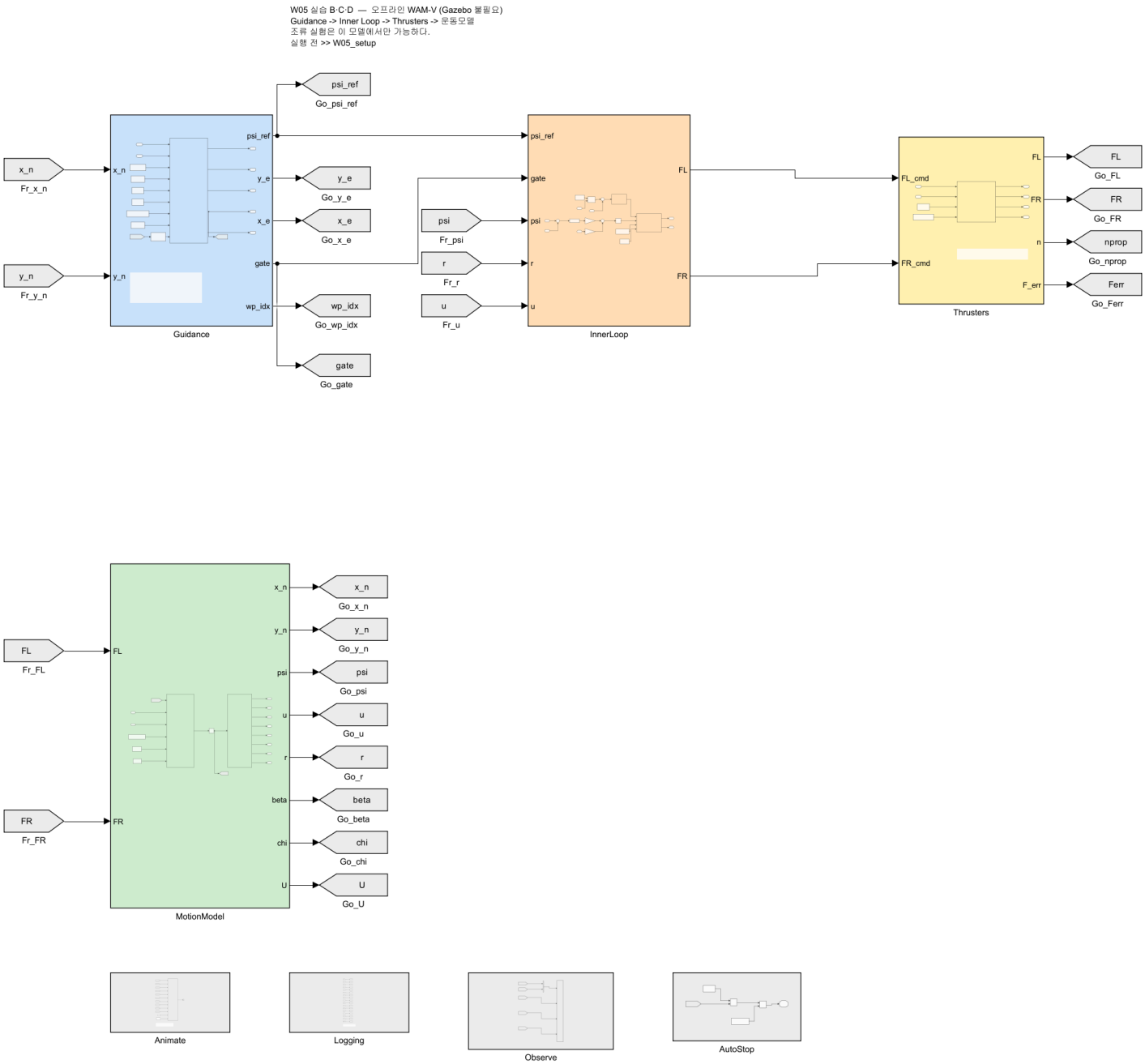
> ✎ **이 화면을 그대로 PNG 로 남기려면**

```
W05_anim_capture('offline')            % img/W05_anim_offline.png
W05_anim_capture('vrx')                % img/W05_anim_vrx.png   (VRX 기동 필요)
W05_anim_capture('offline','Stop',120) % 120초에서 끊어 찍기
```

조건(게인 · 유도 설정 · 정지 시간 · 페이싱)을 구조체로 함께 돌려주므로 그림과 조건이 어긋나지 않음

B. 오프라인 모델 — 구조 읽기 (20분)

```
open_system('W05_0_offline')
```



왼쪽에서 오른쪽으로 읽는다

단	블록	하는 일
1	**Guidance**	위치와 웨이포인트 → `psi_ref`, `y_e`, `x_e`, `gate`
2	**InnerLoop**	헤딩 P-D + 속도 PI + 추력배분 → 좌·우 추력 지령
3	**Thrusters**	Gazebo 플러그인이 하는 일 — 추력 포화 → 회전수 환산 → 왕복 검산
4	**MotionModel**	운동방정식 + 적분 + 상태 산출
—	**Animate**	도는 동안 배와 웨이포인트를 그림 (제어에는 관여 안 함)

- 되먹임은 **Goto/From 태그**로 보냄. 화면을 가로지르는 선이 없음
- 웨이포인트·게인 같은 **설정값은 각 상자 안에** 있음. 최상위에는 신호만 보임
- `Logging` 과 `Animate` 는 **입출력 포트가 없는 상자**. 안에서 From 태그로 받음
- 그래프용 로깅은 `Logging` 상자 안에 모여 있음 (더블클릭해서 열어 볼 것)

블록 색으로 역할을 구분한다

> ℹ **5~8주차의 모든 모델이 같은 색 규칙을 따름**

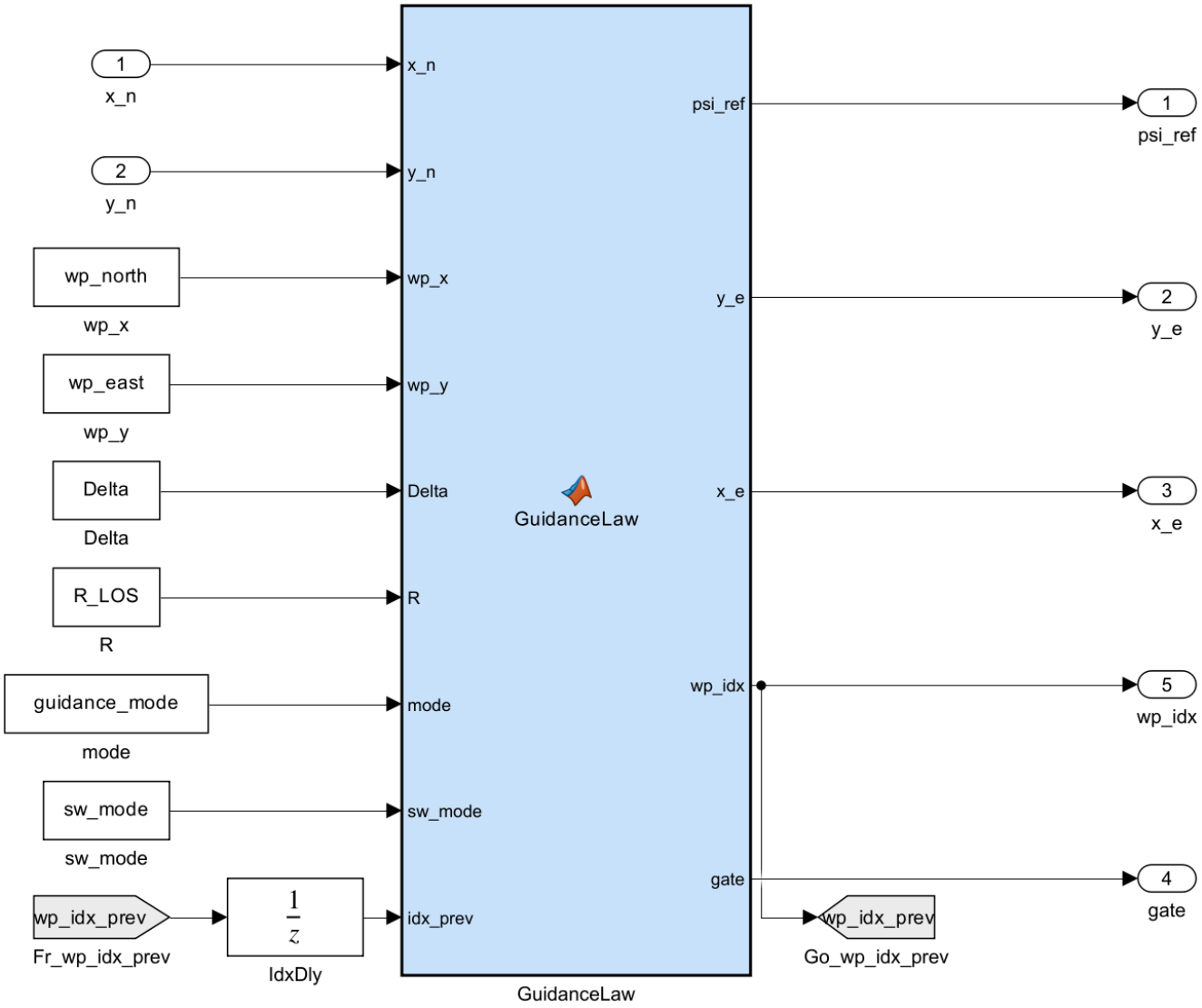
색	역할	예
파랑	유도 (Guidance)	`Guidance`, `WpManager`, `LoiterVF`
보라	미션 판단	`MissionFSM`, `ModeSwitch`, `TurnCount`
주황	제어 (Control)	`InnerLoop`, PID, 추력배분
노랑	추진기	`Thrusters`
초록	운동모델 (Plant)	`MotionModel`
연보라	ROS 통신	`PoseSubscriber`, `CmdPublisher`
회색	로깅·표시	`Logging`, `Animate`, Goto/From 태그
흰색	설정값	Constant

- 선은 **직선 또는 직각**만 씀. 대각선 없음
- 되먹임처럼 뒤로 돌아가는 선만 꺾임
- 배치를 흐트러뜨렸으면 아래 한 줄로 되돌림 (생성 스크립트가 배치까지 다시 만듦)

```
W05_setup
build_w05_models
```

Guidance 블록

- 이 블록 하나가 §1-5 부터 §1-8 까지를 전부 담고 있음. **§1-9 의 표를 옆에 놓고 열어 볼 것**
- 출력은 네 개임 — `psi_ref` (지령), `y_e` (이탈량), `x_e` (경로 방향 오차), `gate` (미션 종료)



유도 — 위치를 받아 목표 선수각을 낸다.
mode = 1 atan2 : 다음 웨이포인트를 곧장 겨눈다
mode = 2 LOS : 두 웨이포인트를 잇는 직선을 따라간다

전환 판정 sw_mode
1 경로 방향 : d - x_e < R (MSS 기본값. 밀려나도 멈추지 않는다)
2 수락 원 : 다음 웨이포인트까지 직선거리 <= R

wp_idx 는 밖으로 나가지 않는다. 지금 몇 번째 구간을 달리는지를
유도부가 스스로 기억하는 값이라, 한 스텝 지연을 거쳐 자기에게 되돌아온다.
이 지연이 없으면 대수 루프가 되어 모델이 컴파일되지 않는다.

두 오차 — 두 유도법칙이 같은 식을 씀

```
pi_p = atan2(yk1 - yk, xk1 - xk);                        % 경로 방위각
x_e  =  (x_n - xk)*cos(pi_p) + (y_n - yk)*sin(pi_p);   % 경로 방향 오차
y_e  = -(x_n - xk)*sin(pi_p) + (y_n - yk)*cos(pi_p);   % 경로 이탈량
```

- `y_e` 를 **두 모드 모두 같은 식으로** 계산함 → 그래야 같은 잣대로 비교할 수 있음

유도법칙 — `mode` 로 고름

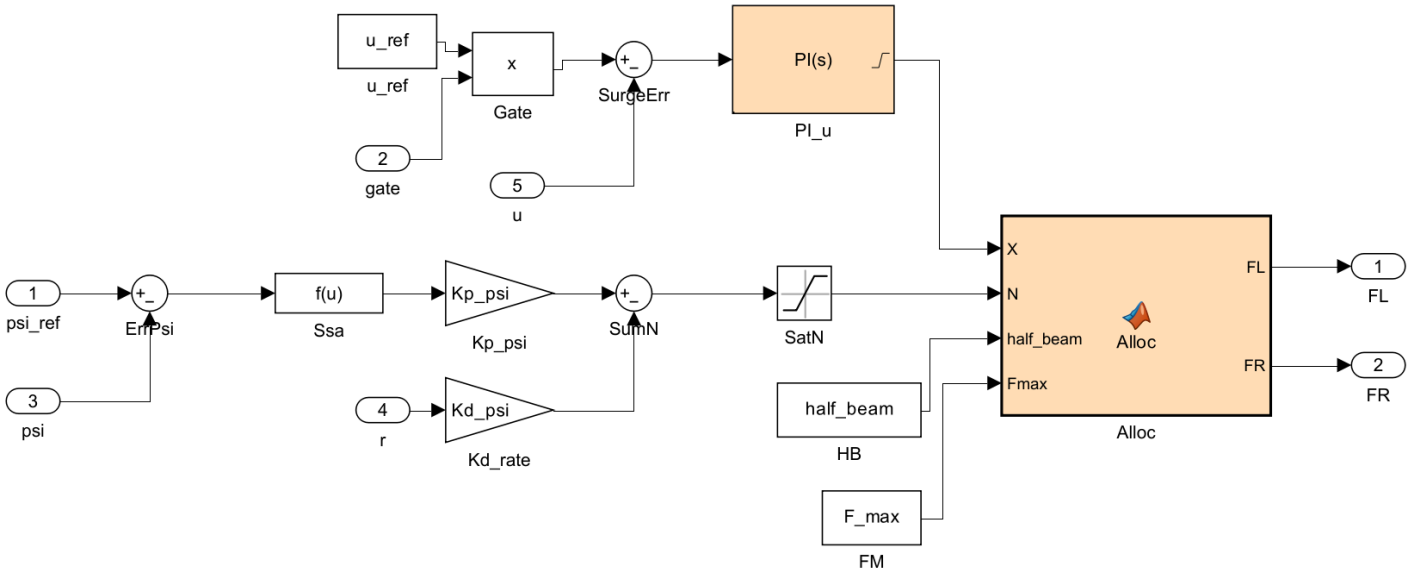
```
if mode == 1
    psi_ref = atan2(yk1 - y_n, xk1 - x_n);    % atan2 유도
else
    psi_ref = pi_p - atan(y_e / Delta);       % LOS 유도
end
```

전환 판정 — `sw_mode` 로 고름

```
d = sqrt((xk1 - xk)^2 + (yk1 - yk)^2);       % 구간 길이
if sw_mode == 1
    reached = (d - x_e) < R;                              % 경로 방향 (기본값)
else
    reached = sqrt((xk1 - x_n)^2 + (yk1 - y_n)^2) <= R;   % 수락 원
end
```

- 웨이포인트 번호는 **Unit Delay** 로 되먹임함 (한 스텝 전 값을 기억). 이 지연이 없으면 대수 루프가 되어 컴파일되지 않음

InnerLoop 서브시스템



헤딩 제어 — P-D 요각속도 되먹임

$$\tau_N \;=\; K_{p,\psi}\,\mathrm{ssa}(\psi_{\mathrm{ref}} - \psi) \;-\; K_{d,\psi}\,r$$

> ★ **오차를 미분하지 않음**
>
> - 4주차 2-7절의 `HeadingCtrl` 과 **같은 식**임. 여기서는 지령이 계속 바뀜
> - 웨이포인트가 바뀌는 순간 `psi_ref` 가 껑충 뛈 → 오차 미분값이 폭발함 (**미분 킥**)
> - 대신 **요각속도 `r` 을 직접 되먹임**함. `r` 은 튀지 않음
> - MSS 툴박스의 Otter 데모가 쓰는 방식이 이것임

- 두 방식을 견주는 실습은 **4주차 2-8절**에 있음 ([[W04_Simulink_PID와_첫_제어기#2-8. 헤딩의 D — 오차가 아니라 r 에서 얻는다|2-8]], 스크립트 `W04_heading_sign`). 거기서 세 가지 D 항을 나란히 놓고 재 봄
- 5주차 모델에는 **요각속도 되먹임 한 가지만** 둠. 고를 스위치가 없음

> ℹ **전에는 `d_mode` 로 고를 수 있었음 — 2026-10-01 에 뺌**
>
> 오차를 미분하는 갈래와 고르는 스위치를 지웠음. 같은 비교가 4주차 2-8절에 이미 있어 설명이 겹쳤고, 스위치가 남아 있으면 **도면만 보고는 어느 쪽이 도는지 알 수 없었음**. 지우기 전에 재 둔 값은 아래와 같음 — 한 스텝당 추력 변화 최대값
>
> | D 항 | Ts = 0.05 | Ts = 0.20 |
> |---|---|---|
> | 요각속도 $-K_{d,\psi} r$ | 51 N | 175 N |
> | 오차 미분 $+K_{d,\psi} \dot{e}$ | **85 N** | **232 N** |
>
> 추종 성능은 둘이 비슷했음. 차이는 **추력 명령이 얼마나 거칠어지는가**에서 났고, 주기를 네 배로 늘리면 그 차이가 함께 커졌음. 실제 추진기에는 이 거친 명령이 그대로 부담이 됨. 지금 모델로는 이 표를 다시 잴 수 없음 (갈래가 없음)

속도 제어 — surge velocity 되먹임 PI (연속 시간)

$$\tau_X = K_{p,u}\, e_u + K_{i,u} \int e_u \, dt, \qquad e_u = u_{\mathrm{ref}} \cdot \mathrm{gate} - u$$

> ℹ **적분기는 $1/s$ 임 — 이산 PI 가 아님 (2026-10-01)**
>
> `PI_u` 블록은 **Continuous-time** 으로 두었고 안티와인드업은 역계산(back-calculation)임 — 되감기 이득 `Kb` $= K_{i,u}/K_{p,u} = 0.333$ 1/s (`W05_setup.m` 의 `Kb`).
> $F_{max} = 500$ N 에서는 포화 시간이 0 % 라 고정(clamping)과 **같은 숫자**가 나옴 (완주 206.1 s · 정착 후 평균 $|y_e|$ 0.724 m, 2026-10-15 재측정).
> 전에는 같은 블록이 Discrete-time 이었는데, 그 근거로 적혀 있던 "VRX 라서 이산" 은 이유가 아니었음 — 솔버를 고정스텝 **연속** 솔버 `ode4` 로 바꾸면 같은 스텝(`Ts_ctrl` = 0.05 s)으로 돎.
> 오프라인·VRX 모델 **둘 다** `ode4` 임

- PI 이득은 $K_{p,u} = 300$, $K_{i,u} = 100$ (코드 `Kp_u` , `Ki_u`). 헤딩 이득은 $K_{p,\psi} = 400$, $K_{d,\psi} = 200$ (코드 `Kp_psi` , `Kd_psi`)

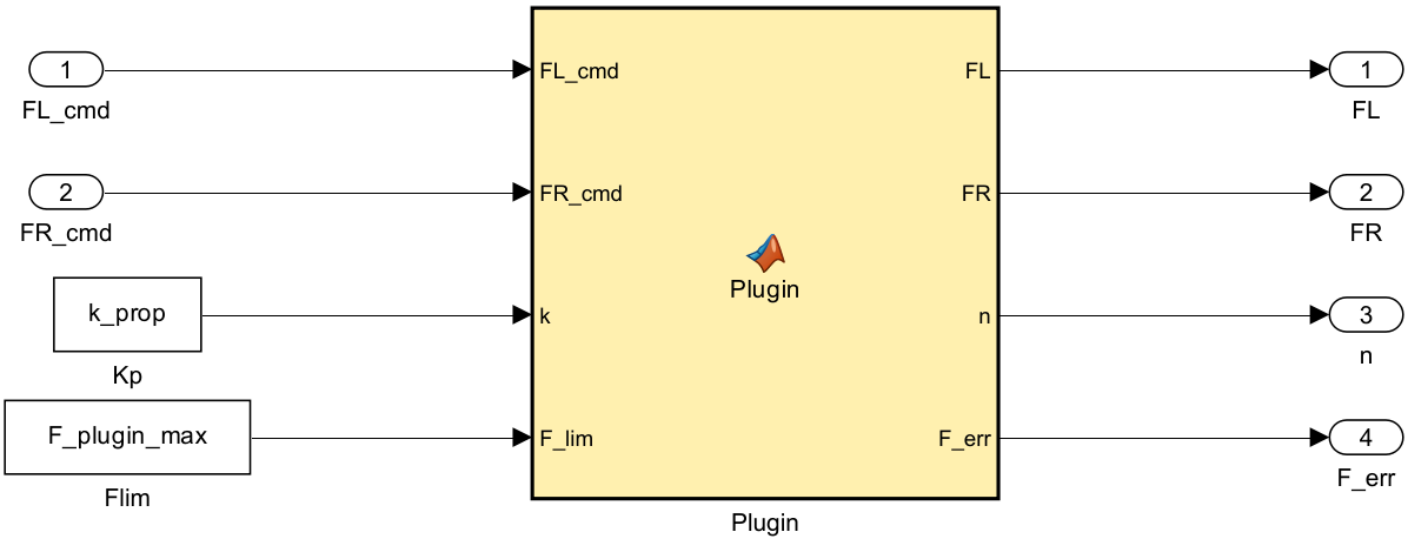
> ⚠ **$U = \sqrt{u^2 + v^2}$ 가 아니라 u 를 되먹임함**
>
> - u, v 모두 대지 기준 속도의 선체 성분임 (1-2절). U 는 그 크기(대지속력)라 옆으로 밀리는 성분 v 까지 포함함
> - 추진기가 만드는 힘은 **전후 방향**이므로 `u` 로 제어하는 것이 맞음
> - MSS 의 `surge velocity control` 서브시스템 입력도 `u_d` 와 `u` 임

추력배분 — 4주차 2-5절의 차동 배분 그대로

$$F_L = \frac{X}{2} + \frac{N}{2b}, \qquad F_R = \frac{X}{2} - \frac{N}{2b}, \qquad b = 1.027\ \mathrm{m}$$

- b = 후방 두 추진기의 좌우 **반폭** (`W05_setup` 의 `half_beam`)
- 여기서 $X = \tau_X$, $N = \tau_N$ (위 두 제어기의 출력을 힘·모멘트 기호로 쓴 것)

Thrusters 서브시스템



Gazebo 추진기 플러그인(gz::sim::systems::Thruster)을 그 순서대로 적은 칸
1) 포화를 추력에 건다 2) 추력 -> 회전수 3) 되돌려 검산 4) 힘은 1) 의 값
회전수는 조인트 속도 지령이 되는 값이다 — velocity_control = true
F_err = 되돌린 추력 - 포화한 추력. 두 식이 서로의 역함수이므로 0 이어야 한다

> ★ **역 프로펠러 법칙은 플러그인 안에 있음. 이 칸은 그것을 소스와 같은 순서로 옮겨 적은 것임**
> VRX 추진기 토픽 `/wamv/thrusters/*/thrust` 는 `std_msgs/Float64` 이고, 그 값이 **뉴턴**임
> 회전수로 바꾸는 일은 Gazebo 쪽 플러그인 `gz::sim::systems::Thruster` 가 안에서 함
> 그 안을 열어 보면 모터 회전수가 추력이 되는 법칙이 그대로 있음 — 제어 배분(`Alloc`)의 **바로 다음 칸**임
> 어디에 있는지는 아래 "법칙이 어디 있는가", 옮겨 적은 것이 맞는지는 "옮겨 적은 것이 맞는가" 에서 확인함

```
FL_cmd, FR_cmd ─┬─ 1) F = clamp(F_cmd, ±F_plugin_max)   포화는 **추력에** 걺
                ├─ 2) n = sign(F)·√(|F|/k_prop)         추력 → 회전수 (조인트 속도 지령)
                ├─ 3) F_chk = k_prop·|n|·n              회전수 → 추력 (검산용)
                └─ 4) 내보내는 힘 = 1) 의 값            링크에 가는 힘은 **포화만 거친 지령**
                   F_err = F_chk − F                     3) − 1) = 0 이어야 함
```

- 서브시스템 안은 MATLAB Function 블록 **하나**임. 입력 넷(`FL_cmd` · `FR_cmd` · `k_prop` · `F_plugin_max`), 출력 넷(`FL` · `FR` · `n` · `F_err`)
- 네 토막을 한 블록에 적은 까닭 — 블록을 나누면 좁은 칸 안에서 가지선이 생겨 선이 꺾임. 포트를 늘려 받는 쪽이 도면이 곧음
- n 과 F_{err} 은 운동모델로 가지 않고 로깅· `Display` 로만 감 — n 은 **보여 주는 눈금**, F_{err} 은 **검산값**임

플러그인 안의 두 식 (`Thruster.cc` 의 `ThrustToAngularVec` · `AngularVelToThrust`)

$$\omega = \operatorname{sign}(F)\sqrt{\frac{|F|}{\rho\, C_t\, D^4}}, \qquad F = \rho\, C_t\, D^4\, |\omega|\,\omega$$

- 포화도 플러그인이 **추력에** 걺 — `clamp(F, ±max_thrust_cmd)` , URDF 값은 편당 **2353.5** N
- `velocity_control = true` 이므로 구한 ω 가 조인트 속도로 바로 들어감. 모터 지연도 PID 도 없음
- 링크에 실제로 실리는 힘은 **되돌린 값이 아니라 포화만 거친 지령**임. 그래서 둘째 식(회전수 → 추력)은 플러그인에서 회전수 지령 모드에만 쓰임

- 이 칸에서도 둘째 식은 신호 경로에 넣지 않고 **검산**으로 둠. 넣어도 $F \to n \to F$ 가 항등 사상이라 아무 일도 하지 않음. 대신 그 차이를 F_{err} 로 내보내 **0 인지 수치로 확인**함

법칙이 어디 있는가 — WAM-V Gazebo 모델 안의 자리

무엇	파일	줄
플러그인 선언 · 계수 셋	`vrx/vrx_urdf/wamv_gazebo/urdf/thruster_layouts/wamv_gazebo_thruster_config.xacro`	10–24
펼친 URDF — 좌현	`bow_wamv/bow_wamv_target.urdf`	462–473
펼친 URDF — 우현	〃	484–495
펼친 URDF — 선수	〃	506–517
포화를 추력에 거는 곳 (`OnCmdThrust`)	`gz-sim7 src/systems/thruster/Thruster.cc`	520–521
추력 → 회전수 (`ThrustToAngularVec`)	〃	562–584
회전수 → 추력 (`AngularVelToThrust`)	〃	593–599
회전수를 조인트에 바로 씀 (`velocity_control`)	〃	740–756
링크에 힘을 주는 곳 (`AddWorldWrench`)	〃	770–773

- Gazebo 판은 `vrx/vrx_gz/CMakeLists.txt:8` 의 `find_package(gz-sim7)` 으로 확인함
- xacro 가 들고 있는 계수 — `thrust_coefficient` $C_t = 0.004422$, `fluid_density` $\rho = 1000$, `propeller_diameter` $D = 0.2$ m. **세 추진기가 모두 같은 값**임
- 포화 한계도 같은 xacro 에서 나옴

$$F_{plugin,\max} = \frac{(X_u + X_{u|u|}\, v_{\max})\, v_{\max}}{2} = \frac{(51.3 + 72.4 \times 7.71667) \times 7.71667}{2} = 2353.5 \text{ N}$$

- 뜻을 풀면 **최고 속도에서의 항력을 추진기 둘이 나눠 낸 몫**임. 임의로 고른 숫자가 아님
- `deadband` 는 기본값이 꺼짐이고 VRX URDF 에 `<deadband>` 항목이 없음 — 그래서 옮겨 적지 않음
- 플러그인 주석이 밝히는 출처는 Fossen, *Guidance and Control of Ocean Vehicles*, p. 246 임

옮겨 적은 것이 맞는가 — 수치로 확인

확인할 것	플러그인	시뮬링크	맞는가
추력 계수 $k = \rho C_t D^4$	$1000 \times 0.004422 \times 0.2^4$	`k_prop` = 0.0070752	같음
포화 한계	`max_thrust_cmd` = 2353.53383667818 N	`F_plugin_max` = 같은 값	같음
포화를 거는 곳	추력 (`OnCmdThrust`)	추력 (`F = clamp(F_cmd, ±F_lim)`)	같음
링크에 가는 힘	포화만 거친 지령	포화만 거친 지령	같음
왕복 $F \to n \to F$	두 식이 서로의 역함수	실측 $\max\lvert F_{err}\rvert = 8.53 \times 10^{-14}$ N	같음 (배정밀도 반올림 수준)

- 왕복 검산은 돌릴 때마다 `W05_plot` 이 찍음 — `왕복 검산 max|F_err|` 줄

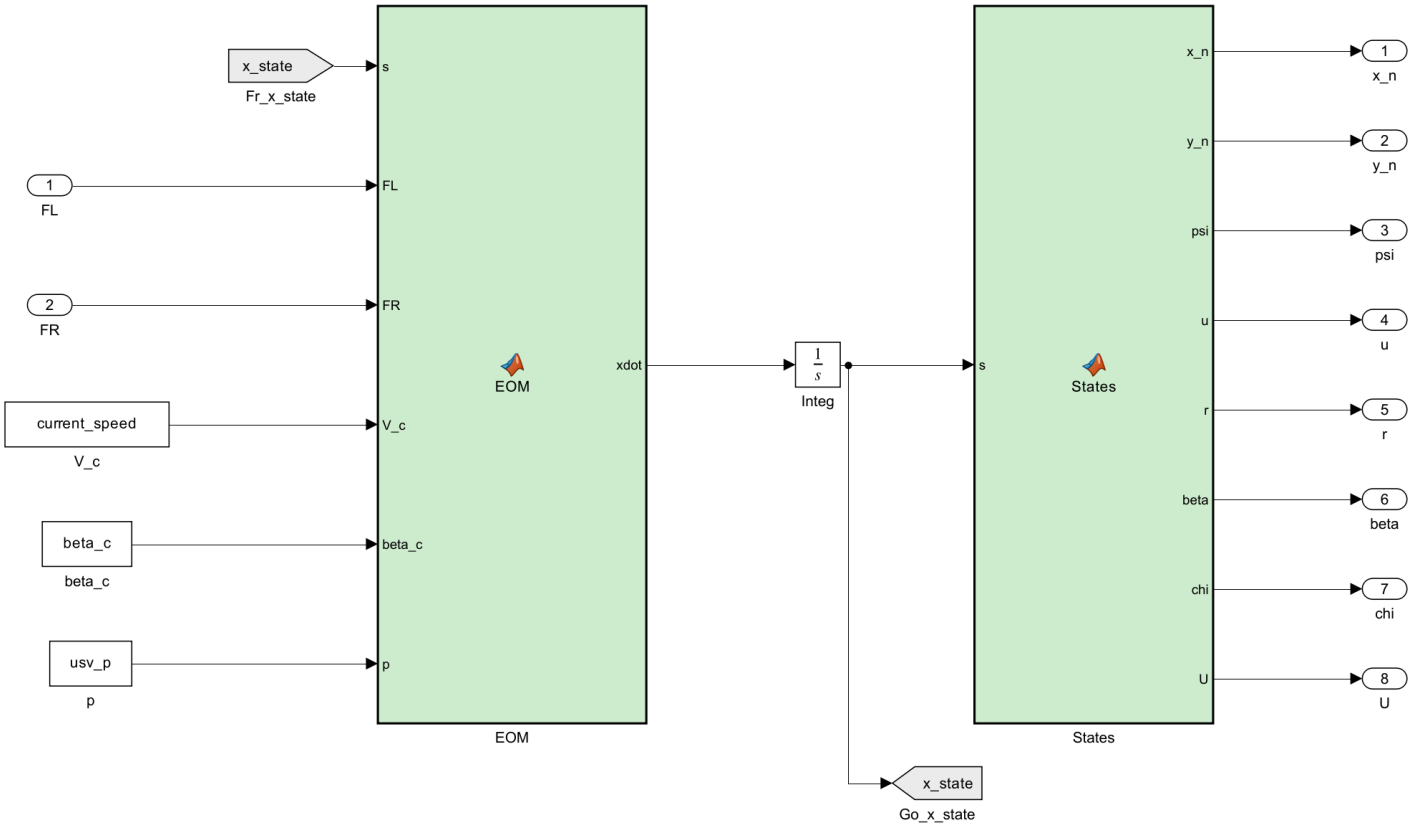
- 8.53×10^{-14} N 은 $\sqrt{\cdot}$ 와 제곱을 한 번씩 거치며 생긴 반올림임. 추력 수백 N 에 견주면 10^{-16} 배이므로 **두 식이 정확히 역함수**라는 뜻임
- 기본 조건에서 $\max|F_L| = 461.1$ N 으로 플러그인 한계 2353.5 N 의 $20\,\%$ 임 — 포화를 **먼저 거는 것은 제어기 쪽 $F_{\max}$** 임 (측정 2026-10-15)

> **⚠ 같은 글자, 다른 뜻 — k 와 n**
>
글자	1부 · §1-9 (유도)	이 절 (추진기)
k	웨이포인트 번호	프로펠러 추력 계수 → 혼동을 피해 `k_prop` 으로 씀 (`W05_setup` 과 같은 이름)
n	웨이포인트 개수 (§1-9 코드)	프로펠러 회전수 [rad/s]

- $k_{\text{prop}} = \rho\, C_t\, D^4 = 1000 \times 0.004422 \times 0.2^4 = 0.0070752$
  — VRX 플러그인이 내부에서 쓰는 값과 **같은 값** (URDF 의 `fluid_density` · `thrust_coefficient` · `propeller_diameter`)
- $n_{\max} = \sqrt{F_{\max}/k_{\text{prop}}} = 265.8$ rad/s (`F_max = 500` 기준, 약 2538 rpm).
  포화 블록이 아니라 참고선임 — 기본 조건 실측 $\max|n| = 255.3$ rad/s 로 그 아래에 머묾
- 포화는 힘에 걸림. 추력이 회전수의 제곱이므로 F 를 반으로 줄여도 n 은 $1/\sqrt{2}$ 배로만 줄어듦

> **★ `F_max` 를 왜 500 N 으로 두는가**
>
> 1.5 m/s 를 유지하는 데만 $X = (100 + 150 \times 1.5) \times 1.5 = 488$ N, **편당 244 N** 이 듦
> 4주차처럼 250 N 으로 제한하면 직진에 다 써 버려 **선회할 여유가 없음**
>
`F_max`	포화 시간 비율	정착 후 평균 |y_e|	완주 [s]	실측 max |n| [rad/s]	정상상태 u [m/s]
250 N	**57.2 %**	1.366 m	215.7	188.0	1.457 — 목표 미달
400 N	2.8 %	0.742 m	206.3	237.8	1.500
500 N	**0.0 %**	**0.724 m**	206.1	255.3	1.500
>
> (2026-10-15 측정. 조류 없음 · LOS · $\Delta = 10$ m · $R = 5$ m)
> 250 N 에서는 포화가 절반 넘게 걸려 목표 속도조차 못 냄 — 완주가 9.6 초 늦고 이탈량이 두 배임
> VRX 플러그인의 물리 한계는 편당 2353.5 N 이므로 500 N 은 그 21 % 이고, Gazebo 가 먼저 자르는 일은 없음
> **포화는 게인 문제로 위장함** — 게인을 아무리 만져도 나아지지 않으면 포화부터 확인할 것.

MotionModel 서브시스템



- 운동방정식 → **연속 적분기** → 상태 산출을 한 블록에 담음
- VRX 모델에서 **Gazebo 가 차지하는 자리**와 정확히 같음
- `p` 상수 블록이 `usv_p` $= [m;\ I_{zz};\ X_u;\ X_{u|u|};\ Y_v;\ Y_{v|v|};\ N_r;\ N_{r|r|};\ b]$ 를 `EOM` 에 넘김
  (벡터를 블록 안에 적으면 Simulink 가 값을 감추고 `-C-` 만 그리므로 `W05_setup` 에서 이름 하나로 묶음)

> ★ **이 운동모델은 Gazebo 와 같은 계수를 씀**
> 임의로 만든 모델이 아님. VRX 플러그인 설정에서 그대로 가져온 값임

항목	값	출처
질량	211 kg	`wamv_base.urdf.xacro` 180 + 엔진 2×15 + 프로펠러 2×0.5
요 관성	653 kg·m²	위 파일의 `izz` 446 + 엔진 이동항
전후 항력	`−(100 + 150·	u
좌우 항력	`−(100 + 100·	v
요 항력	`−(800 + 800·	r
부가질량	**0**	`xDotU = yDotV = nDotR = 0`

- 검산: 추력 400 N 에서 $400 = (100 + 150u)\,u \rightarrow u = 1.333$ **m/s**
- 4주차에 Gazebo 로 실측한 값은 **1.332 m/s** 였음 (4주차 2-19절)

ℹ 조류는 오프라인 모델에만 존재함

VRX 2.4.0-2 에는 해류 플러그인이 없음. 레포 전체를 검색해도 나오지 않고,
`SimpleHydrodynamics` 가 항력을 **대지속도**로 계산하므로 구조적으로 해류가 존재할 수 없음
그래서 조류 실험은 이 오프라인 모델에서만 할 수 있음
구현은 간단함 — 항력을 **물에 대한 상대속도**로 계산하면 됨

```
u_r = u - V_c·cos(beta_c - psi)
v_r = v - V_c·sin(beta_c - psi)
```

- V_c = 조류 속도 (`current_speed`)
- β_c = 조류가 **흘러가는** 방위, 북에서 시계방향 (`current_direction` 을 라디안으로 바꾼 값)
- u, v 는 대지 기준 속도의 선체 성분이므로, 물에 대한 상대속도 u_r, v_r 을 따로 만들어 항력에 넣음

⚠ 같은 글자, 다른 뜻 — $\beta \cdot \beta_r \cdot \beta_c$

기호	뜻	기준
β	크랩각 (1-2절)	뱃머리 ψ 에서 진행 방향 χ 까지
β_r	횡류각 (1-2절)	뱃머리 ψ 에서 물살 방향 χ_r 까지
β_c	조류 방향	북에서 조류가 흘러가는 방향까지

앞의 둘은 **뱃머리**에서, 마지막 하나는 **북**에서 잼. 1-2절의 해류 삼각형에 셋이 함께 그려져 있음

C. 실습 1 — atan2 vs LOS (25분)

C-1. LOS 로 한 번

```
W05_setup
```

- 실시간 화면에서 **배가 20 m 벗어난 곳에서 출발해 경로에 붙는 것**을 눈으로 확인. 오른쪽 ② 의 y_e 가 −20 m 에서 0 으로 올라옴
- 이어서 성능 그림 5장 · 추진기 그림 2장과 지표가 나옴

C-2. atan2 로 바꿔서

- `W05_setup.m` 에서 `guidance_mode = 1` 로 고치고 다시 실행

```
W05_setup
```

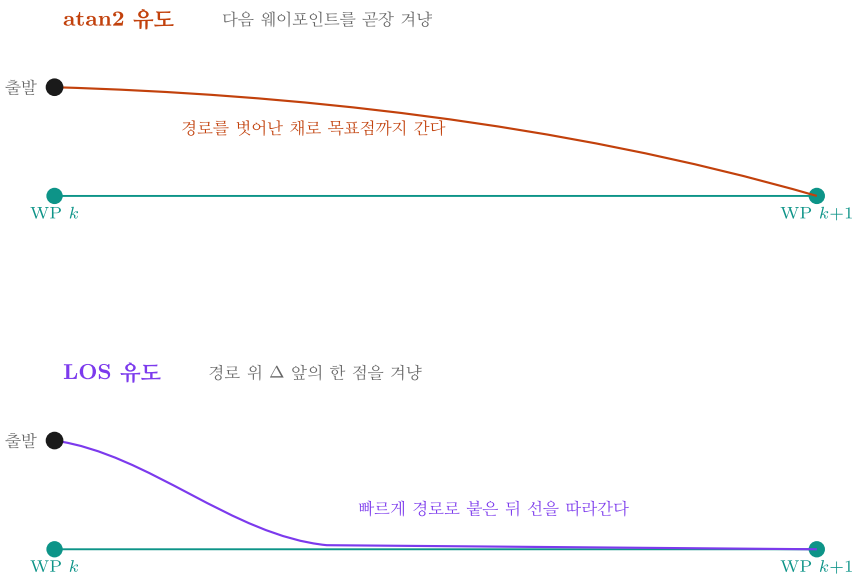
- 두 궤적의 모양이 어떻게 다른지 비교할 것

C-3. 네 조건을 한 번에

```
W05_compare
```

- 정상 출력

```
조건                    mean|ye|       정착후     max|ye|      완주[s]   크랩[deg]
------------------------------------------------------------------------------
atan2 / 조류 없음          4.004      1.851     20.000      201.4      1.92
LOS   / 조류 없음          1.996      0.724     20.000      206.1      2.95
atan2 / 조류 0.4           4.773      3.911     20.000      202.6      8.14
LOS   / 조류 0.4           2.692      1.531     20.000      206.7      8.06
------------------------------------------------------------------------------
  정착후 = 초기 수렴 60 초를 뺀 평균. 유도법칙 비교에는 이 값을 쓴다
  atan2 는 LOS 보다 2.55 배 (조류 없음), 2.56 배 (조류 있음) 더 벗어난다
```



오프라인 WAM-V 실측 — 평균 경로 이탈량 (초기 수렴 60초 제외)

조건	atan2	LOS	배율
조류 없음	1.76 m	0.70 m	2.5 배

관찰할 것

> ★ **다음 세 가지를 확인함**

1. **조류가 없어도** LOS 가 2.55 배 정확함 — 유도법칙 자체의 차이
2. **조류를 켜면** 두 법칙 모두 나빠지고 **벌어지는 거리가 커짐** — 정착 후 차이가 1.13 m 에서 2.38 m 로. 비율은 2.55 배 그대로임
3. **크랩각이 3° → 8.1°** 로 커짐 (조류 0.4 m/s) — 뱃머리와 진행 방향이 8도 어긋남

> ℹ **비율만 보면 놓침**
>
> atan2 는 1.85 m → 3.91 m 로 **2.06 m** 나빠졌고, LOS 는 0.72 m → 1.53 m 로 **0.81 m** 나빠졌음
> 조류가 같은 만큼 밀었는데 atan2 가 2.5 배 더 밀렸음. 비율이 같게 나오는 것은
> 둘 다 나빠졌기 때문임. **비교할 때는 비율과 차이를 함께 봄**

> ℹ **LOS 도 조류에서 오차가 남음**
>
> 조류 없음 0.72 m → 조류 있음 1.53 m. 완전히 0 이 되지는 않음
> 선수각 ψ 를 제어하기 때문임 (1-2절). 이 잔류 오차를 없애는 것이
> **적분 LOS(ILOS)** 임. 관심 있는 팀은 Term Project 에서 시도해 볼 것

D. 실습 2 — Δ · R · 전환 판정 바꿔 보기 (20분)

D-1. lookahead distance

- 먼저 §1-7 의 예제를 그대로 돌려 봄. 네 조건을 알아서 돌리고 그림과 표를 냄

```
>> W05_lookahead_demo
```

- 그다음 `W05_setup.m` 에서 `Delta` 만 바꾸며 미션 전체를 실행함. 나머지는 손대지 않음

```
Delta = 2;      % 2 · 3 · 5 · 10 · 20 · 30 으로 여섯 번
```

- 각 조건에서 `W05_plot` 이 찍는 **정착 후 평균** `|y_e|` 와 **완주 시각**을 적음
- 궤적 그림에서 **경로를 몇 번 가로지르는지**도 눈으로 셈

볼 것	Δ 가 작을 때	Δ 가 클 때
경로에 붙는 속도	빠름	느림
궤적 모양	지그재그	완만
요 모멘트를 쓰는 양	많음	적음

- **§1-7 의 표와 맞는지 대조할 것.** 조류 없이 `Delta = 10` 이면 정착 후 평균이 **0.72** m 여야 함
- 그다음 `current_speed = 0.4`, `current_direction = 90` 으로 두고 다시 여섯 번 돌림. **최적 Δ 가 어디로 옮겨 가는지** 봄 (1-7절 두 번째 표와 같은 조건)

D-2. 수락반경

- 이번에는 `R_LOS` 만 바꿈

```
R_LOS = 2;      % 2 · 5 · 10 · 20
```

R	볼 것
2	모서리를 정확히 돎. 회전이 급함
5	기본값
10 · 20	모서리를 크게 잘라먹고 그만큼 일찍 끝남

- 궤적 그림에서 배가 웨이포인트에 **얼마나 가까이 갔는지**를 봄 (§1-8 의 "모서리 컷")

D-3. 전환 판정 — 수락 원은 걸릴 수 있다

- §1-8 에서 계산으로 보인 것을 **직접 걸리게 만들어 봄**

```
R_LOS         = 2;      % 수락 원을 좁게
current_speed = 0.4;    % 조류로 옆으로 밀어낸다
current_direction = 90;
sw_mode       = 2;      % 수락 원 판정
```

- 실행하면 배가 **WP2 (60, 20) 옆을 스쳐 지나간 뒤 전환하지 못하고, 첫 구간의 연장선을 따라 계속 북쪽으로 감.** 미션이 끝나지 않음
  - LOS 지령에는 x_e 가 없어서 지나친 웨이포인트로 되돌아가지 않음 (§1-8)
- `sw_mode = 1` 로만 바꿔 다시 돌림. 같은 조건에서 정상적으로 완주함

`sw_mode`	결과	전환 횟수
2 (수락 원)	미완주	0
1 (경로 방향)	214.1 s 완주	3

- **왜 그런지를 §1-8 의 부등식으로 설명할 수 있어야 함**

> ✎ `Scope` 대신 `W05_plot` 사용을 권장함
>
> 궤적·이탈량·헤딩·크랩각·속도·추력을 한 번에 그려 주고
> 성능 지표를 표로 찍어 줌

E. 실습 3 — VRX 연동 (50분)

E-1. VRX 띄우기 — 3주차 경량 모델 그대로 (그대로 따라 하면 됨)

> **★ 새로 설치하는 것은 없음**
> 3주차 2-2 에서 설치한 VRX(`~/vrx_ws`)와 2주차 2-6 에서 받은 강의자료(`~/Capstone-Design`)만 씀. 띄우는 명령도 3주차 2-3 의 `run_vrx.sh` 그대로임
>
> - 아래 순서는 2026-10-16 에 GitHub 에서 **새로** `git clone` 한 강의자료로, 기준 노트북(VRX 원본 그대로 설치)에서 한 줄씩 실행해 확인한 것임

터미널을 둘 연다 — VS Code 의 WSL 창 터미널(2주차 2-4)이나 Ubuntu 터미널 어느 쪽이든 됨

터미널	하는 일	끝날 때까지
1	VRX 를 띄움	**닫지 않고** 떠 있는 채로 둠
2	토픽 · RTF 확인	확인이 끝나면 닫아도 됨

1. (터미널 1) 강의자료를 최신으로 받음

```
cd ~/Capstone-Design && git pull
```

- 정상 출력: `Already up to date.` 또는 바뀐 파일 목록. 이 주차의 `W05_vrx_run` 수정(2026-10-16)이 여기서 들어옴

2. (터미널 1) 도메인 번호 확인

```
echo $ROS_DOMAIN_ID
```

- 정상 출력: `7` . 비어 있거나 다른 값이면 `~/.bashrc` 의 `export ROS_DOMAIN_ID=7` 을 고친 뒤 `source ~/.bashrc`
- MATLAB 쪽 둘도 7 이어야 함 — `W05_setup.m` 의 `ros_domain_id` 와 Simulink 툴스트립 **시뮬레이션 → ROS 네트워크** 의 Domain ID (4주차 2-14-5)

3. (터미널 1) 앞 실험의 VRX 가 남아 있으면 끔 — **실험마다 이 줄부터**

```
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W03_vrx_lite
bash stop_vrx.sh
```

- 정상 출력 (아무것도 안 떠 있을 때)

```
끄기 전: 0 개
떠 있는 것이 없음 — 바로 띄우면 됨
```

- 정상 출력 (남아 있었을 때)

```
끄기 전: 10 개
TERM 뒤 남은 것: 2 개 → KILL
끈 뒤: 0 개 — 정리 완료
```

4. (터미널 1) VRX 띄우기

```
bash run_vrx.sh
```

- 맨 앞 네 줄이 **실제로 실행하는** `ros2 launch` **명령**을 찍음. 직접 치는 것과 똑같음

```
화면 정상 (DISPLAY=:0, 창 표시기 재시작 0 번)
ROS_DOMAIN_ID=7  (MATLAB W0X_setup 의 값과 같아야 함)
실행 명령:
  ros2 launch vrx_gz competition.launch.py world:=sydney_regatta "urdf:=/home/<사용자명>/Capstone-Design/.../W03_vrx_lite/wamv_lite.urdf" "extra_gz_args:=--render-engine-gui ogre"
끝낼 때는 이 터미널에서 Ctrl+C
```

- 그 뒤로 노란 `[Wrn]` 과 빨간 `[Err]` 줄이 수백 줄 쏟아짐 — **정상임** (아래 그림)
- 20~40 초 뒤 **Gazebo Sim** 창이 뜨고 물 위에 배가 보이면 성공
- 첫 줄 `화면 정상` 은 `run_vrx.sh` 가 Gazebo 를 띄우기 **전에** WSL 의 화면 상태를 본 결과임 (`check_gui.sh`, 바로 아래 경고 상자)

✕ 로그는 쏟아지는데 Gazebo 창이 안 뜰 때 — `wsl --shutdown` 한 줄로 고침 (2026-10-23 기준 노트북 실측)

- 증상: `run_vrx.sh` 뒤 경고 줄이 계속 나오고 `ros2 topic list` 에 토픽도 보이는데, Windows 화면에 Gazebo 창이 **나타나지 않음**. 오류 메시지는 없음
- 원인: 리눅스 창을 Windows 화면에 그려 주는 프로그램(`msrdc.exe`, WSLg 의 일부)이 30 초마다 꺼졌다 켜지기를 반복하는 상태. Gazebo 만이 아니라 **리눅스 창이 전부** 안 뜸. 노트북을 재웠다 깨우거나 며칠 켜 두면 생김
- `run_vrx.sh` 가 이 상태를 먼저 알아보고 멈춤 — 첫 줄이 이렇게 나옴

```
화면 이상: Windows 쪽 창 표시기(msrdc.exe)가 5 번 다시 켜졌음 → Gazebo 창이 안 뜸
  고치는 법: Windows PowerShell 에서  wsl --shutdown
```

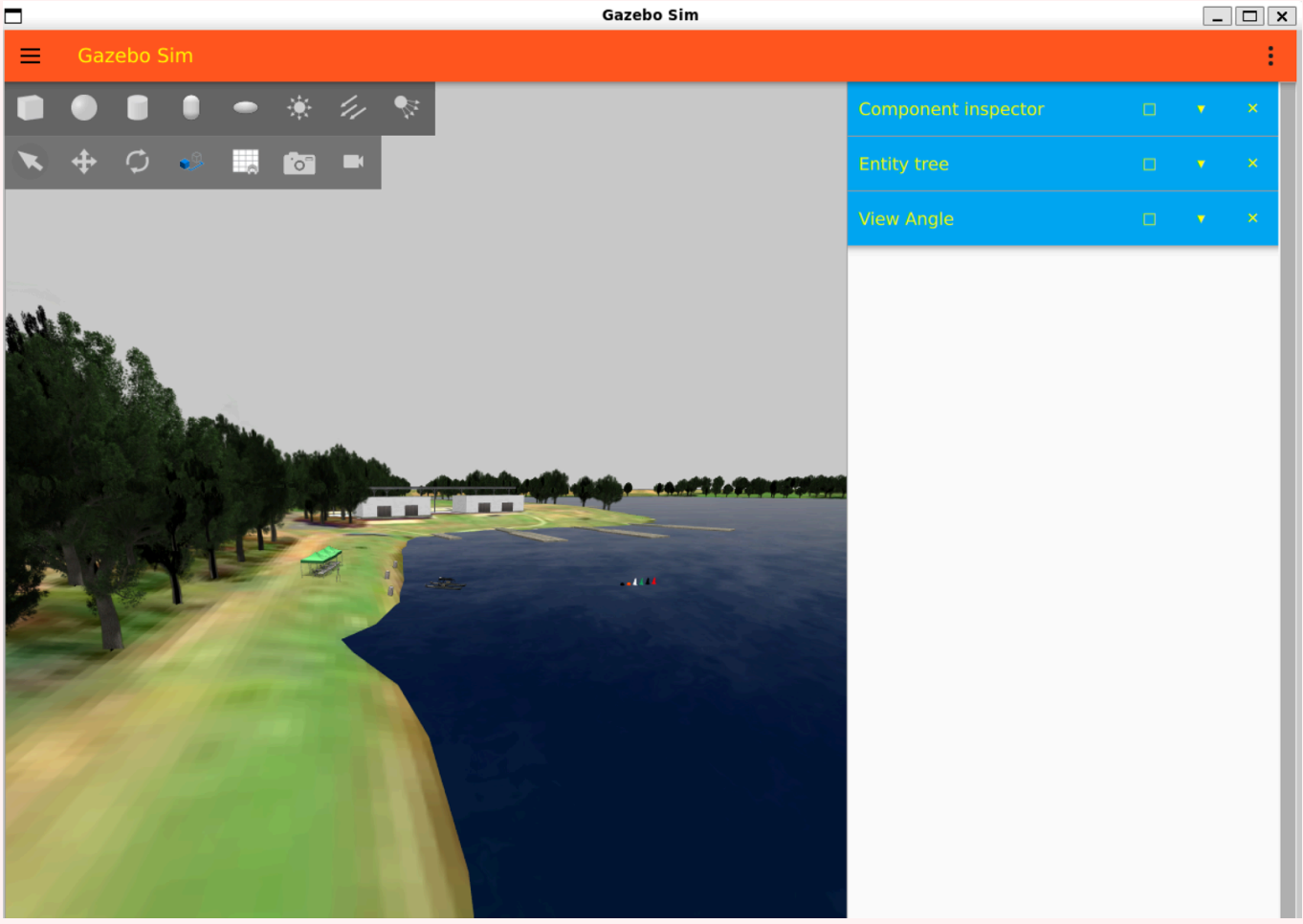
고치는 순서

1. 열려 있는 WSL 터미널을 모두 닫음 (VS Code 의 WSL 창도)
2. `Win` 키 → `powershell` 입력 → **Windows PowerShell** 클릭
3. 아래 한 줄을 침 — 아무것도 찍히지 않고 바로 끝나는 것이 정상

```
wsl --shutdown
```

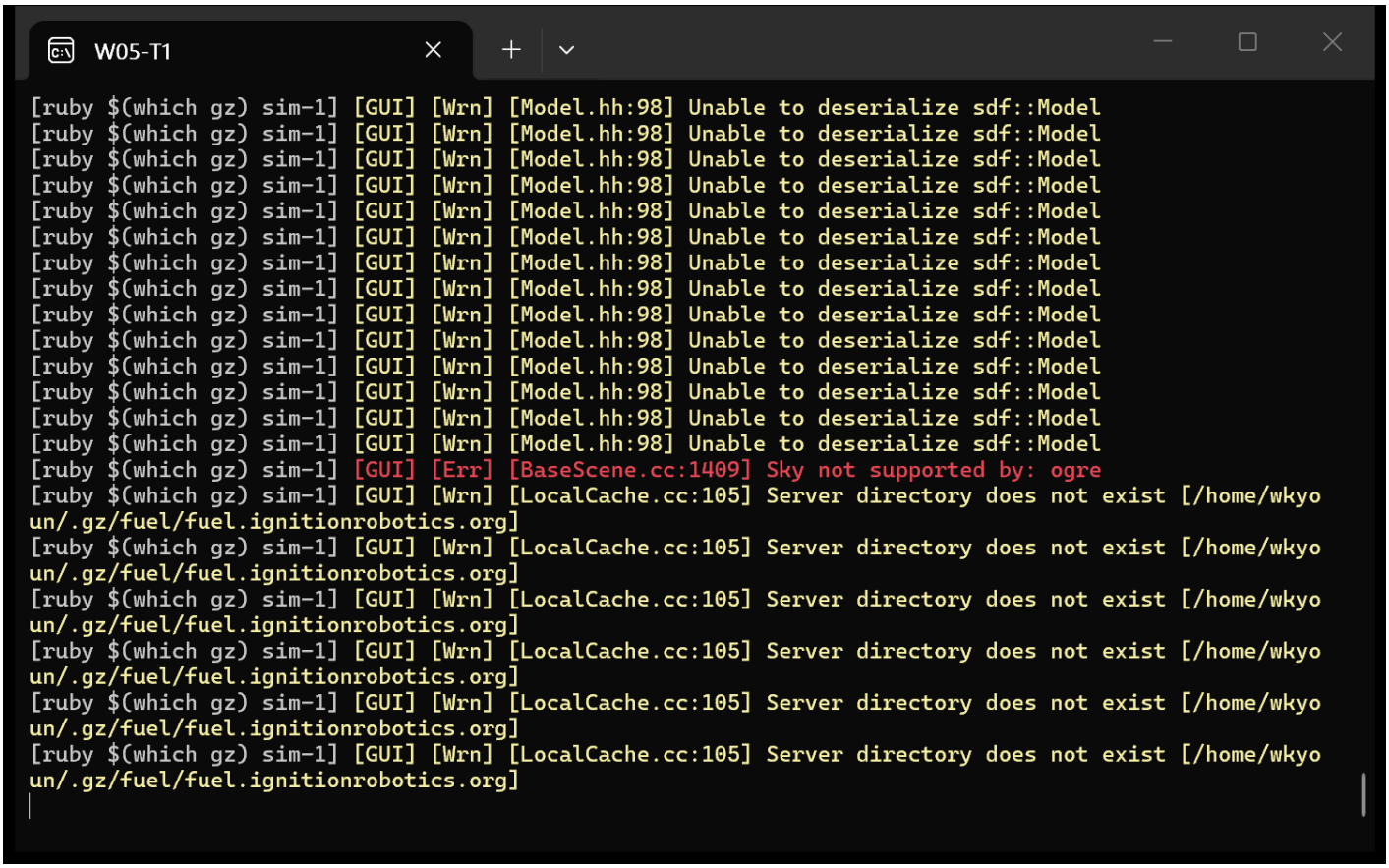
4. 터미널을 새로 열고 (WSL 이 다시 켜지는 데 10~30 초) 위 3번 `bash stop_vrx.sh` 부터 다시
5. 따로 확인하려면 `bash check_gui.sh` → `화면 정상 (... 재시작 0 번)` 이면 됨

고친 뒤의 화면 — Windows 바탕화면에 **Gazebo Sim** 창이 따로 뜸

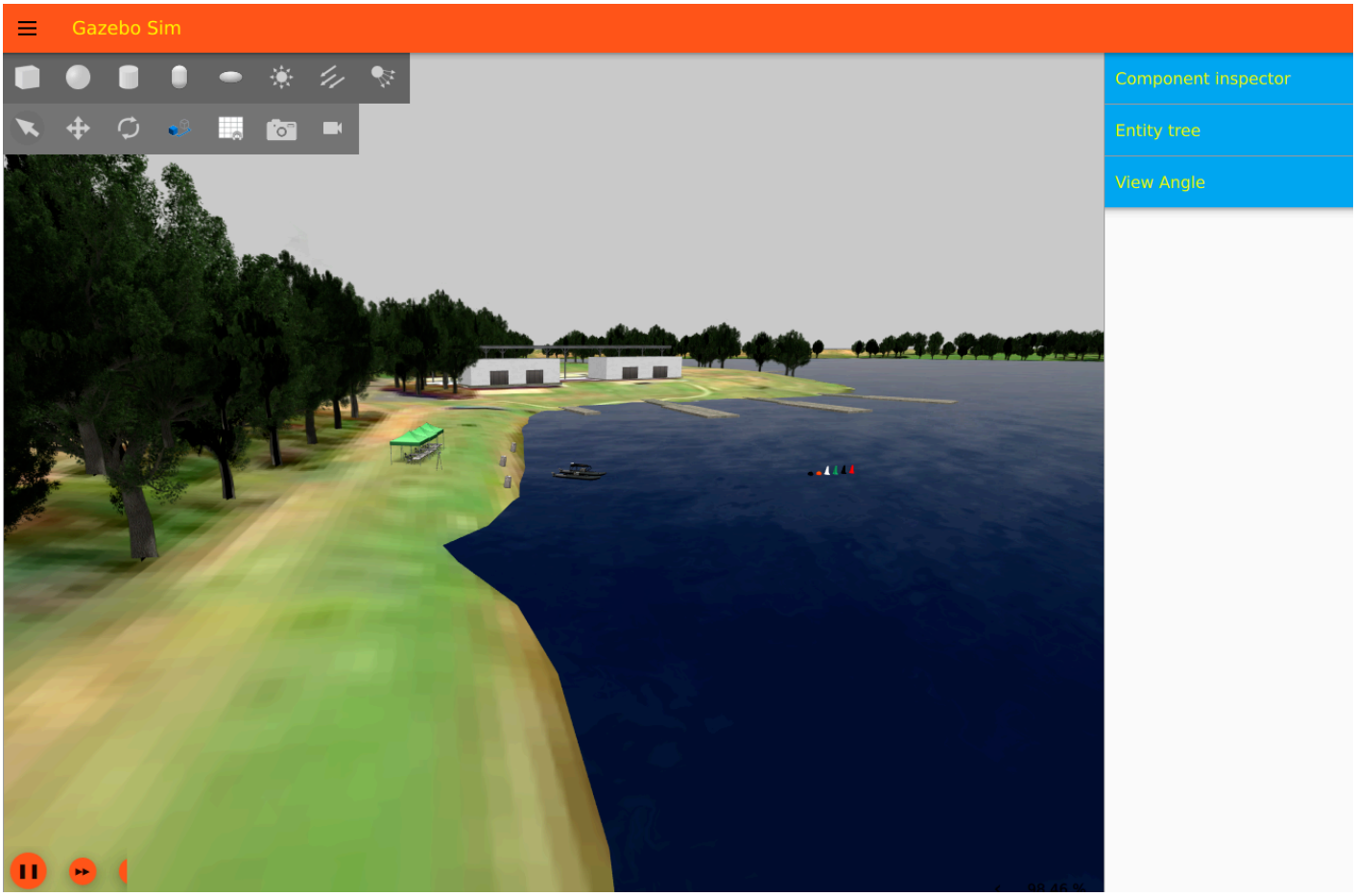


✎ **창은 떴는데 3D 화면만 흰색이면 — 기다리면 됨 (고장이 아님)**

- 증상: Gazebo Sim 창은 떴고 왼쪽 패널·메뉴도 보이는데, 가운데 3D 화면이 **흰 바탕**임
- 원인: 배·부두·물의 모양 파일(메시)과 무늬 파일(텍스처)을 디스크에서 읽어 들이는 중임. 처음 띄울 때는 캐시가 비어 있어 더 걸림
- 20~40 초 더 두면 물과 배가 그려짐. 토픽은 이미 나오고 있으므로 Simulink 를 먼저 돌려도 결과는 같음
- 1 분이 지나도 흰색이면 그때는 위 `wsl --shutdown` 순서로



보이는 줄	뜻
`[Wrn] [Model.hh:98] Unable to deserialize sdf::Model`	GUI 가 모델 정보를 다 읽기 전에 생기는 경고. 무시함
`[Err] [BaseScene.cc:1409] Sky not supported by: ogre`	GUI 엔진을 가벼운 `ogre` 로 낮춰서 하늘 그림만 빠짐. 무시함
`[Wrn] [LocalCache.cc:105] Server directory does not exist`	모델 내려받기 캐시 경고. 무시함



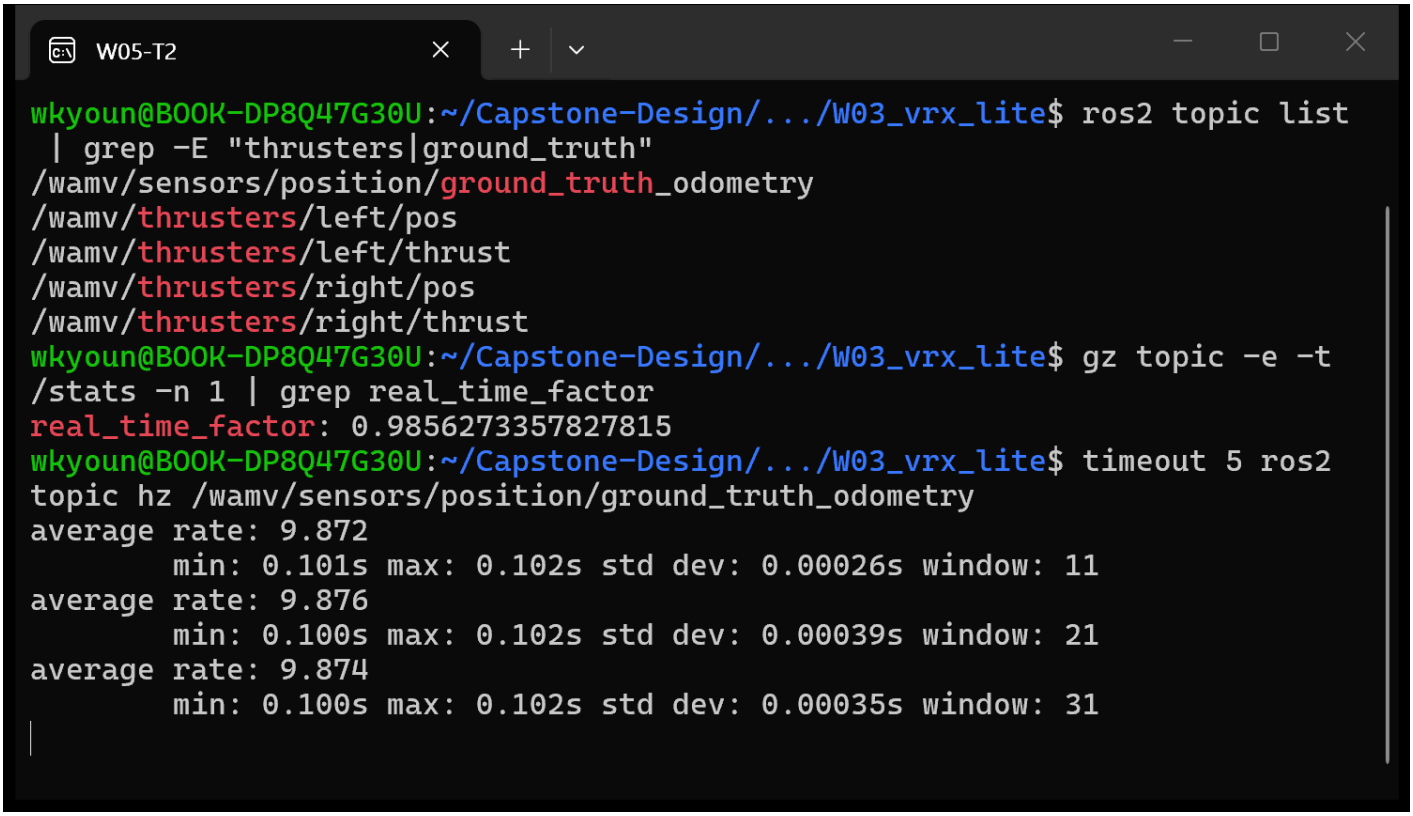
화면의 위치	무엇인가
가운데 물 위의 검은 배	WAM-V. **스폰 자리가 서쪽 해안 바로 옆**임 — E-4 의 "경로 원점" 이 필요한 까닭
오른쪽 작은 색 원뿔	부표
왼쪽 아래 주황 단추	일시정지 · 재생. 건드리지 않음
오른쪽 아래 `99.xx %`	실시간 비율(RTF). 90 % 이상이면 됨

5. (터미널 2) 토픽과 RTF 확인

```
echo $ROS_DOMAIN_ID
ros2 topic list | grep -E "thrusters|ground_truth"
gz topic -e -t /stats -n 1 | grep real_time_factor
timeout 5 ros2 topic hz /wamv/sensors/position/ground_truth_odometry
```

- 정상 출력

```
7
/wamv/sensors/position/ground_truth_odometry
/wamv/thrusters/left/pos
/wamv/thrusters/left/thrust
/wamv/thrusters/right/pos
/wamv/thrusters/right/thrust
real_time_factor: 0.99...
average rate: 9.88
```



- `ground_truth_odometry` 가 없으면 MATLAB 이 시작하지 않음 — 경량 모델(`wamv_lite.urdf`)에는 이미 켜져 있음
- 띄운 직후 30 초 안에는 RTF 가 0.5 안팎으로 낮게 나올 수 있음. 1 분쯤 뒤 다시 잼 (기준 노트북 실측 0.47 → 1.00)

> **★ 실험마다 VRX 를 다시 띄움 — 3번(`stop_vrx.sh`)부터 다시**
>
> 앞 실험이 끝난 자리(경로 끝)에서 다음 실험을 시작하면 결과가 완전히 달라짐
>
> - 모든 실험은 스폰 위치 · 초기 선수각이 같은 상태에서 시작하는 것을 전제로 함
> - 7주차 로이터링 실습도 같음 — 이어 돌리면 원에 도달하지 못함

> **✕ 터미널 창을 Ctrl+C 없이 닫으면 Gazebo 가 고아로 남음**
>
> 남은 `gz sim server` 는 보통의 종료 신호를 무시함 (2026-10-16 실측). 다음에 띄우면 서버가 둘이 되어 배가 두 척 보이거나 토픽 값이 튐
>
> - `stop_vrx.sh` 가 3 초 기다린 뒤 남은 것을 강제로 끔. 그래도 남으면 Windows PowerShell 에서 `wsl --shutdown`

E-2. 참고 — 경량 모델 없이 `ros2 launch` 를 직접 칠 때

- 4주차 2-14-4 의 ground truth 를 켠 xacro 로 직접 띄우는 방법. **수업에서는 E-1 을 씀**

```
mkdir -p ~/capstone_ws/wamv
cp ~/vrx_ws/src/vrx/vrx_urdf/wamv_gazebo/urdf/wamv_gazebo.urdf.xacro \
   ~/capstone_ws/wamv/w7_wamv.urdf.xacro
```

- `ground_truth_enabled` 의 `default` 를 `true` 로 수정

```
nano ~/capstone_ws/wamv/w7_wamv.urdf.xacro
```

```
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta urdf:=$HOME/capstone_ws/wamv/w7_wamv.urdf.xacro "ex
tra_gz_args:=--render-engine-server ogre --render-engine-gui ogre"
```

> ⚠ **한 줄 명령임. `extra_gz_args` 를 빼면 노트북에서는 사실상 멈춤**
>
> 이 xacro 는 카메라 3대 + 3D LiDAR 를 켬. 기본 엔진(`ogre2`)으로 띄운 기준 노트북 실측 (2026-10-16)

명령	RTF	화면
위 명령에서 `extra_gz_args` 를 뺌	**0.0013**	1 fps
위 명령 그대로 (서버·GUI `ogre`)	0.83 ~ 0.98	49 fps (3주차 2-3)
E-1 `run_vrx.sh` (센서 최소)	**0.99 ~ 1.00**	49.5 fps

E-3. 실시간 계수(RTF) 측정 — 반드시 할 것

> ✕ **이 단계를 건너뛰면 결과가 틀림**
>
> Gazebo 는 컴퓨터에 따라 실시간보다 느리게 돎
> Simulink 를 벽시계에 맞춰 돌리면 배는 그만큼 적은 시뮬레이션 시간을 살게 됨

```
ros2 topic echo /wamv/sensors/position/ground_truth_odometry --once | grep -m1 "sec:"
```

- 20초 기다렸다가 다시 실행해서 **시뮬레이션 시각이 몇 초 늘었는지** 확인

```
20초 기다렸는데 시뮬 시각은 10초만 늘었다  →  RTF = 0.5
```

- 한 줄로 보려면 (WSL)

```
gz topic -e -t /stats -n 1 | grep real_time_factor
```

환경	RTF
기준 환경 (2026-09-04)	**0.48**
기준 노트북 (2026-09-15, `ogre` 옵션)	**0.83 ~ 0.98** — 같은 월드에서도 흔들림

E-4. 모델 실행 — 한 줄이면 된다

```
setappdata(0, 'W05_skip_run', true);   % W05_setup 의 오프라인 자동 실행을 건너뜀
W05_setup
W05_vrx_run
```

- `W05_vrx_run.m` 이 아래를 대신 해 줌

단계	하는 일
0	Simulink 프로필과 MATLAB 의 ROS 도메인을 맞춤
1	토픽이 보이는지 확인 (안 보이면 **여기서 멈추고 이유를 알려 줌**)
2	**RTF 를 20초 동안 직접 측정** — 손으로 재지 않아도 됨. 스폰 위치를 읽고 **경로 원점은 월드에 고정**함 (아래 경고 상자)
3	페이싱 비율을 RTF 에 맞추고 실행. 실시간 화면은 **끈 채로** 돎
4	끝나면 오프라인과 **같은 형식의 결과 그림** + **동기 점검 k**
5	같은 출발점 · 같은 초기 선수각으로 오프라인을 돌려 **두 궤적을 겹쳐 그림**. 오프라인 지표도 같은 형식으로 찍음

- 기본 270 s — 임무(약 232 s)를 끝까지 봄. `W05_vrx_run(150)` 이면 완주 전에 끊김. 대조를 건너뛰려면 `W05_vrx_run(270, false)`
- MATLAB 은 강의자료 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\1_2026-2학기_강의자료\10-주차별-강의자료\W05_simulink` 로 엶 (주소창에 붙여 넣고 `Enter`). 이 경로 그대로 모델이 돎 — 2026-10-16 확인

✕ 경로는 스폰이 아니라 월드의 한 점에 붙임 — VRX 원본은 스폰이 해안 옆임 (2026-10-16 수정)

- VRX 원본 `competition.launch.py:46` 의 스폰은 ENU $(-532,\ 162)$. 같은 2.4.0-2 인데 기준 데스크톱만 $(-532,\ 200)$ 이었음
- 전에는 스폰을 경로 원점 $(0, 0)$ 으로 삼았음 → 스폰 162 인 설치(학생 노트북 전부)에서는 경로가 **38 m 남쪽으로 밀려** 4번 웨이포인트 부근 해안에 얹힘. 속도 1.5 → **0 m/s**, 미완주, 평균 크랩각 67.5° 로 기록됨
- 지금은 경로 원점을 `W05_setup.m` 의 `path_north0 = 200` · `path_east0 = -532` (월드 좌표)에 고정함. 배는 스폰에서 출발하므로 162 설치에서는 **경로 좌표 $(-38,\ 0)$ 에서 출발**해 첫 구간으로 들어감
- 2단계 출력의 `경로 원점 ... 배는 경로 좌표 (-38.0, -0.1) 에서 출발` 이 그 줄임. 데스크톱(200)에서는 $(0,\ 0)$ 이 찍힘
- 오프라인 대조도 같은 출발점에서 돌림 — 그래서 두 궤적이 처음부터 겹침

⚠ VRX 실행 중에는 실시간 화면을 끔 — 켜면 Simulink 가 VRX 를 못 따라감 (2026-10-16 실측)

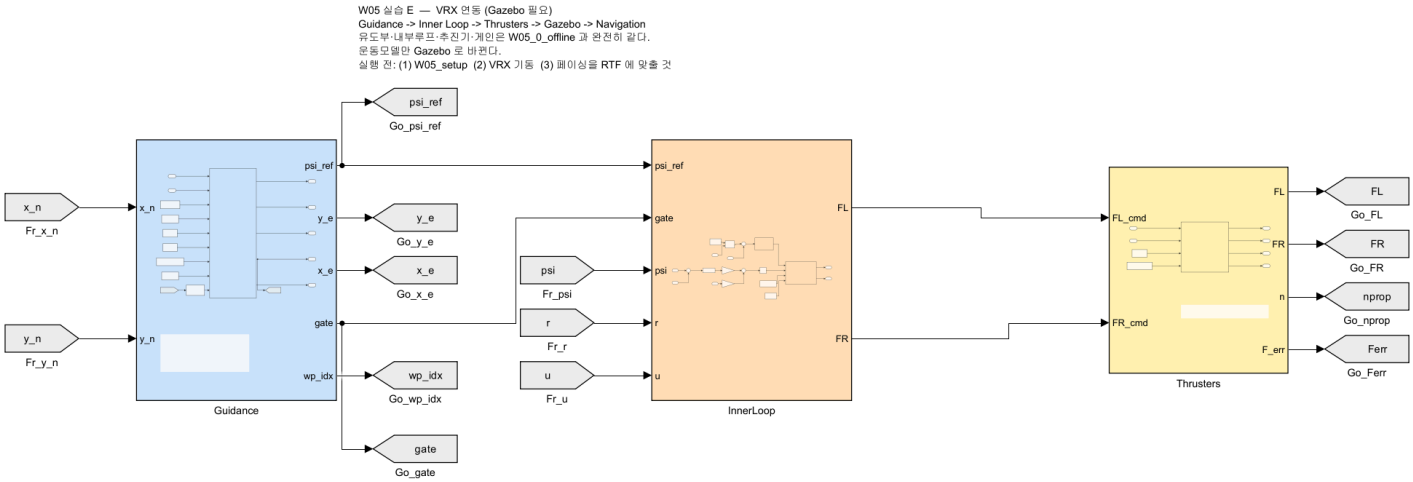
- 화면을 그리는 동안 Simulink 가 늦어져, 같은 모델 1 초 동안 배가 **1.7 배** 더 감. 메시지 손실 경고가 155 건 찍히고 완주 시각이 140.6 s 로 줄어듦 (끄면 232.7 s)
- 그래서 `W05_vrx_run` 은 `animate = 0` 으로 돎. 도는 모습은 오프라인 모델(`W05_setup`)의 실시간 화면으로 봄
- 빠른 PC 에서 켜 보려면 `W05_vrx_run(270, true, 1)` — 끝난 뒤 4단계의 **동기 점검 k** 가 1.08 을 넘으면 그 결과는 쓰지 않음

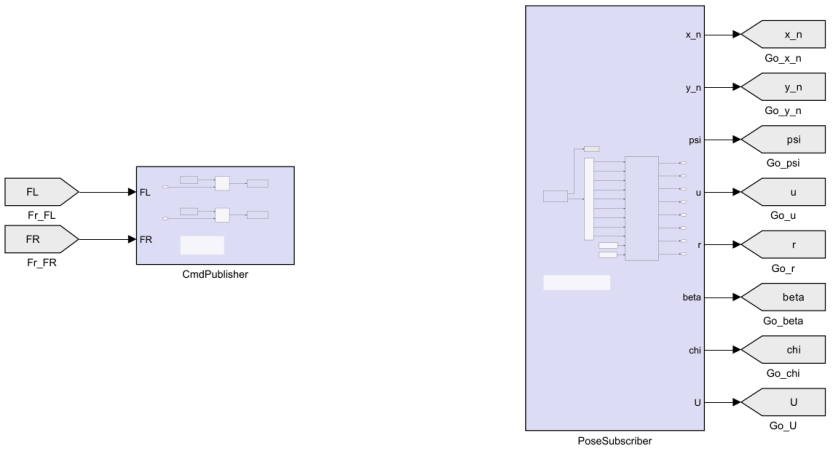
동기 점검 k — 로그만으로 "Simulink 가 VRX 를 따라갔는가" 를 판정함

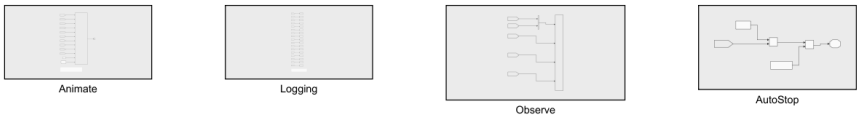
$$k = \frac{\text{지나간 거리 } \sum|\Delta \mathbf{p}|}{\text{속도 적분 } \int |u|\, dt_{\text{모델}}}$$

- VRX 가 주는 u 는 "시뮬레이터 1 초에 몇 m" 임. Simulink 가 제때 돌았으면 모델 시각으로 적분한 거리와 실제로 지나간 거리가 같음 → $k = 1$
- Simulink 가 늦어지면 모델 1 초 동안 배가 더 가므로 $k > 1$ 이 됨

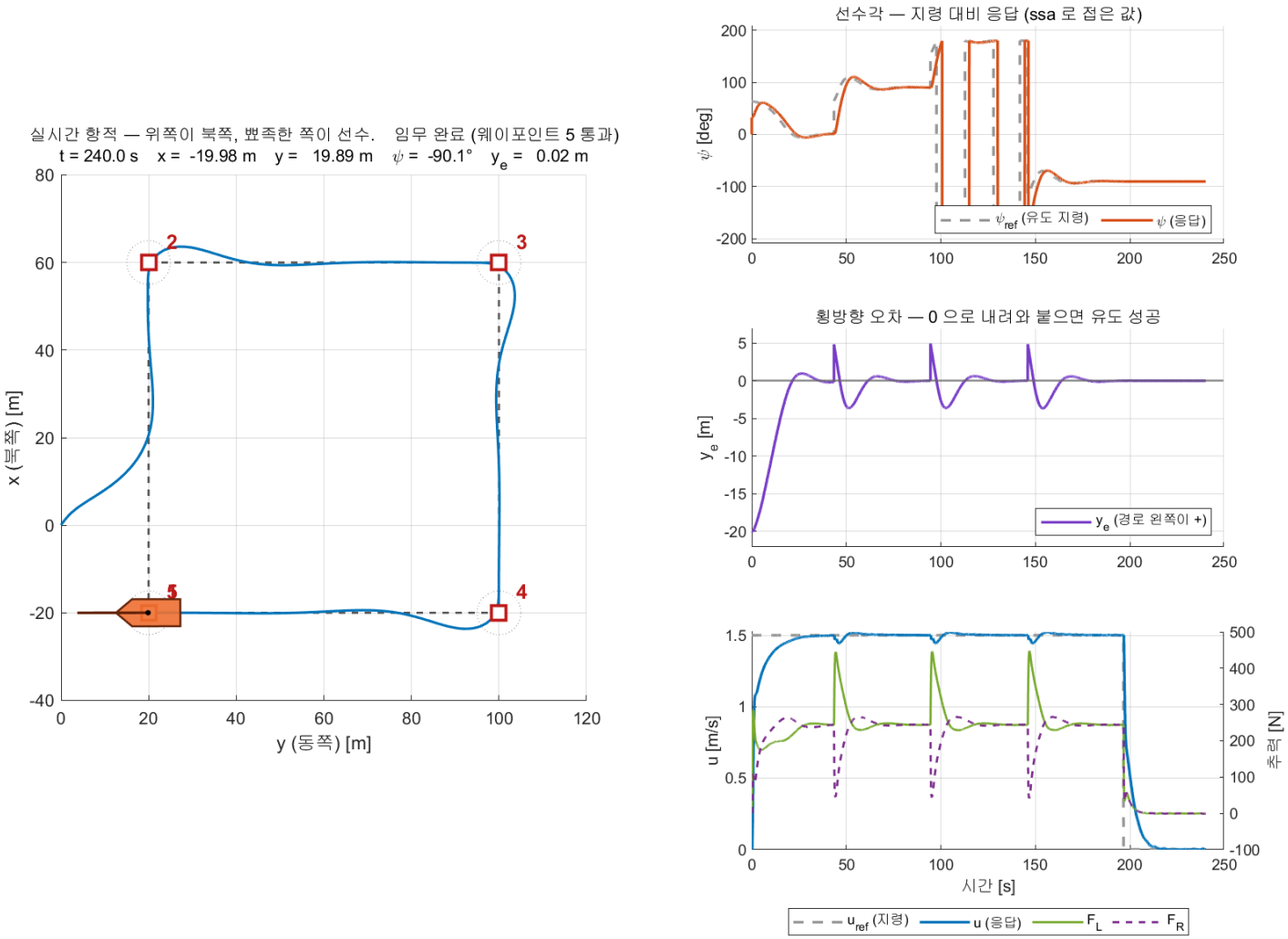
조건 (기준 노트북, 2026-10-16)	k	완주 [s]	판정
실시간 화면 끔 (기본)	**1.003**	232.7	동기 정상
실시간 화면 켬	**1.692**	140.6	쓰지 않음


W05 실습 E — VRX 연동 (Gazebo 필요)
Guidance -> Inner Loop -> Thrusters -> Gazebo -> Navigation
유도부·내부루프·추진기·게인은 W05_0_offline 과 완전히 같다.
운동모델만 Gazebo 로 바꾼다.
실행 전: (1) W05_setup (2) VRX 기동 (3) 페이싱을 RTF 에 맞출 것
psi_ref
Go_psi_ref
x_n
Fr_x_n
y_n
Fr_y_n
Guidance
y_e
Go_y_e
x_e
Go_x_e
wp_idx
Go_wp_idx
gate
Go_gate
psi
Fr_psi
r
Fr_r
u
Fr_u
InnerLoop
Thrusters
FL
Go_FL
FR
Go_FR
nprop
Go_nprop
Ferr
Go_Ferr


FL
Fr_FL
FR
Fr_FR
CmdPublisher
PoseSubscriber
x_n
Go_x_n
y_n
Go_y_n
psi
Go_psi
u
Go_u
r
Go_r
beta
Go_beta
chi
Go_chi
U
Go_U


Animate
Logging
Observe
AutoStop

실시간 화면 — VRX



- 조건 — `W05_1_vrx`, **3주차 경량 모델**(`W03_vrx_lite/run_vrx.sh lite`), `ROS_DOMAIN_ID=7`, 스폰 ENU $(-532.3,\ 200.0)$, 페이싱 1.072(= MATLAB 이 20초 동안 잰 RTF), 240 s 주행에 벽시계 241.7 s, 유도·게인은 위 오프라인과 동일 (2026-09-24 실측)
- 경량 모델의 Gazebo RTF 는 **0.989** 였음 (`W03_vrx_lite/measure_vrx.sh 60`, 60초, 표준편차 0.001, 화면 49.5 fps)
- 읽는 법은 위 A절의 네 칸 표와 **완전히 같음.** 같은 함수가 그린 같은 배치이기 때문임

볼 곳	오프라인	VRX	뜻
완주 시각 (③ 의 지령이 0 으로 떨어지는 시점)	약 207 s	약 197 s	초기 선수각이 달라 첫 구간에서 벌어짐
② 의 정착 후 y_e	0 부근	0 부근	**유도 성능은 시뮬레이터가 바뀌어도 같음**
③ 의 정상상태 u	1.50 m/s	1.50 m/s	추력·항력 계수가 같다는 증거

> **ℹ 두 화면의 ② 가 거의 겹침**
> 왼쪽 항적은 눈으로는 구분이 안 됨. **숫자로 대조하는 것은 E-6** 이며, 이 화면은 "도는 동안 무엇을 보고 있어야 하는가" 를 위한 것임

- 손으로 하고 싶으면 (수업에서 한 번은 해 볼 것). 비율은 E-3 에서 잰 RTF, 원점은 경로 원점(월드 고정점)

```
animate = 0;                                   % 실시간 화면 끔 (위 경고 상자)
origin_north = path_north0;   origin_east = path_east0;   % 경로 원점 = 월드 (200, -532). 스폰이 아님
set_param('W05_1_vrx','EnablePacing','on','PacingRate','0.70','StopTime','270');
out = sim('W05_1_vrx');
W05_plot(out, 'VRX / LOS')
```

1. Run 옆 화살표 → **Simulation Pacing** → Enable 체크

2. **비율을 RTF 값으로** 설정 ← 4주차와 다른 점
3. 그대로 Run — **정지 시간을 손으로 맞출 필요 없음**

> ℹ **마지막 웨이포인트를 지나면 스스로 멈춤 (2026-10-02)**
>
> 전에는 VRX 모델의 정지 시간이 `inf` 였음. Run 을 누르면 끝나지 않아 Stop 을 손으로 눌러야 했고, 언제 눌러야 하는지 알려 주는 것이 없었음.
>
> 이제 `AutoStop` 상자가 임무가 끝난 것을 보고 Stop 을 누름. 무엇을 보는가 — 유도부가 내는 `gate` 임.
>
> | 값 | 뜻 |
> |---|---|
> | `gate = 1` | 아직 갈 구간이 남음 |
> | `gate = 0` | 마지막 구간을 지남 — `wp_idx` 가 n 으로 **빗장이 걸림** |
>
> - `gate` 가 0 이 된 뒤 `stop_hold` (기본 10 s) 만큼 더 돌고 끝냄. 도착해서 멈추는 모습이 그래프에 남아야 읽을 수 있기 때문임
> - 버틴 시간은 **연속 적분기** 하나로 셈 (입력 1, 외부 리셋 = `gate`). 외부 리셋은 `level` 이어야 함 — `rising` 은 신호가 올라가는 순간만 리셋하므로, 처음부터 `gate = 1` 이면 올라가는 모서리가 없어 리셋이 영영 안 걸림
> - `T_run` (기본 600 s) 은 그래도 안 끝날 때를 받치는 안전장치임
>
> **실측 (2026-10-15, 오프라인 모델)** — 임무 완료 206.1 s → 10.0 s 더 돌고 **216.1 s 에 종료**. 전에는 600 s 까지 돌았음

E-5. 결과 확인

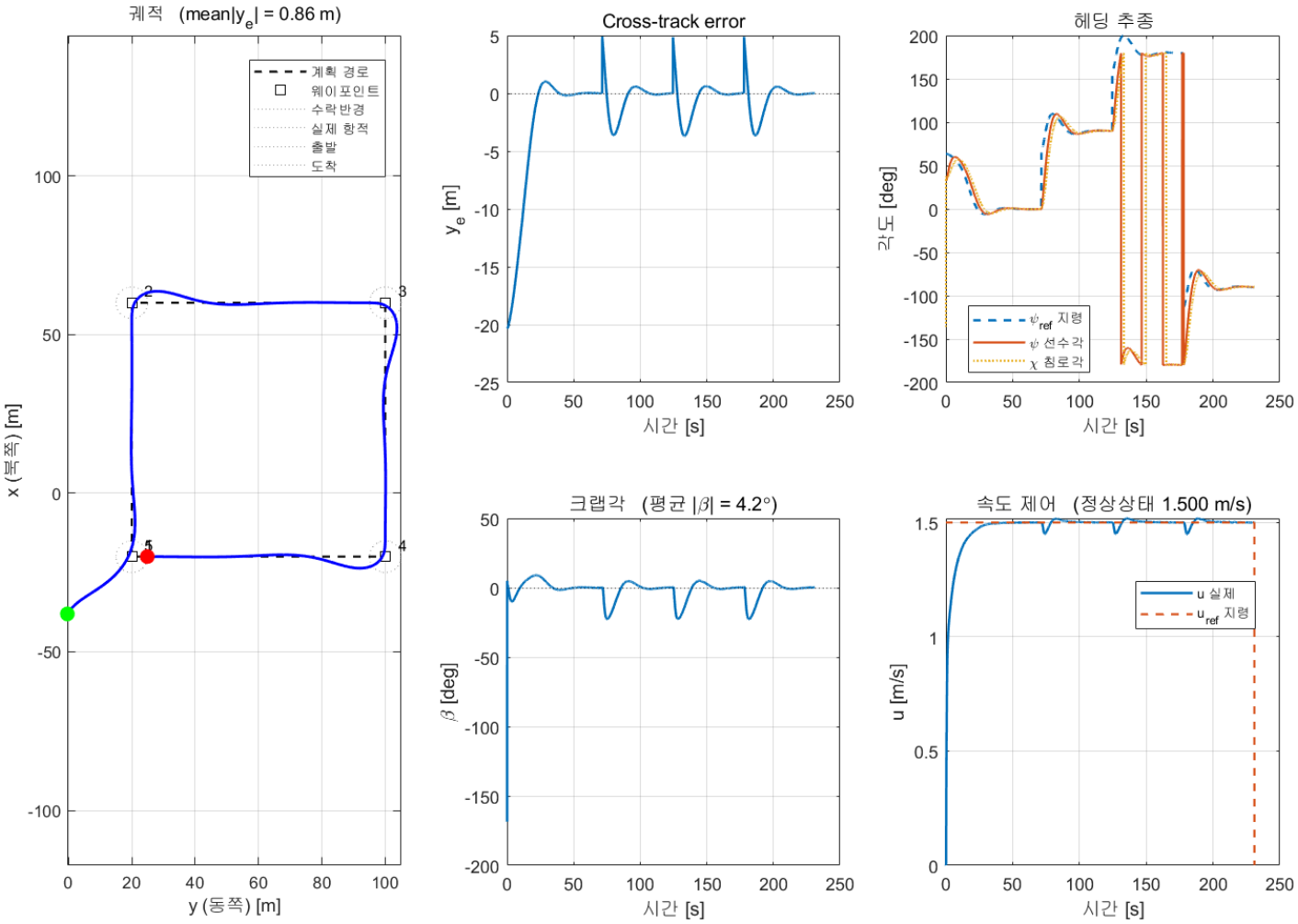
- 기준 환경 실측값 (2026-10-16, 기준 노트북, GitHub 에서 새로 `git clone` 한 강의자료, VRX 2.4.0-2 원본 그대로 — 스폰 ENU $(-532,\ 162)$, E-1 의 순서대로 VRX 를 새로 띄운 뒤 실행)
- 추진기 단을 고친 뒤(2026-10-15, §2-5)의 모델로 잰 값임. 아래 콘솔은 1회째 실행을 그대로 옮긴 것임

```
0) ROS_DOMAIN_ID = 7
1) VRX 토픽 확인
   OK — 토픽 20개
2) RTF 측정 (20초)
   스폰 위치 (x, y) = (162.0, -532.1) m
   경로 원점 (x, y) = (200.0, -532.0) m — 배는 경로 좌표 (-38.0, -0.1) 에서 출발
   (스폰이 경로 원점과 다른 설치임. 오프라인 대조도 같은 출발점에서 돌린다)
   RTF = 0.987  ->  페이싱 비율을 이 값으로 둔다
3) W05_1_vrx 실행 (270초)
4) 결과 그림

===== VRX / LOS 유도 =====
  평균 |y_e|              1.735 m   (전 구간)
  평균 |y_e|              0.856 m   (초기 60초 제외)  <- 과제에 쓸 값
  최대 |y_e|             20.260 m
  완주 시각               232.7 s
  평균 |크랩각|            4.14 deg
  정상상태 속도          1.500 m/s  (목표 1.50)
  왕복 검산 max|F_err| 1.14e-13 N    (F -> n -> F, 0 이어야 한다)
   그림 저장: img/W05_1_vrx_result.png
   동기 점검 k = 지나간 거리 349.6 m / 속도 적분 348.7 m = 1.003  (1.08 이하 — 동기 정상)
5) 같은 초기 선수각으로 오프라인 모델 실행
   VRX 초기 선수각 32.70 deg
```

- 이어서 오프라인 지표 7줄과 대조 결과(E-6)가 출력됨
- 토픽 개수는 VRX 판·센서 설정에 따라 다름 (경량 모델 20개, 기준 데스크톱 37 · 39개). 개수보다 **필요한 토픽이 있는지**가 중요함

읽는 법	
`경로 원점 ... (-38.0, -0.1) 에서 출발`	스폰 162 설치. 배가 경로 원점보다 38 m 남쪽에서 출발해 첫 구간으로 들어감 (E-4 경고)
완주 시각 232.7 s	오프라인 231.7 s 와 1.0 s 차이. 출발점이 38 m 뒤라 데스크톱(약 206 s)보다 길어짐
정착 후 $\lvert y_e \rvert$ 0.856 m	오프라인 0.840 m. 유도 성능은 시뮬레이터가 바뀌어도 같음
정상상태 속도 1.500 m/s	추력·항력 계수가 같다는 증거
최대 $\lvert y_e \rvert$ 20 m	첫 웨이포인트가 출발선에서 20 m 떨어져 있어서 나오는 값
평균 크랩각 4.14°	오프라인 3.43° 보다 큼. VRX 선체는 선회 중 옆으로 조금 더 미끄러짐
동기 점검 k = 1.003	Simulink 가 VRX 를 제때 따라감. 이 실행의 숫자를 써도 됨

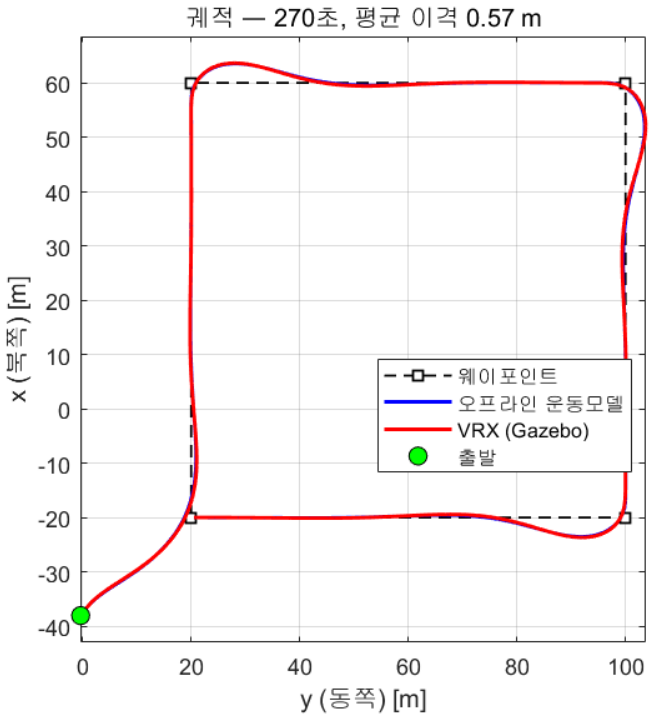


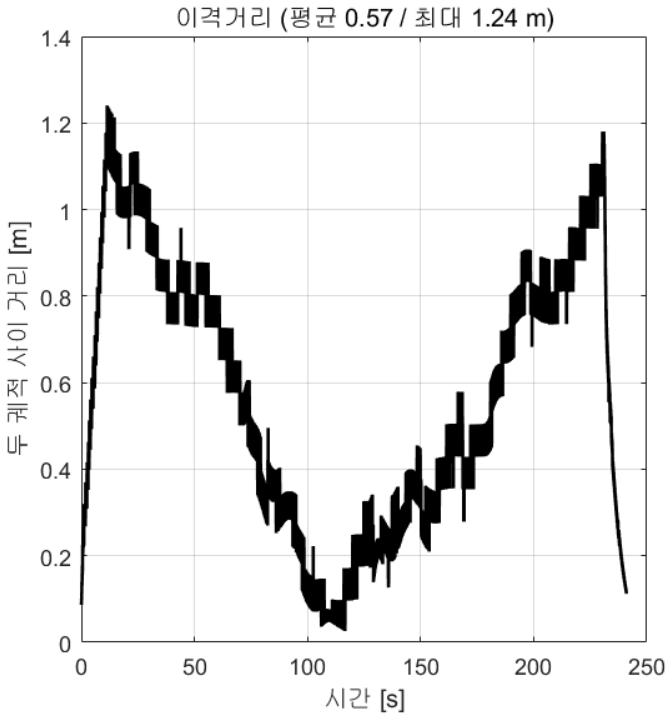
> **★ 주행거리 ÷ 시간 ≈ `u` 평균이 되어야 함**
>
> 이 둘이 2배쯤 차이 나면 **페이싱 비율이 RTF 와 맞지 않았거나 Simulink 가 늦은 것**임. 4단계의 동기 점검 k 가 그것을 숫자 하나로 알려 줌

> **⚠ 첫 샘플은 사용하지 않음**
>
> Subscribe 블록은 **첫 메시지가 도착하기 전에는 0 으로 채운 버스**를 내보냄
> 그대로 좌표 변환에 넣으면 위치가 스폰 원점만큼(약 570 m) 튐
> `PoseSubscriber` 서브시스템 안의 `Nav` 함수 첫머리에 `if ex == 0 && ey == 0` 가드가 들어 있는 이유임
> **구독한 값을 쓰기 전에 유효한지 확인하는 것** — ROS 를 쓰는 내내 필요한 습관임

E-6. 오프라인 모델과 비교 — 본 절의 결론

- `W05_vrx_run` 이 자동으로 그려 줌. 아래 그림은 2회째 실행임





그림의 위치	읽는 법
왼쪽 초록 점 $(-38,\ 0)$	스폰 자리 = 출발점. 경로 원점 $(0,\ 0)$ 보다 38 m 남쪽
왼쪽 빨강(VRX) · 파랑(오프라인)	처음부터 끝까지 **겹침** — 같은 출발점·같은 초기 선수각에서 돌렸기 때문임
오른쪽 이격거리	1 m 안팎에서 오르내리다 임무 끝에서 0 으로 떨어짐 → 경로가 아니라 **시각이 조금 어긋난 것**(위상 오차)

- 기준 환경 실측값 (2026-10-16, 기준 노트북, VRX 를 새로 띄워 두 번)

항목	오프라인 1회	VRX 1회	오프라인 2회	VRX 2회
측정 RTF	—	0.987	—	0.977
주행거리 [m]	348.8	349.6	349.0	349.6
완주 시각 [s]	231.7	232.7	231.8	231.0
60초 이후 평균 $\lvert y_e \rvert$ [m]	0.840	0.856	0.839	0.861
정상상태 `u` [m/s]	**1.500**	**1.500**	**1.500**	**1.500**
동기 점검 k	—	1.003	—	1.010
궤적 평균 / 최대 이격 [m]		1.49 / 1.80		0.57 / 1.24

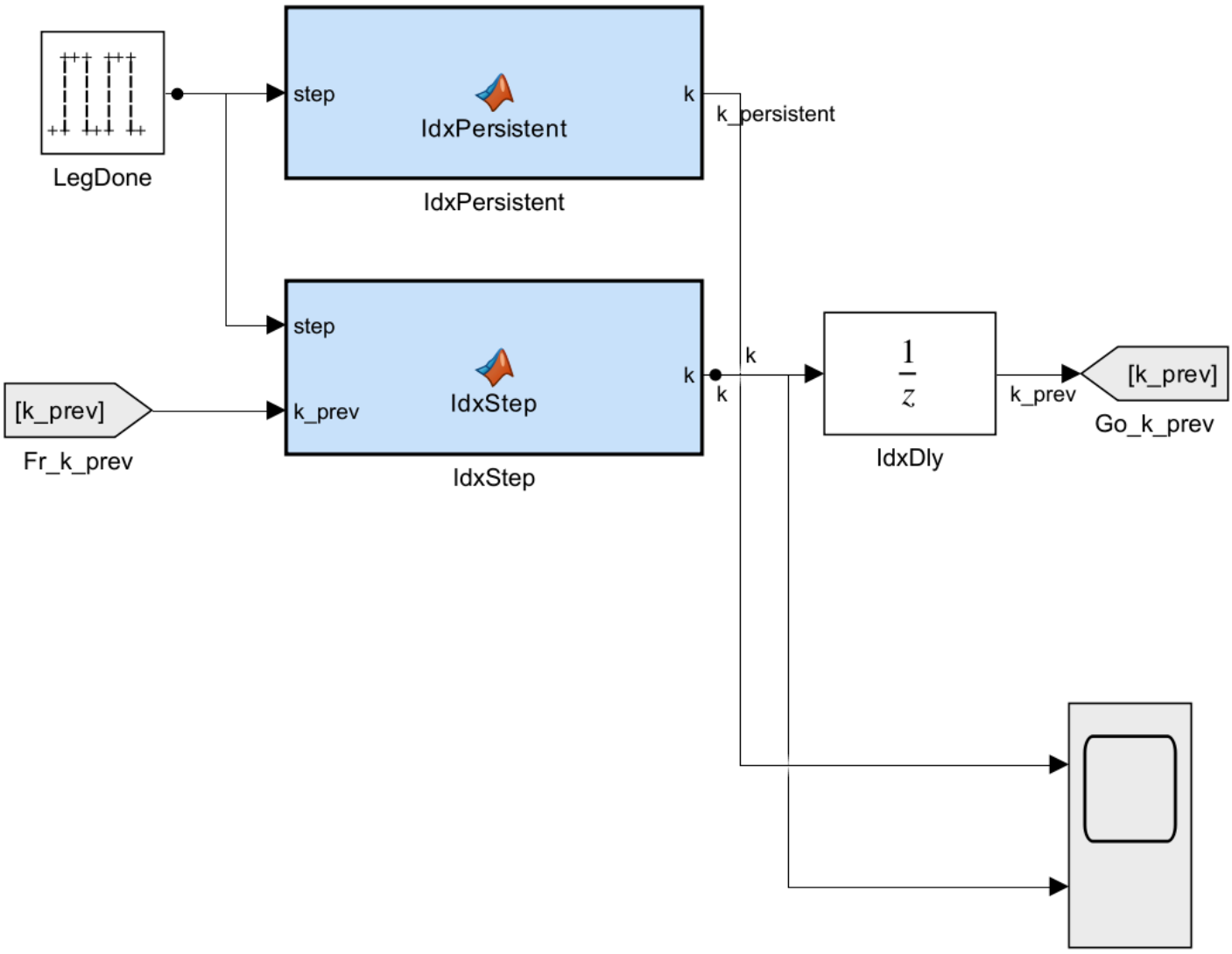
- 2026-10-23 3회째 (`wsl --shutdown` 으로 화면을 고친 뒤 같은 노트북): 완주 232.2 / 231.7 s, 정착 후 $\lvert y_e vert$ 0.858 / 0.839 m, @C431@ 1.500 / 1.500 m/s, k = 1.005, 평균 / 최대 이격 0.65 / 1.15 m (VRX / 오프라인 순)
- 오프라인 값이 두 번 조금 다른 까닭: 오프라인은 매번 **그 실행의 VRX 초기 선수각**으로 출발함. 선수각이 0.01° 단위로 달라짐
- 참고 — 기준 데스크톱(스폰 200, 2026-09-19, 추진기 단을 고치기 전)은 같은 표가 완주 205.9 / 206.7 s, 정착 후 $\lvert y_e \rvert$ 0.784 / 0.702 m, `u` 1.501 / 1.500 m/s 였음. 출발점이 $(0,\ 0)$ 이라 완주가 26 s 짧음

> **★ 이격 값 하나가 아니라, 정착 후 $\lvert y_e \rvert$ (0.856 · 0.861 vs 0.840 · 0.839) 와 정상상태 속도 (1.500 vs 1.500) 가 같다는 것이 결론**
>
> - 궤적 평균 이격은 1.49 · 0.57 m 로 실행마다 달라짐 (데스크톱에서는 0.28 · 2.59 · 0.95 m). 페이싱 비율(측정 RTF)과 실제 진행이 어긋난 만큼 같은 길을 다른 시각에 지나기 때문 → 위상 오차
> - 게인을 하나도 바꾸지 않았음. 오프라인에서 맞춘 것이 실제 시뮬레이터에서도 통한다는 뜻
> - **이것이 이번 주차 실습의 결론임** → 오프라인-모델이-시뮬레이터와-같아야-게인이-옮겨간다

> ⚠ **이격이 몇 m 로 커지면 먼저 세 가지를 의심함**
>
> - **동기**: 4단계의 k 가 1.08 을 넘으면 Simulink 가 늦은 것임. 두 선은 겹치는데 VRX 쪽이 **훨씬 빨리** 끝남 (E-4 경고)
> - **원점**: 2단계의 `경로 원점` 이 `(200.0, -532.0)` 이 아니면 경로가 밀림. 궤적 그림에서 두 선이 **평행하게 떨어져** 있음
> - **페이싱**: 비율이 실제 진행과 어긋나면 같은 길을 다른 시각에 지남. 두 선은 **겹치는데** 이격이 시간에 **직선으로** 커짐 → 위상(시간) 오차
> - 판정 기준은 이격이 아니라 **정착 후 $|y_e|$ 와 정상상태 속도**임 → 궤적이-겹치면-이격은-오차가-아니라-지연이다

F. 실습 4 — 웨이포인트 번호를 어디에 기억할 것인가 (15분)

W05 실습 F — 웨이포인트 번호를 어디에 기억할 것인가
위: persistent (상태가 코드 안에 숨는다)
아래: Unit Delay (상태가 도면에 보이고 Scope 에 걸린다)
두 선이 겹친다 — 답은 같고 다른 것은 상태를 어디에 두느냐다.
실행 전 >> W05_setup



유도법칙은 "지금 몇 번째 구간을 달리는가" 를 기억해야 함. 그 번호 k 를 어디에 둘 것인가가 이 절의 질문임.

두 가지 방법

`W05_matlab/LOSchi.m` 과 `Simple_guidance.m` 은 **MATLAB 함수**라 `persistent` 에 둠. 함수를 거듭 부르는 동안 값이 살아 있어야 하는데, 함수에는 그것을 둘 자리가 그곳뿐이기 때문임.

```
persistent k;   % 함수가 혼자 기억한다
```

Simulink 로 옮기면 선택지가 하나 더 생김 — **Unit Delay 블록**임. 함수는 기억하지 않고, **한 스텝 전의 값을 입력으로 받아** 다음 값을 돌려주기만 함.

```
function k = IdxStep(step, k_prev)   % 순수 함수 — 기억은 밖의 Unit Delay 가 맡는다
```

무엇이 다른가

	`persistent`	Unit Delay
상태가 보이는가	아니오 — 코드 안에 숨음	도면에 **블록으로 보임**
초기값	첫 호출 때 코드로	블록 파라미터로
되돌리기	`clear 함수이름`	Run 하면 저절로
기록·관찰	밖에서 못 봄	신호라서 Scope·로깅에 걸림
대수 루프	막지 못함	**끊어 줌**

마지막 줄이 Simulink 에서 결정적임. 유도부가 낸 번호가 유도부로 되돌아오므로 그대로 두면 대수 루프가 되고, Simulink 가 풀지 못함. Unit Delay 가 그 고리를 끊음.

> **★ 본 모델은 이미 Unit Delay 쪽임**
>
> `W05_0_offline` · `W05_1_vrx` 의 `Guidance` 안에 `IdxDly` 가 그것임. 들어가는 신호 이름은 `wp_idx_prev` — **한 스텝 전의** 번호라는 뜻임

해 볼 것

```
W05_setup
open_system('W05_2_idx_memory')   % Run 하면 Scope 에 두 선이 겹쳐 보인다
```

- 두 선이 겹침 — **답은 같음.** 다른 것은 상태를 어디에 두느냐임
- `IdxDly` 의 초기값을 `1` 에서 `3` 으로 바꿔 보기. 아래 선만 3 에서 시작함
- `clear IdxPersistent` 없이 두 번 연속 Run 해 보기. Simulink 는 실행마다 상태를 되돌리므로 두 방식 모두 1 에서 다시 시작함 — 같은 코드를 **MATLAB 에서** 직접 부르면 그렇지 않음

G. 모델을 다시 만들어야 할 때

```
W05_setup
build_w05_models
```

- 두 모델이 새로 생성됨
- 스크립트를 읽으면 각 블록이 어떻게 만들어지는지 알 수 있음

마무리

이번 주차 요약

순서	한 일	확인 방법
1	설정 파일 이해	`W05_setup` 출력
2	오프라인 모델 구조 읽기	4단 GNC 순서 설명 가능
3	atan2 vs LOS 비교	`W05_compare` 표
4	조류 실험	크랩각 2.95° → 8.06° (LOS, 조류 0.4 m/s)
5	Δ · R · 전환 판정 스윕	Δ = 10 m 에서 정착 후 |y_e| 최저 0.724 m
5-2	수락 원 판정 걸리게 하기	R = 2 m · 조류 0.4 m/s 에서 미완주
6	VRX 연동	20 m 벗어난 곳에서 경로 복귀
7	두 모델 대조	정착 후 |y_e| 차이 약 0.08 m, 세 번 실행 모두 같음 (이격은 페이싱에 좌우됨)

수업 진도 체크

> ★ 다음 주 수업을 따라가기 위한 최소 조건

이론 이해

- [ ] 유도 · 항법 · 제어를 각각 한 문장으로 구분해 말할 수 있음
- [ ] **선수각 ψ, 침로각 χ, 크랩각 β** 의 관계 $\chi = \psi + \beta$ 를 설명할 수 있음
- [ ] 해류 삼각형 $\boldsymbol{U} = \boldsymbol{U}_r + \boldsymbol{U}_c$ 를 그리고 **크랩각 β 와 횡류각 β_r** 이 어디가 다른지 말할 수 있음
- [ ] $\hat{t} = (\cos\pi_p,\ \sin\pi_p)$, $\hat{n} = (-\sin\pi_p,\ \cos\pi_p)$ 를 **방위각의 정의에서** 끌어낼 수 있음
- [ ] 두 오차 식이 $\boldsymbol{R}(\pi_p)^{\top}(\boldsymbol{p} - \boldsymbol{p}_k)$ 라는 **회전 하나**임을 설명할 수 있음
- [ ] **두 오차** `x_e` 와 `y_e` 가 각각 무엇을 재고 어디에 쓰이는지 앎
- [ ] `y_e` 가 **경로 좌표계**의 값이지 동쪽 방향 오차가 아님을 앎
- [ ] 구간 길이 $d = \|\boldsymbol{p}_{k+1} - \boldsymbol{p}_k\|$ 를 계산할 수 있고, 본 과목 미션에서 80 m 임을 확인했음
- [ ] LOS 법칙 $\psi_{\text{ref}} = \pi_p - \arctan(y_e/\Delta)$ 의 각 항이 무엇인지 앎
- [ ] Δ 를 키우면 무엇이 달라지는지 말할 수 있음
- [ ] 수락반경 R 이 가장 짧은 구간보다 커지면 안 되는 이유를 앎
- [ ] **전환 판정 두 가지**를 쓰고, 수락 원이 걸릴 수 있는 이유를 설명할 수 있음
- [ ] 헤딩 제어에서 **오차를 미분하지 않고** `r` **을 되먹임하는** 이유를 앎
- [ ] 속도 되먹임에 `U` 가 아니라 `u` 를 쓰는 이유를 앎
- [ ] VRX 에 **해류가 없다**는 것과, 그래서 조류 실험을 어디서 하는지 앎

실습 완료

- [ ] `W05_setup` 실행만으로 시뮬레이션이 돌고 창 3개가 떴음
- [ ] **실시간 화면** 네 칸의 뜻을 알고, ② 의 y_e 가 0 으로 붙는 것을 눈으로 확인했음
- [ ] 오프라인 모델을 LOS 로 실행하고 궤적을 확인했음
- [ ] `guidance_mode = 1` 로 바꿔 atan2 궤적을 확인했음
- [ ] `W05_compare` 로 4조건 표를 얻었음
- [ ] **조류를 켜서** 크랩각이 커지는 것을 확인했음
- [ ] Δ 를 2 / 3 / 5 / 10 / 20 / 30 으로 바꿔 궤적 변화를 관찰했음
- [ ] `bash stop_vrx.sh` 가 `0 개` 를 찍은 뒤 `bash run_vrx.sh` 로 VRX 를 띄우고, 터미널 2 에서 `ground_truth_odometry` 와 RTF 0.9 이상을 확인했음
- [ ] VRX 에서 **RTF 를 측정**했음

- [ ] 페이싱 비율을 RTF 에 맞추고 VRX 모델을 실행했음
- [ ] VRX 에서 배가 경로에 복귀하는 것을 확인했음
- [ ] 주행거리 ÷ 시간 이 `u` 평균과 맞는 것을 확인했음
- [ ] `W05_vrx_run` 4단계의 **동기 점검 k 가 1.08 이하**이고, 2단계에 `경로 원점 (x, y) = (200.0, -532.0)` 이 찍힌 것을 확인했음

관찰 기록

- [ ] atan2 와 LOS 의 궤적 모양 차이를 그림으로 남겼음
- [ ] 조류 유무에 따른 `mean|y_e|` 를 표로 남겼음
- [ ] 오프라인과 VRX 궤적을 겹쳐 그려 비교했음

과제 5 — 유도법칙 성능 비교

- **제출 기한**: 6주차 수업 전
- **제출물**: 그래프 + 성능표 + 분석

① atan2 vs LOS (조류 유/무)

- `W05_compare` 결과를 표로 제출

조건	평균 |y_e| [m]	정착 후 |y_e| [m]	완주 시간 [s]	평균 크랩각 [deg]
atan2 / 조류 없음				
LOS / 조류 없음				
atan2 / 조류 0.4				
LOS / 조류 0.4				

- 궤적 4개를 **한 그림에 겹쳐** 제출

② lookahead distance 스윕

- Δ = 2, 3, 5, 10, 20, 30 m (LOS). **조류 없음과 조류 0.4 m/s(90°) 두 번**

조류 없음

Δ [m]	평균 |y_e| [m]	정착 후 |y_e| [m]	완주 시간 [s]	궤적 특징
2				
3				
5				
10				
20				
30				

조류 0.4 m/s (90°)

Δ [m]	평균 |y_e| [m]	정착 후 |y_e| [m]	완주 시간 [s]	궤적 특징
2				
3				
5				
10				
20				
30				

- **각 조건에서 가장 좋은 Δ 를 고르고 근거를 쓸 것**

- **두 조건의 최적 Δ 가 같은가 다른가.** 다르다면 왜 그런지 쓸 것

③ 조류 방향 3종

- 조류 0.4 m/s, 방향 0° / 90° / 180° (LOS) — ① · ② 와 §1-7 표와 같은 세기로 두어 결과를 바로 견줄 수 있게 함
- `current_direction` 은 조류가 **흘러가는** 방위 (북 = 0°, 시계방향)
- 사각형 미션에서는 한 방향의 조류가 구간마다 역조류 · 옆조류 · 순조류로 바뀜
  - 예: 0° (북쪽으로 흐름) 는 첫 구간(북향)에서는 **순조류**, 셋째 구간(남향)에서는 **역조류**
- **평균 크랩각**과 **평균 |y_e|** 를 측정

조류 방향	평균 크랩각 [deg]	평균 |y_e| [m]
0° (북쪽으로 흐름)		
90° (동쪽으로 흐름)		
180° (남쪽으로 흐름)		

- 표 아래에 각 방향이 **어느 구간에서 옆조류인지** 적고, 그것으로 결과 차이를 설명할 것

④ 오프라인 ↔ VRX 대조

- `W05_vrx_run` 기본값(270 s, 임무 완주, 실시간 화면 끔)으로 실행 — 오프라인 대조까지 자동으로 함. 4단계의 동기 점검 k 도 함께 적을 것 (1.08 을 넘은 실행은 쓰지 않음)
- 궤적을 겹쳐 그리고 **평균 이격 거리**를 수치로 제시
- 게인은 **바꾸지 말 것**

⑤ 분석 (8~12줄)

1. 조류가 없을 때도 LOS 가 나은 이유를 `y_e` 를 써서 설명하시오
2. 조류에서 atan2 가 크게 성능이 저하되는 이유를 크랩각으로 설명하시오
3. LOS 도 조류에서 오차가 남음. 왜 그런가?
4. 오프라인 모델과 VRX 가 이만큼 잘 맞는 이유는 무엇인가?

평가 기준

항목	배점
네 조건 실행 및 표 완성	20%
성능 수치의 정확성 (그래프에서 제대로 읽었는가)	30%
오프라인 ↔ VRX 대조 실험	25%
분석의 타당성	25%

막혔을 때

증상	원인	해결
`먼저 W05_setup 을 실행하십시오`	워크스페이스에 변수 없음	`W05_setup` 먼저
모델은 도는데 배가 안 움직임	페이싱 미설정	Simulation Pacing 켜기
주행거리와 `u` 가 2배 차이	**페이싱 비율 ≠ RTF**	RTF 측정 후 그 값으로 설정
배가 갑자기 멈춤	육지에 얹힘	웨이포인트를 $x \in [-28, 60]$, $y \in [-18, 100]$ 안으로 (경로 원점 = 월드 $(200,\ -532)$ 기준)
VRX 에서 4번 웨이포인트 부근 해안에 얹힘 · 속도 0 · 미완주	옛 `W05_vrx_run` 이 스폰(162)을 경로 원점으로 삼음	`git pull` — 2단계에 `경로 원점 (x, y) = (200.0, -532.0)` 이 찍혀야 함 (E-4 경고)
동기 점검 k > 1.08 · 메시지 손실 경고가 수십 건	Simulink 가 VRX 를 못 따라감	`W05_vrx_run` 기본값(화면 끔)으로, 무거운 창을 닫고 VRX 를 새로 띄워 재실행
동기 점검 k < 0.9 · 추력은 나가는데 배가 몇 m 만 감	ROS 2 데몬이 깨져 명령이 Gazebo 까지 안 감	WSL 에서 `ros2 daemon stop && ros2 daemon start` → VRX 를 새로 띄워 재실행
배가 두 척 보임 · 토픽 값이 튐 · RTF 가 갑자기 반	앞서 띄운 VRX 가 고아로 남음	`bash stop_vrx.sh` → `끈 뒤: 0 개` 확인 후 다시 띄움. 안 되면 PowerShell 에서 `wsl --shutdown`
Gazebo 화면이 1 fps · RTF 0.001	카메라·LiDAR 를 켠 xacro 를 기본 엔진으로 띄움	E-1 의 `run_vrx.sh` 로 (E-2 경고 표)
`run_vrx.sh` 뒤 `[Wrn]` · `[Err]` 가 수백 줄	정상 경고	무시함 (E-1 4번의 표)
로그는 나오는데 Gazebo 창이 안 뜸 · `화면 이상: ... msrdc.exe`	WSLg 의 창 표시기가 재시작을 반복	WSL 터미널을 모두 닫고 PowerShell 에서 `wsl --shutdown` → 새 터미널에서 다시 (E-1 경고 상자)
배가 마지막 점을 지나 계속 감	`gate` 미연결	`build_w05_models` 로 재생성
웨이포인트를 다 건너뜀	R 이 웨이포인트 간격보다 큼	R 을 줄일 것
배가 한 바퀴 돌아 목표로 감	각도 wrap 누락	`HeadingErr` 의 `atan2(sin,cos)` 확인
헤딩이 반대로 돎	부호 오류	`PoseSubscriber` 안 `Nav` 함수의 `r = -wz` 확인
코너에서 크게 흔들림	Δ 가 너무 작음	Δ 를 키울 것
`y_e` 가 계속 큼	atan2 모드임	`guidance_mode = 2`
MATLAB 에서 토픽이 안 보임	Domain ID 불일치	4주차 2-14-5 참조

참고 자료

이 주차의 원본

- **Fossen, T. I.** *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed. (2021) §12.3 — `50-원본자료/` 에 PDF 있음
- **Fossen 2024 Lecture on 2D and 3D path-following control** — `50-원본자료/`, LOS 유도 전반
- **MSS 툴박스** — https://github.com/cybergalactic/MSS
  - `SIMULINK/mssSimulinkDemos/` — Otter USV 경로 추종 데모 (`demoOtterUSVPathFollowingHeadingControl.slx` · `demoOtterUSVPathFollowingCourseControl.slx`)
  - `GNC/crosstrackWpt.m` — 본 과목의 두 오차 식과 같은 회전 형태
  - `GNC/LOSchi.m` — 침로각 χ 지령을 내는 LOS. 전환 판정은 경로 방향$(d - x_e < R)$
  - `GNC/ILOSpsi.m` · `GNC/ALOSpsi.m` — 선수각 ψ 지령을 내는 적분 · 적응 LOS. 지령이 침로냐 선수각이냐의 차이는 §1-2
  - 변수 이름 대응은 §1-9 의 표 참고 (`pi_h` ↔ `pi_p`, `Delta_h` ↔ `Delta`, `R_switch` ↔ `R_LOS`)

교수 강의자료 — 이 주차가 그대로 따라간 두 자료

자료	이 주차의 어느 절
`2. USV 제어기 설계.pptx` 37~46쪽	§1-5 두 오차, §1-6 LOS 법칙, §1-9 수식↔코드 (46쪽이 코드 원본)
`2. USV 제어기 설계.pptx` 53~58쪽	§1-7 lookahead distance, 가변 Δ
`2. USV 제어기 설계.pptx` 59~71쪽	본 주차 범위 밖 — 이탈량 동역학, ILOS. 대학원 W04 §4-7·4-8
대학원 `W04_Waypoint_Following_and_LOS_Guidance`	§1-5~§1-8 전부. 유도 과정과 안정성까지 보려면 이쪽

> ℹ **기호를 두 자료와 맞춰 두었음**
>
> $\pi_p \cdot x_e \cdot y_e \cdot \Delta \cdot R$ 은 위 자료들이 쓰는 이름 그대로임
> 대학원 자료는 좌표계를 밝히려고 x_e^p, y_e^p 처럼 위첨자를 붙이는데,
> 본 과목은 경로 좌표계 외의 오차를 쓰지 않으므로 위첨자를 생략했음 (§1-9)

볼트 내 문서

- W04_Simulink_PID와_첫_제어기 — 내부루프, ground truth, 페이싱
- W09_라이다_장애물회피_파이썬유도와_시뮬링크제어 — 이 주차의 LOS 유도단에 회피 명령을 하나 더 붙임
- ENU와-NED를-섞으면-조용히-틀린다 — `PoseSubscriber` 안 `Nav` 함수의 좌표 변환
- 조류가-있으면-뱃머리와-진행방향이-다르다 — 크랩각
- 목표를-향해-가는-것과-경로를-따라가는-것은-다르다 — atan2 vs LOS

배포 파일

파일	내용
`W05_simulink/W05_setup.m`	**학생이 고치는 설정 파일. 실행하면 바로 시뮬레이션이 돎**
`W05_simulink/W05_animate.m`	실시간 화면 네 칸 그리기 — 오프라인·VRX 공용 (모델 안 `Animate` 블록이 호출)
`W05_simulink/W05_anim_capture.m`	그 화면을 조건과 함께 PNG 로 재현 (`img/W05_anim_offline.png` · `W05_anim_vrx.png`)
`W05_simulink/W05_0_offline.slx`	오프라인 WAM-V (조류 실험)
`W05_simulink/W05_1_vrx.slx`	VRX 연동
`W05_simulink/W05_plot.m`	성능 그림 5장 + 추진기 그림 2장 + 성능 지표
`W05_simulink/W05_sweep.m`	1-7 · 1-8 절 표 네 개를 다시 재는 스크립트 (오프라인, 1~2 분)
`W05_simulink/W05_compare.m`	4조건 비교 (과제 ①)
`W05_simulink/W05_aim_demo.m`	**점을 겨냥할 때와 선을 겨냥할 때** — 직선 경로에서 atan2 와 LOS 의 이탈량 비교 (1-4절)
`W05_simulink/W05_lookahead_demo.m`	**Δ 네 값의 경로 복귀 비교** (§1-7, 실습 D-1)
`W05_simulink/W05_crab_demo.m`	**조류 속 뱃머리 · 진행 방향 · 크랩각** — 정상 이탈이 $\Delta \tan\beta$ 로 맞는지 확인 (1-7절)
`W05_simulink/W05_vrx_run.m`	**VRX 연동 실행** — 토픽 확인 · RTF 측정 · 페이싱 · 오프라인 대조 (실습 E-4)
`W05_simulink/build_w05_models.m`	모델 재생성
`W05_simulink/tidy_layout.m`	블록 배치 정리
`W05_matlab/`	연구실 참고 코드 (읽기용). `LOSchi.m` 은 MSS 원본의 **연구실 수정본** — 전환 판정이 수락 원. MSS 원본은 https://github.com/cybergalactic/MSS 의 `GNC/`

다음 주 예고

- **6주차 — 선수추진기와 위치제어 접안**
- 추진기가 **둘에서 셋**으로 늘어남. 횡방향으로 힘을 낼 수 있게 되므로 "경로를 따라간다" 를 넘어 **정해진 위치에 정해진 선수각으로 서 있기**가 비로소 됨

- 할 일
  - **추력 배분** — 힘 셋($\boldsymbol{X}$ 전후 · $\boldsymbol{Y}$ 횡 · $\boldsymbol{N}$ 요)을 추진기 셋에 나눠 주기
  - 고리 **세 겹** — 위치 고리 · 속도 고리 · 선수각 고리
  - **접안 세 단계** — 접근 → 선수각 정렬 → 옆걸음 접안, 단계 전이 판정까지
  - 오차를 **어느 좌표계에서 셀 것인가** — 선체고정 축과 NED 축을 같은 과업으로 견줌
- 이번 주차에 작성한 추진기·운동모델·실시간 화면 틀은 **그대로 씀.** 추진기 한 대가 늘고, 안쪽 고리와 추력 배분이 커짐
- 쉬운 모델에서 복잡한 모델로 **여덟 단**을 하나씩 쌓음. 1단은 한 점에 가서 서 있기 하나뿐임
- 준비물
  - 과제 5 의 그래프와 표